



# École Centrale des Logiciels Libres et de Télécommunications

## Support Cours nodejs.

Présenté par :

**Ahamadi Nasry**

Sous la supervision du :

**Pr. Samuel Ouya**



**Centre d'Informatique et de Recherche de  
l'Armée et de la Sécurité (CIRAS)**



**Institut Supérieur de Technologie de  
l'Université de Bangui (IST/UB)**

# Objectifs du cours sur Node.js

Ce cours a pour objectif de fournir aux étudiants une compréhension approfondie de Node.js, en couvrant à la fois les concepts fondamentaux et les techniques avancées. L'approche pédagogique inclut des explications théoriques, des démonstrations pratiques, et des exercices pour permettre aux apprenants d'acquérir des compétences solides dans le développement backend avec Node.js.

## Objectifs pédagogiques

À l'issue de ce cours, les participants seront capables de :

### **Comprendre les bases de Node.js et son architecture**

- Expliquer ce qu'est Node.js et comment il fonctionne.
- Comprendre le moteur V8, l'Event Loop, et le modèle asynchrone non bloquant.

### **Installer et configurer un environnement Node.js**

- Installer Node.js avec NVM (Node Version Manager).
- Utiliser npm pour gérer les dépendances.
- Comprendre et manipuler le fichier package.json.

### **Développer des applications avec Node.js**

- Créer et exécuter des scripts avec Node.js.
- Travailler avec les modules intégrés (fs, path, http, events).
- Gérer les fichiers et les flux de données (lecture/écriture).

### **Construire des serveurs web avec Node.js**

- Créer un serveur HTTP avec le module http.
- Utiliser Express.js pour simplifier la gestion des routes et middlewares.
- Manipuler les requêtes et réponses HTTP.

### **Gérer les bases de données avec Node.js**

- Connecter Node.js à MySQL et MongoDB.
- Effectuer des requêtes SQL et NoSQL.
- Utiliser un ORM (Sequelize, Mongoose) pour gérer les données.

### **Créer des APIs RESTful et GraphQL**

- Développer une API CRUD complète avec Express.js.
- Ajouter l'authentification avec JWT (JSON Web Tokens).

- Documenter l'API avec Swagger (swagger-autogen, swagger-ui-express).

#### **Travailler avec les middlewares et la gestion des erreurs**

- Comprendre le rôle des middlewares dans Express.js.
- Gérer les erreurs et exceptions dans Node.js.
- Implémenter des middlewares personnalisés.

#### **Sécuriser une application Node.js**

- Gérer l'authentification et l'autorisation avec JWT et bcrypt.
- Mettre en place la validation des données avec Joi ou express-validator.
- Protéger une API contre les attaques courantes (CORS, CSRF, XSS).

## Prérequis

- ✚ Savoir coder en JavaScript (ES6+).
- ✚ Comprendre les bases des API et HTTP.
- ✚ Avoir des notions de bases de données (SQL ou NoSQL).
- ✚ Savoir utiliser la ligne de commande et installer des outils.
- ✚ Disposer des logiciels nécessaires (Node.js, VS Code, Postman.).



# Table des matières

|                                                                            |     |
|----------------------------------------------------------------------------|-----|
| Objectifs du cours sur Node.js .....                                       | i   |
| Objectifs pédagogiques .....                                               | i   |
| Prérequis.....                                                             | iii |
| Chapitre 1 : Introduction à Node.js .....                                  | 1   |
| 1.1    Présentation .....                                                  | 1   |
| 1.2    Qu'est-ce que Node.js ? .....                                       | 1   |
| 1.2.1    Définition .....                                                  | 1   |
| 1.2.2    Origine.....                                                      | 1   |
| 1.3    Caractéristiques principales .....                                  | 2   |
| 1.3.1    Exécution JavaScript côté serveur.....                            | 2   |
| 1.3.2    Modèle non bloquant.....                                          | 2   |
| 1.3.3    Utilisation du moteur V8.....                                     | 2   |
| 1.4    Avantages et Cas d'utilisation.....                                 | 2   |
| 1.4.1    Avantages de Node.js.....                                         | 3   |
| 1.4.2    Cas d'utilisation .....                                           | 3   |
| 1.5    Fonctionnement interne .....                                        | 3   |
| 1.5.1    Architecture de Node.js.....                                      | 3   |
| 1.6    Installation de Nodejs sur différents systèmes d'exploitation ..... | 4   |
| 1.6.1    Installation de Nodejs sous linux(ubuntu).....                    | 4   |
| 1.6.2    Installation de NodeJs sous Windows.....                          | 6   |
| 1.6.3    Vérification de la version de nodejs sous Windows. ....           | 13  |
| Chapitre 2 : Installation et Gestion des Versions avec NVM.....            | 14  |
| 2.1    Présentation de NVM .....                                           | 14  |
| 2.2    Qu'est-ce que NVM ?.....                                            | 14  |
| 2.3    Comment fonctionne NVM ? .....                                      | 14  |

|                                                                |                                                                  |    |
|----------------------------------------------------------------|------------------------------------------------------------------|----|
| 2.4                                                            | Avantages et Inconvénients de NVM.....                           | 14 |
| 2.4.1                                                          | Avantages .....                                                  | 15 |
| 2.4.2                                                          | Inconvénients .....                                              | 15 |
| 2.5                                                            | Avantages d'utiliser NVM pour l'installation de Node.js .....    | 15 |
| 2.6                                                            | Installation de NVM sur différents systèmes d'exploitation ..... | 15 |
| 2.6.1                                                          | Installation sur Linux (Ubuntu).....                             | 15 |
| 2.7                                                            | Liste de toutes les commandes de NVM .....                       | 16 |
| 2.8                                                            | Gestion des versions de Node.js avec NVM .....                   | 17 |
| 2.8.1                                                          | Commandes principales de NVM .....                               | 17 |
| 2.8.2                                                          | Installation de nvm sous Windows.....                            | 18 |
| Chapitre 3 : Les Fondamentaux de JavaScript pour Node.js ..... |                                                                  | 27 |
| 3.1                                                            | Présentation .....                                               | 27 |
| 3.2                                                            | Concepts de base JavaScript.....                                 | 27 |
| 3.2.1                                                          | Variables et Types de Données .....                              | 27 |
| 3.2.1.1                                                        | Définition des Variables .....                                   | 27 |
| 3.2.1.2                                                        | Déclaration de Variables .....                                   | 27 |
| 3.2.2                                                          | Types de Données Primitifs .....                                 | 27 |
| 3.2.2.1                                                        | Number .....                                                     | 27 |
| 3.2.2.2                                                        | String.....                                                      | 28 |
| 3.2.2.3                                                        | Boolean .....                                                    | 28 |
| 3.2.2.4                                                        | Null et Undefined.....                                           | 28 |
| 3.2.3                                                          | Types de Données Complexes .....                                 | 28 |
| 3.2.3.1                                                        | Object.....                                                      | 28 |
| 3.2.3.2                                                        | Array .....                                                      | 29 |
| 3.2.4                                                          | Fonctions et Portée .....                                        | 29 |
| 3.2.4.1                                                        | Définition des Fonctions .....                                   | 29 |
| 3.2.4.2                                                        | Types de Fonctions .....                                         | 29 |

|                                                  |                                                  |    |
|--------------------------------------------------|--------------------------------------------------|----|
| 3.2.4.3                                          | Déclaration de Fonction .....                    | 29 |
| 3.2.4.4                                          | Expression de Fonction .....                     | 30 |
| 3.2.4.5                                          | Fonction Fléchée .....                           | 30 |
| 3.2.4.6                                          | Portée (Scope) .....                             | 30 |
| 3.2.4.7                                          | Types de Portée .....                            | 30 |
| 3.3                                              | Spécificités JavaScript côté Serveur .....       | 31 |
| 3.3.1                                            | Différences entre JS Navigateur et Node.js ..... | 31 |
| 3.3.1.1                                          | Environnement d'Exécution .....                  | 31 |
| 3.3.1.2                                          | APIs Disponibles .....                           | 31 |
| 3.3.1.3                                          | Gestion des Modules .....                        | 32 |
| 3.3.2                                            | CommonJS vs ES Modules .....                     | 32 |
| 3.3.2.1                                          | CommonJS .....                                   | 32 |
| 3.3.2.2                                          | ES Modules .....                                 | 33 |
| 3.3.3                                            | Le Système de Modules Node.js .....              | 33 |
| 3.3.3.1                                          | Définition des Modules .....                     | 33 |
| 3.3.3.2                                          | Types de Modules .....                           | 33 |
| 3.3.3.3                                          | Résolution des Modules .....                     | 34 |
| 3.3.3.4                                          | Ordre de résolution : .....                      | 34 |
| Chapitre 4 : Le Système de Modules Node.js ..... |                                                  | 35 |
| 4.1                                              | Présentation .....                               | 35 |
| 4.2                                              | Modules Natifs (Core Modules) .....              | 35 |
| 4.2.1                                            | Qu'est-ce qu'un Module Natif ? .....             | 35 |
| 4.2.2                                            | Principaux Modules Natifs .....                  | 35 |
| 4.2.2.1                                          | Module fs (File System) .....                    | 35 |
| 4.2.2.2                                          | Module path .....                                | 36 |
| 4.2.2.3                                          | Module http/https .....                          | 37 |
| 4.2.3                                            | Utilisation des Modules Built-in .....           | 37 |

|              |                                                             |    |
|--------------|-------------------------------------------------------------|----|
| 4.2.3.1      | Bonnes Pratiques.....                                       | 37 |
| 4.3          | Gestion des Packages.....                                   | 38 |
| 4.3.1        | npm.....                                                    | 38 |
| 4.3.2        | Installation et Configuration.....                          | 39 |
| 4.3.3        | package.json et ses Composants.....                         | 39 |
| 4.3.4        | Structure de base .....                                     | 39 |
| 4.3.5        | Composants Principaux.....                                  | 40 |
| 4.3.5.1      | Métadonnées .....                                           | 40 |
| 4.3.5.2      | Dépendances .....                                           | 40 |
| 4.3.5.3      | Scripts .....                                               | 40 |
| 4.3.6        | Gestion des Dépendances.....                                | 40 |
| 4.3.6.1      | Versions et SemVer .....                                    | 40 |
| 4.3.7        | Commandes Importantes.....                                  | 41 |
| 4.3.8        | Création et Publication de Packages .....                   | 41 |
| 4.3.8.1      | Création d'un Package.....                                  | 41 |
| Chapitre 5 : | Gestionnaire de Paquets et Initialisation d'un Projet ..... | 42 |
| 5.1          | Introduction à npm.....                                     | 42 |
| 5.2          | Qu'est-ce qu'un gestionnaire de paquets ?.....              | 42 |
| 5.3          | npm (Node Package Manager) .....                            | 42 |
| 5.3.1        | Présentation .....                                          | 42 |
| 5.3.2        | Fonctionnalités Principales .....                           | 42 |
| 5.3.3        | Commandes essentielles de npm.....                          | 42 |
| 5.3.4        | Initialisation d'un projet : .....                          | 42 |
| 5.3.5        | Installation d'un paquet :.....                             | 43 |
| 5.3.6        | Pour lister les paquets installés : .....                   | 44 |
| 5.3.7        | Suppression un paquet :.....                                | 44 |
| 5.3.8        | Les scripts npm.....                                        | 45 |

|                                                |                                                                                                 |    |
|------------------------------------------------|-------------------------------------------------------------------------------------------------|----|
| 5.3.9                                          | Avantages de npm .....                                                                          | 45 |
| 5.3.10                                         | Inconvénients de npm.....                                                                       | 46 |
| 5.4                                            | Initialisation d'un Projet Node.js.....                                                         | 46 |
| 5.5                                            | Structure Complète d'un Fichier app.js Simplifié dans nodejs .....                              | 47 |
| 5.5.1                                          | Exemple de fichier app.js :.....                                                                | 47 |
| 5.5.2                                          | Explications Détail par Section .....                                                           | 48 |
| 5.5.3                                          | Exemple pratique : .....                                                                        | 49 |
| 5.5.4                                          | Décomposition de la commande .....                                                              | 50 |
| 5.5.5                                          | Où est installé Express.js ? .....                                                              | 50 |
| 5.5.6                                          | Exemple 2 :.....                                                                                | 51 |
| Chapitre 6 : Framework Express.js .....        |                                                                                                 | 54 |
| 6.1                                            | Présentation d'Express.js.....                                                                  | 54 |
| 6.2                                            | Définition.....                                                                                 | 54 |
| 6.3                                            | Fonctionnalités principales :.....                                                              | 54 |
| 6.4                                            | Pourquoi utiliser Express.js ? .....                                                            | 55 |
| 6.5                                            | Pourquoi Express.js est essentiel .....                                                         | 56 |
| Chapitre 7 : Les Middlewares dans Node.js..... |                                                                                                 | 57 |
| 7.1                                            | Concepts des Middlewares .....                                                                  | 57 |
| 7.1.1                                          | Définition et Rôle des Middlewares .....                                                        | 57 |
| 7.1.2                                          | Métaphore pour Comprendre un Middleware : Le Contrôle de Sécurité à l'Entrée d'un Bâtiment..... | 57 |
| 7.1.3                                          | Les Éléments de la Métaphore : .....                                                            | 57 |
| 7.1.4                                          | Comment ça Fonctionne ?.....                                                                    | 57 |
| 7.1.5                                          | Application au Code d'Exemple : .....                                                           | 58 |
| 7.1.6                                          | Structure de Base d'un Middleware : .....                                                       | 58 |
| 7.1.7                                          | Types de Middlewares .....                                                                      | 58 |
| 7.1.8                                          | Exemple : Middleware d'erreur : .....                                                           | 58 |

|                                             |                                                             |    |
|---------------------------------------------|-------------------------------------------------------------|----|
| 7.2                                         | Utilisation des Middlewares avec Express.js .....           | 59 |
| 7.2.1                                       | Middlewares Intégrés .....                                  | 59 |
| 7.2.2                                       | Exemple : Utilisation des Middlewares Intégrés : .....      | 59 |
| 7.2.3                                       | Middlewares Tiers.....                                      | 59 |
| 7.2.3.1                                     | Morgan.....                                                 | 60 |
| 7.2.3.2                                     | Helmet.....                                                 | 60 |
| 7.2.3.3                                     | CORS (Cross-Origin Resource Sharing) .....                  | 60 |
| 7.2.3.4                                     | Body-Parser.....                                            | 61 |
| 7.2.3.5                                     | Compression .....                                           | 61 |
| 7.2.3.6                                     | Multer.....                                                 | 61 |
| 7.2.3.7                                     | Express-Rate-Limit .....                                    | 62 |
| 7.2.3.8                                     | Cookie-Parser.....                                          | 62 |
| 7.2.3.9                                     | Express-Validator .....                                     | 62 |
| 7.2.3.10                                    | Passport.....                                               | 63 |
| 7.2.3.11                                    | Jsonwebtoken (JWT) .....                                    | 63 |
| 7.2.3.12                                    | Winston.....                                                | 64 |
| 7.2.3.13                                    | Session .....                                               | 65 |
| 7.2.3.14                                    | CSURF .....                                                 | 65 |
| 7.2.4                                       | Exemple : Utilisation d'un Middleware Tiers (morgan) :..... | 66 |
| 7.3                                         | Création de Middlewares Personnalisés .....                 | 66 |
| 7.3.1                                       | Structure d'un Middleware.....                              | 66 |
| 7.3.2                                       | Exemple : Middleware de Journalisation .....                | 66 |
| 7.3.3                                       | Exemple : Middleware de Validation des Données .....        | 67 |
| 7.4                                         | Cas Pratiques .....                                         | 67 |
| 7.4.1                                       | Middleware de Gestion des Erreurs .....                     | 67 |
| Chapitre 8 : Les Moteurs de Templates ..... |                                                             | 68 |
| 8.1                                         | Présentation .....                                          | 68 |

|                                                         |                                                       |    |
|---------------------------------------------------------|-------------------------------------------------------|----|
| 8.2                                                     | Définition.....                                       | 69 |
| 8.3                                                     | Objectifs des moteurs de templates .....              | 69 |
| 8.4                                                     | Fonctionnement d'un moteur de templates .....         | 69 |
| 8.5                                                     | Les principaux moteurs de templates pour Node.js..... | 70 |
| 8.5.1                                                   | EJS (Embedded JavaScript) .....                       | 70 |
| 8.5.2                                                   | Handlebars.....                                       | 70 |
| 8.5.3                                                   | Pug (anciennement Jade).....                          | 71 |
| 8.5.4                                                   | Mustache .....                                        | 71 |
| 8.5.5                                                   | Nunjucks.....                                         | 72 |
| 8.5.6                                                   | Liquid .....                                          | 72 |
| 8.5.7                                                   | Dust.js.....                                          | 73 |
| 8.5.8                                                   | Marko .....                                           | 73 |
| 8.6                                                     | Comparaison entre les moteurs de templates .....      | 74 |
| 8.7                                                     | Choix du moteur selon le projet .....                 | 74 |
| 8.8                                                     | Exemple d'utilisation avec EJS .....                  | 75 |
| 8.8.1                                                   | Installation d'EJS : .....                            | 75 |
| 8.8.2                                                   | Configuration d'Express.js : .....                    | 76 |
| 8.8.3                                                   | Code complet : .....                                  | 76 |
| 8.8.4                                                   | Création du html.....                                 | 77 |
| 8.8.5                                                   | Démarrage du serveur .....                            | 77 |
| 8.8.6                                                   | Résultat : .....                                      | 77 |
| Chapitre 9 : Interaction avec les Bases de Données..... |                                                       | 78 |
| 9.1                                                     | Présentations des bases de données .....              | 78 |
| 9.2                                                     | Pourquoi interagir avec une base de données ? .....   | 78 |
| 9.3                                                     | Types de bases de données .....                       | 78 |
| 9.4                                                     | Les Requêtes directes avec Node.js.....               | 78 |
| 9.4.1                                                   | Définition .....                                      | 78 |

|                                                  |                                                              |    |
|--------------------------------------------------|--------------------------------------------------------------|----|
| 9.4.2                                            | Requêtes SQL (avec MySQL).....                               | 78 |
| 9.4.3                                            | Exemple d'installation : .....                               | 79 |
| 9.4.4                                            | Configuration de la connexion : .....                        | 79 |
| 9.4.5                                            | Requêtes CRUD : .....                                        | 79 |
| 9.4.6                                            | Requêtes NoSQL (avec MongoDB).....                           | 81 |
| 9.4.6.1                                          | Connexion avec MongoDB : .....                               | 81 |
| 9.4.6.2                                          | Exemples de requêtes CRUD : .....                            | 81 |
| 9.5                                              | Les ORM (Object-Relational Mapping).....                     | 82 |
| 9.5.1                                            | Qu'est-ce qu'un ORM ? .....                                  | 83 |
| 9.6                                              | Pourquoi utiliser un ORM ?.....                              | 83 |
| 9.6.1                                            | Avantages : .....                                            | 83 |
| 9.6.2                                            | Inconvénients : .....                                        | 83 |
| 9.7                                              | ORM populaires dans l'écosystème Node.js .....               | 83 |
| 9.7.1                                            | Exemple avec Sequelize.....                                  | 84 |
| Chapitre 10 : Développement d'APIs RESTful ..... |                                                              | 86 |
| 10.1                                             | Présentation d'API.....                                      | 86 |
| 10.2                                             | Évolution des API.....                                       | 86 |
| 10.2.1                                           | Premières générations d'API : .....                          | 86 |
| 10.2.2                                           | Apparition de REST (Representational State Transfer) : ..... | 86 |
| 10.2.3                                           | APIs modernes : .....                                        | 87 |
| 10.3                                             | Concepts théoriques des APIs RESTful .....                   | 87 |
| 10.3.1                                           | Définition de REST .....                                     | 87 |
| 10.3.2                                           | Comment fonctionne une API REST ? .....                      | 87 |
| 10.3.3                                           | Quels sont les principes de l'API REST ? .....               | 88 |
| 10.3.4                                           | Méthodes HTTP et leurs usages.....                           | 89 |
| 10.4                                             | Création d'API avec Node.js .....                            | 89 |
| 10.4.1                                           | Création du projet.....                                      | 89 |



|          |                                                         |     |
|----------|---------------------------------------------------------|-----|
| 10.4.2   | Initialisation du projet .....                          | 89  |
| 10.4.3   | Installation des dépendances nécessaires : .....        | 90  |
| 10.4.4   | Importation des Modules.....                            | 90  |
| 10.4.5   | Initialisation de l'Application Express .....           | 91  |
| 10.4.6   | Configuration des Middlewares .....                     | 91  |
| 10.4.7   | Déclaration de la liste de clients.....                 | 91  |
| 10.4.8   | Route GET pour afficher tous les clients .....          | 92  |
| 10.4.9   | Route GET pour afficher un client par ID.....           | 93  |
| 10.4.10  | Route POST pour ajouter un nouveau client .....         | 93  |
| 10.4.11  | Route PUT pour modifier un client existant par ID ..... | 94  |
| 10.4.12  | Route DELETE pour supprimer un client par ID.....       | 96  |
| 10.4.13  | Lancement du serveur.....                               | 97  |
| 10.4.14  | Démarrage de l'api .....                                | 97  |
| 10.4.15  | Test de l'api .....                                     | 98  |
| 10.4.16  | Ajouter un nouveau client.....                          | 99  |
| 10.4.17  | Mettre à jour un client .....                           | 99  |
| 10.4.18  | Supprimer un client .....                               | 100 |
| 10.4.19  | Commandes cURL .....                                    | 100 |
| 10.5     | Postman.....                                            | 100 |
| 10.5.1   | Qu'est-ce que Postman ? .....                           | 101 |
| 10.5.2   | Rôle de Postman dans le développement d'API : .....     | 101 |
| 10.5.3   | Pourquoi utiliser Postman ?.....                        | 101 |
| 10.5.4   | Méthodes HTTP prises en charge .....                    | 101 |
| 10.5.5   | Les Requêtes http .....                                 | 101 |
| 10.5.5.1 | Composantes principales d'une requête : .....           | 101 |
| 10.5.5.2 | Exemple de requête avec Postman :.....                  | 102 |
| 10.5.6   | Collections.....                                        | 102 |

|          |                                                              |     |
|----------|--------------------------------------------------------------|-----|
| 10.5.7   | Environnements.....                                          | 102 |
| 10.5.8   | Tests Automatisés .....                                      | 102 |
| 10.5.9   | Scripts de Pré-Requête et Tests :.....                       | 103 |
| 10.5.10  | Historique des Requêtes .....                                | 103 |
| 10.6     | Avantages et Limites de Postman .....                        | 103 |
| 10.6.1   | Avantages .....                                              | 103 |
| 10.6.2   | Limites.....                                                 | 103 |
| 10.6.3   | Installation de postman.....                                 | 103 |
| 10.6.4   | Démarrage de postman.....                                    | 104 |
| 10.6.5   | Affichage de la liste des clients.....                       | 104 |
| 10.6.6   | Affichage d'un client en utilisant son id .....              | 105 |
| 10.6.7   | Ajout d'un client.....                                       | 106 |
| 10.6.8   | Suppression d'un client .....                                | 107 |
| 10.6.9   | Comparaison entre Postman et d'autres outils similaires..... | 107 |
| 10.7     | Insomnia.....                                                | 107 |
| 10.7.1   | Qu'est-ce qu'Insomnia ? .....                                | 108 |
| 10.7.2   | Pourquoi utiliser Insomnia ? .....                           | 108 |
| 10.7.3   | Concepts clés d'Insomnia.....                                | 108 |
| 10.7.3.1 | Requêtes HTTP.....                                           | 108 |
| 10.7.3.2 | Collections et dossiers.....                                 | 108 |
| 10.7.3.3 | Environnements .....                                         | 109 |
| 10.7.3.4 | Authentification .....                                       | 109 |
| 10.7.3.5 | Documentation intégrée .....                                 | 109 |
| 10.7.4   | Avantages spécifiques d'Insomnia .....                       | 109 |
| 10.7.5   | Comparaison Insomnia vs Postman .....                        | 109 |
| 10.7.6   | Installation et configuration .....                          | 110 |
| 10.7.7   | Cas pratique : Tester une API REST simple .....              | 111 |

|                                                                                     |                                                                     |     |
|-------------------------------------------------------------------------------------|---------------------------------------------------------------------|-----|
| 10.7.7.1                                                                            | Exemple d'API.....                                                  | 111 |
| 10.7.7.2                                                                            | Requêtes à tester : .....                                           | 111 |
| 10.7.8                                                                              | Étapes pour tester une API avec Insomnia.....                       | 112 |
| 10.7.8.1                                                                            | Tester un endpoint GET .....                                        | 112 |
| 10.7.8.2                                                                            | Tester un endpoint POST.....                                        | 112 |
| Chapitre 11 : Création d'une api complet en utilisant la base de données MySQL..... |                                                                     | 115 |
| 11.1                                                                                | Création de la base de données MySQL .....                          | 115 |
| 11.1.1                                                                              | Démarrer le serveur MySQL.....                                      | 115 |
| 11.1.2                                                                              | Création de la base de données : .....                              | 115 |
| 11.1.3                                                                              | Sélection de la base de données : .....                             | 115 |
| 11.1.4                                                                              | Créer une table pour les clients .....                              | 116 |
| 11.1.5                                                                              | Création d'un utilisateur.....                                      | 116 |
| 11.1.6                                                                              | Attribution et application des privilèges .....                     | 116 |
| 11.1.7                                                                              | Insertion des données .....                                         | 116 |
| 11.2                                                                                | Création de l'api.....                                              | 117 |
| 11.3                                                                                | Création du projet Node.js .....                                    | 117 |
| 11.4                                                                                | Initialisation du projet Node.js :.....                             | 117 |
| 11.5                                                                                | Installation des dépendances nécessaires :.....                     | 117 |
| 11.5.1                                                                              | Installation d'express.....                                         | 118 |
| 11.5.2                                                                              | Décomposition de la commande .....                                  | 118 |
| 11.5.3                                                                              | Où est installé Express.js ? .....                                  | 118 |
| 11.5.4                                                                              | Installation de mysql2 .....                                        | 118 |
| 11.5.5                                                                              | Décomposition de la commande .....                                  | 119 |
| 11.5.6                                                                              | Ce que fait cette commande .....                                    | 119 |
| 11.5.7                                                                              | Où est installé mysql2 ? .....                                      | 119 |
| 11.5.8                                                                              | Installation de body-parser .....                                   | 120 |
| 11.6                                                                                | Création du fichier app.js dans le répertoire de votre projet. .... | 120 |

|                                         |                                                                |     |
|-----------------------------------------|----------------------------------------------------------------|-----|
| 11.6.1                                  | Importation des modules nécessaires .....                      | 120 |
| 11.6.2                                  | Initialisation de l'application et configuration du port ..... | 120 |
| 11.6.3                                  | Middleware pour traiter les données JSON.....                  | 121 |
| 11.6.4                                  | Configuration de la connexion MySQL .....                      | 121 |
| 11.6.5                                  | Connexion à la base de données.....                            | 121 |
| 11.6.6                                  | Définition des routes CRUD .....                               | 122 |
| 11.6.6.1                                | Récupérer tous les clients.....                                | 122 |
| 11.6.6.2                                | Récupérer un client par ID .....                               | 122 |
| 11.6.6.3                                | Ajouter un nouveau client .....                                | 123 |
| 11.6.6.4                                | Mise à jour un client .....                                    | 124 |
| 11.6.6.5                                | Supprimer un client.....                                       | 124 |
| 11.6.7                                  | Démarrage du serveur .....                                     | 125 |
| 11.7                                    | Test de l'api avec le client curl .....                        | 125 |
| 11.7.1                                  | Affichage des clients .....                                    | 125 |
| 11.7.2                                  | Récupérer un client par ID : .....                             | 125 |
| 11.7.3                                  | Insertion d'un client .....                                    | 126 |
| 11.7.4                                  | Modification d'un client.....                                  | 126 |
| 11.7.5                                  | Suppression d'un client .....                                  | 126 |
| 11.7.6                                  | Vérification cote base de données .....                        | 127 |
| 11.8                                    | Utilisation du client isomia .....                             | 127 |
| Chapitre 12 : Documentation d'API ..... |                                                                | 129 |
| 12.1                                    | Présentation.....                                              | 129 |
| 12.2                                    | Qu'est-ce que la documentation d'API ? .....                   | 129 |
| 12.3                                    | Pourquoi documenter une API ? .....                            | 129 |
| 12.3.1                                  | Avantages pour les développeurs .....                          | 129 |
| 12.3.2                                  | Avantages pour les équipes .....                               | 129 |
| 12.4                                    | Bonnes pratiques pour une documentation d'API.....             | 130 |

|                                      |                                                         |     |
|--------------------------------------|---------------------------------------------------------|-----|
| 12.5                                 | Outils populaires pour documenter les API .....         | 130 |
| 12.6                                 | Génération d'une documentation d'API avec Swagger ..... | 130 |
| 12.6.1                               | Aperçu des outils Swagger .....                         | 130 |
| 12.6.2                               | Les principaux outils Swagger : .....                   | 130 |
| 12.7                                 | Mise en œuvre .....                                     | 130 |
| 12.7.1                               | Installer les modules nécessaires .....                 | 130 |
| 12.7.2                               | Préparation des fichiers .....                          | 131 |
| 12.7.3                               | Génération de la spécification Swagger .....            | 131 |
| 12.7.4                               | Intégration du Swagger UI dans l'application.....       | 132 |
| Chapitre 13 : Sécurité des API ..... |                                                         | 135 |
| 13.1                                 | Présentation .....                                      | 135 |
| 13.2                                 | Concepts fondamentaux de la sécurité des API .....      | 135 |
| 13.2.1                               | Définition et enjeux de la sécurité des API .....       | 135 |
| 13.3                                 | Principes fondamentaux de la sécurité des API .....     | 135 |
| 13.4                                 | Pourquoi sécuriser une API ? .....                      | 135 |
| 13.5                                 | Principaux risques de sécurité des API .....            | 135 |
| 13.6                                 | Menaces courantes sur les API .....                     | 136 |
| 13.6.1                               | Injection SQL et NoSQL .....                            | 136 |
| 13.6.2                               | Injection SQL (SQLi).....                               | 136 |
| 13.6.3                               | Injection NoSQL .....                                   | 136 |
| 13.6.4                               | Vol d'identité (Attaque JWT) .....                      | 136 |
| 13.6.5                               | Cross-Site Scripting (XSS).....                         | 137 |
| 13.6.6                               | Attaques CSRF (Cross-Site Request Forgery) .....        | 137 |
| 13.6.7                               | Comment fonctionne une attaque CSRF ? .....             | 137 |
| 13.6.8                               | Comment se protéger contre CSRF ? .....                 | 137 |
| 13.6.8.1                             | Utiliser un Token CSRF (Anti-CSRF Token) .....          | 137 |
| 13.6.8.2                             | Vérifier l'Origine et le Referer .....                  | 137 |

|           |                                                                 |     |
|-----------|-----------------------------------------------------------------|-----|
| 13.6.8.3  | Utiliser des Cookies SameSite=Strict .....                      | 138 |
| 13.6.9    | Attaque par Force Brute .....                                   | 138 |
| 13.6.10   | Types d'attaques par Force Brute.....                           | 138 |
| 13.6.10.1 | Attaque par dictionnaire .....                                  | 138 |
| 13.6.10.2 | Attaque exhaustive (Brute Force pure).....                      | 138 |
| 13.6.10.3 | Attaque par force brute sur le hachage .....                    | 138 |
| 13.6.11   | Comment se protéger contre les attaques par Force Brute ? ..... | 138 |
| 13.6.11.1 | Limiter le nombre de tentatives .....                           | 138 |
| 13.6.11.2 | Utiliser une authentification forte.....                        | 138 |
| 13.6.11.3 | Implémenter une authentification multi-facteurs (MFA) .....     | 139 |
| 13.6.11.4 | Hacher et saler les mots de passe.....                          | 139 |
| 13.6.11.5 | Surveiller les tentatives de connexion .....                    | 139 |
| 13.7      | Authentification des utilisateurs dans une API.....             | 139 |
| 13.7.1    | Qu'est-ce que l'authentification et l'autorisation ? .....      | 139 |
| 13.7.2    | Authentification multi-facteurs (MFA/2FA).....                  | 139 |
| 13.7.3    | Pourquoi la MFA est-elle importante ? .....                     | 140 |
| 13.7.4    | Comment fonctionne la MFA ? .....                               | 140 |
| 13.7.5    | Différents types d'authentification en API .....                | 140 |

# Chapitre 1 : Introduction à Node.js

## 1.1 Présentation

Node.js est une plateforme de développement Javascript. Ce n'est pas un serveur, ce n'est pas un framework, c'est juste le langage Javascript avec des bibliothèques permettant de réaliser des actions comme écrire sur la sortie standard, ouvrir/fermer des connections réseau ou encore créer un fichier.

Il est souvent confondu avec un serveur car c'est son origine : Node.js a été créé par Ryan Dahl dans le but de pouvoir créer des applications temps réel où le serveur est capable de pousser de l'information au client. C'est dans ce but qu'il utilise la bibliothèque libuv pour réaliser son modèle d'entrée sortie non bloquante.

## 1.2 Qu'est-ce que Node.js ?

Node.js est un environnement d'exécution JavaScript basé sur le moteur V8 de Chrome. Il utilise un modèle I/O non bloquant et basé sur les événements, ce qui le rend léger et efficace. Le gestionnaire de paquets de Node.js, npm, est l'un des plus grands écosystèmes de bibliothèques open-source au monde.

### 1.2.1 Définition

Node.js est une plateforme open-source et multiplateforme conçue pour exécuter du JavaScript en dehors d'un navigateur. Contrairement à JavaScript classique utilisé pour le développement front-end, Node.js permet d'utiliser JavaScript côté serveur pour créer des applications backend puissantes.

### 1.2.2 Origine

Node.js a été créé en 2009 par Ryan Dahl. Sa principale motivation était de concevoir un environnement capable de traiter simultanément plusieurs requêtes de manière efficace, sans bloquer les opérations d'entrée/sortie (I/O).

- 🚦 Première version : Publiée en 2009.
- 🚦 Basé sur : Le moteur V8 de Google Chrome, un moteur rapide et performant pour exécuter du JavaScript.
- 🚦 Organisation responsable : Aujourd'hui, Node.js est géré par la OpenJS Foundation.

## 1.3 Caractéristiques principales

### 1.3.1 Exécution JavaScript côté serveur

Node.js révolutionne la manière de développer des applications en permettant d'utiliser JavaScript pour gérer le backend. Cela permet aux développeurs de travailler avec un seul langage (JavaScript) sur l'ensemble du projet, aussi bien pour le front-end que le back-end.

✚ **Avantage :** Simplifie le développement et facilite la collaboration entre équipes front-end et back-end.

✚ **Exemple d'utilisation :** Création d'API REST, applications en temps réel (chat, jeux multijoueurs), automatisation de tâches, etc.

### 1.3.2 Modèle non bloquant

Node.js utilise un modèle non bloquant pour gérer les opérations d'entrée/sortie (I/O). Cela signifie qu'il peut traiter plusieurs requêtes simultanément sans attendre qu'une opération soit terminée avant d'en démarrer une autre.

✚ **Fonctionnement :**

- Les opérations lourdes (comme la lecture d'un fichier ou la requête à une base de données) sont exécutées en arrière-plan.
- Une fois terminées, elles signalent leur résultat via un mécanisme de callback ou de promesse.

✚ **Avantage :** Très efficace pour les applications nécessitant un grand nombre de connexions simultanées, comme les serveurs web.

### 1.3.3 Utilisation du moteur V8

Node.js repose sur le moteur V8 de Google, qui est le moteur JavaScript utilisé dans le navigateur Chrome.

✚ **Pourquoi V8 ?**

- **Performances élevées :** Compile le JavaScript directement en code machine (langage natif) pour une exécution rapide.
- **Stabilité :** Développé par Google, constamment optimisé et amélioré.

## 1.4 Avantages et Cas d'utilisation



### 1.4.1 Avantages de Node.js

- ✚ **Un langage unique** : Utiliser JavaScript pour tout le cycle de développement réduit la complexité et facilite la collaboration entre équipes.
- ✚ **Performances élevées** : L'architecture non bloquante et le moteur V8 garantissent des performances exceptionnelles, même sous une charge importante.
- ✚ **Communauté et écosystème** : Grâce à npm (Node Package Manager), Node.js dispose de l'un des plus grands écosystèmes de bibliothèques open-source.
- ✚ **Applications en temps réel** : Idéal pour les applications nécessitant des mises à jour instantanées, comme les chats, jeux multijoueurs, ou tableaux de bord.

### 1.4.2 Cas d'utilisation

- ✚ **Applications en temps réel** :
  - Chats en direct (exemple : WhatsApp Web).
  - Jeux multijoueurs en ligne.
- ✚ **APIs RESTful** : Node.js est couramment utilisé pour développer des APIs légères et rapides pour la communication entre le client et le serveur.
- ✚ **Microservices** : Node.js est idéal pour les architectures modulaires et scalables.
- ✚ **Automatisation** : Utilisé pour automatiser des tâches avec des outils comme Gulp.js ou Webpack.

## 1.5 Fonctionnement interne

### 1.5.1 Architecture de Node.js

- ✚ **Moteur V8**
  - Compile le JavaScript en code machine.
  - Garantit une exécution rapide.
- ✚ **Libuv**
  - Une bibliothèque C utilisée par Node.js pour implémenter le modèle d'E/S non bloquant.
  - Fournit la boucle d'événements (Event Loop) et gère les threads.
- ✚ **Event Loop (boucle d'événements)**
  - Cœur de Node.js, elle permet de gérer plusieurs requêtes sans bloquer l'exécution.
  - Fonctionne en assignant des tâches (callbacks) à des files d'attente.
  - Exemple :

```
console.log("Début");

setTimeout(() => {

    console.log("Exécution différée");

}, 2000);

console.log("Fin");
```

## 1.6 Installation de Nodejs sur différents systèmes d'exploitation

### 1.6.1 Installation de Nodejs sous linux(ubuntu)

root@nasry-ahamadi:/home/nasry# apt install curl

```
root@nasry-ahamadi:/home/nasry# apt install curl
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances
Lecture des informations d'état... Fait
Le paquet suivant a été installé automatiquement et n'est plus nécessaire :
  gir1.2-goa-1.0
Veuillez utiliser « apt autoremove » pour le supprimer.
Les NOUVEAUX paquets suivants seront installés :
  curl
0 mis à jour, 1 nouvellement installés, 0 à enlever et 0 non mis à jour.
Il est nécessaire de prendre 162 ko dans les archives.
Après cette opération, 414 ko d'espace disque supplémentaires seront utilisés.
Réception de :1 http://sn.archive.ubuntu.com/ubuntu focal-updates/main amd64 curl amd64 7.68.0-1ubuntu2.25 [162 kB]
125 ko réceptionnés en 1s (151 ko/s)
Sélection du paquet curl précédemment désélectionné.
(Lecture de la base de données... 184275 fichiers et répertoires déjà installés.)
Préparation du dépaquetage de .../curl_7.68.0-1ubuntu2.25_amd64.deb ...
Dépaquetage de curl (7.68.0-1ubuntu2.25) ...
Paramétrage de curl (7.68.0-1ubuntu2.25) ...
Traitement des actions différées (« triggers ») pour man-db (2.9.1-1) ...
root@nasry-ahamadi:/home/nasry#
```

root@nasry-ahamadi:/home/nasry# curl -fsSL https://deb.nodesource.com/setup\_20.x | sudo -E bash -

```

root@nasry-ahamadi:/home/nasry# curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
2025-01-02 22:10:18 - Installing pre-requisites
Atteint :1 http://sn.archive.ubuntu.com/ubuntu focal InRelease
Atteint :2 http://sn.archive.ubuntu.com/ubuntu focal-updates InRelease
Atteint :3 http://security.ubuntu.com/ubuntu focal-security InRelease
Atteint :4 http://sn.archive.ubuntu.com/ubuntu focal-backports InRelease
Lecture des listes de paquets... Fait
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances
Lecture des informations d'état... Fait
ca-certificates est déjà la version la plus récente (20240203~20.04.1).
ca-certificates passé en « installé manuellement ».
curl est déjà la version la plus récente (7.68.0-1ubuntu2.25).
gnupg est déjà la version la plus récente (2.2.19-3ubuntu2.2).
gnupg passé en « installé manuellement ».
Le paquet suivant a été installé automatiquement et n'est plus nécessaire :
  gir1.2-goa-1.0
Veuillez utiliser « sudo apt autoremove » pour le supprimer.
Les NOUVEAUX paquets suivants seront installés :
  apt-transport-https
0 mis à jour, 1 nouvellement installés, 0 à enlever et 0 non mis à jour.
Il est nécessaire de prendre 1704 o dans les archives.
Après cette opération, 162 ko d'espace disque supplémentaires seront utilisés.

```

root@nasry-ahamadi:/home/nasry# apt install nodejs

```

root@nasry-ahamadi:/home/nasry# apt install nodejs
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances
Lecture des informations d'état... Fait
Le paquet suivant a été installé automatiquement et n'est plus nécessaire :
  gir1.2-goa-1.0
Veuillez utiliser « apt autoremove » pour le supprimer.
Les NOUVEAUX paquets suivants seront installés :
  nodejs
0 mis à jour, 1 nouvellement installés, 0 à enlever et 0 non mis à jour.
Il est nécessaire de prendre 31,8 Mo dans les archives.
Après cette opération, 197 Mo d'espace disque supplémentaires seront utilisés.
Réception de :1 https://deb.nodesource.com/node_20.x nodistro/main amd64 nodejs amd64 20.18.1-1nodeso
urce1 [31,8 MB]
31,8 Mo réceptionnés en 31s (1019 ko/s)
Sélection du paquet nodejs précédemment désélectionné.
(Lecture de la base de données... 184286 fichiers et répertoires déjà installés.)
Préparation du dépaquetage de .../nodejs_20.18.1-1nodesource1_amd64.deb ...
Dépaquetage de nodejs (20.18.1-1nodesource1) ...
Paramétrage de nodejs (20.18.1-1nodesource1) ...
Traitement des actions différées (« triggers ») pour man-db (2.9.1-1) ...
root@nasry-ahamadi:/home/nasry#

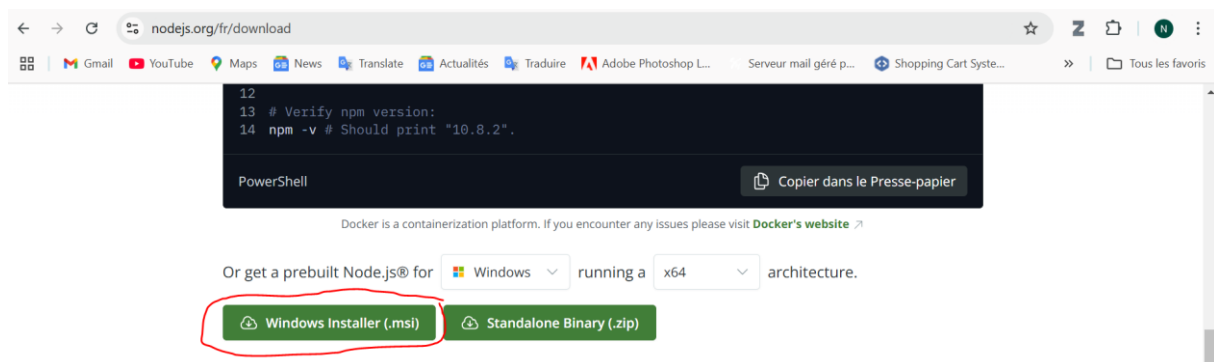
```

```

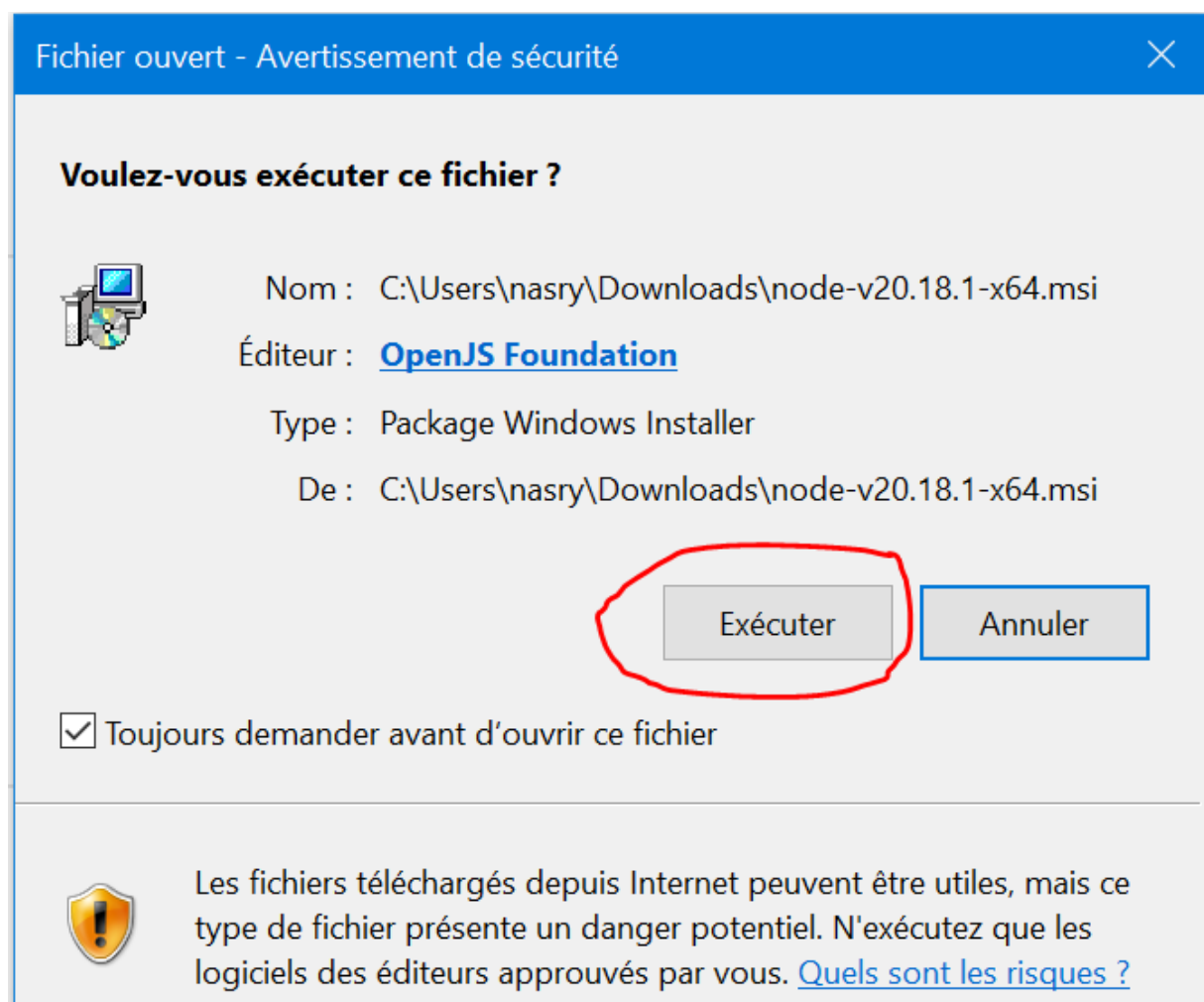
root@nasry-ahamadi:/home/nasry# node --version
v20.18.1
root@nasry-ahamadi:/home/nasry#

```

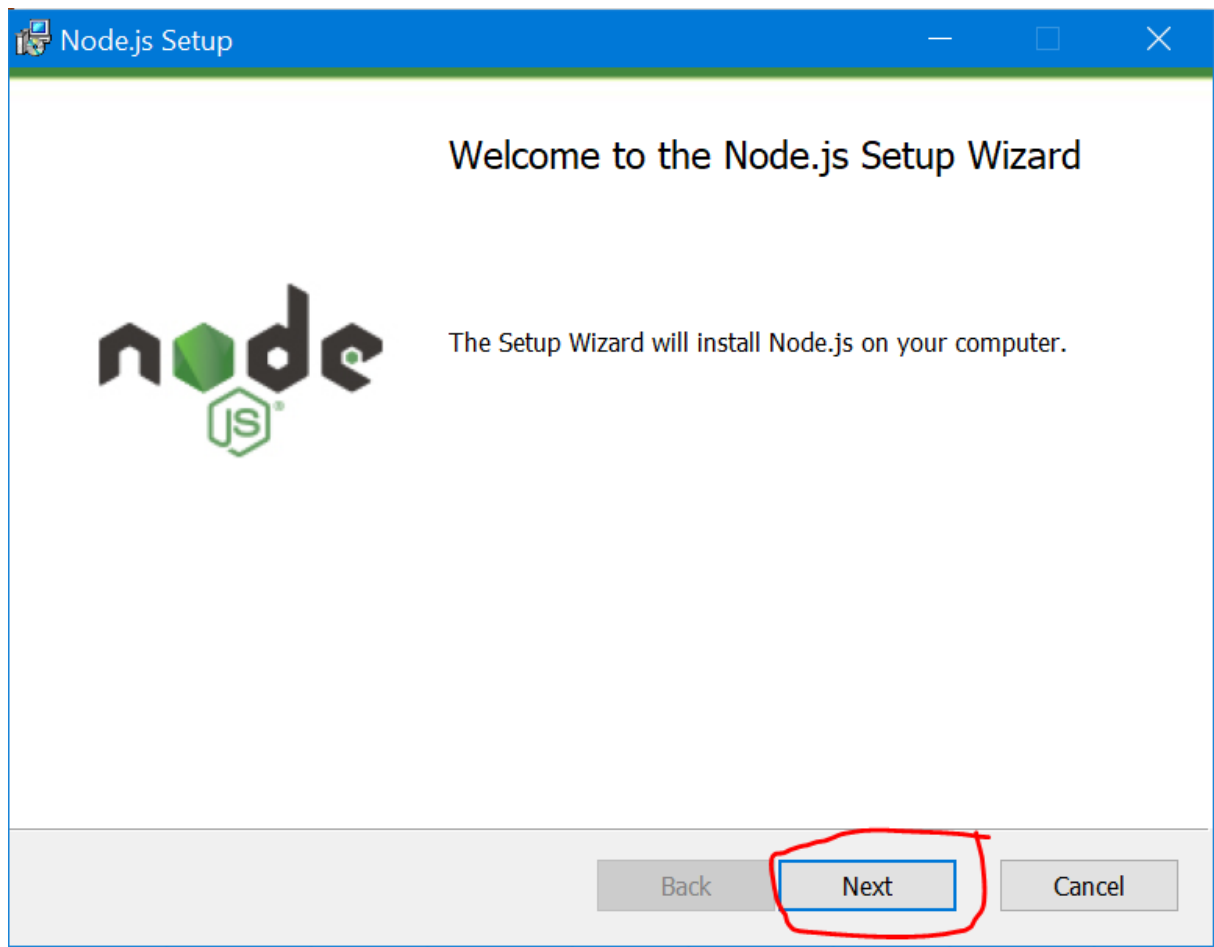
## 1.6.2 Installation de NodeJs sous Windows.



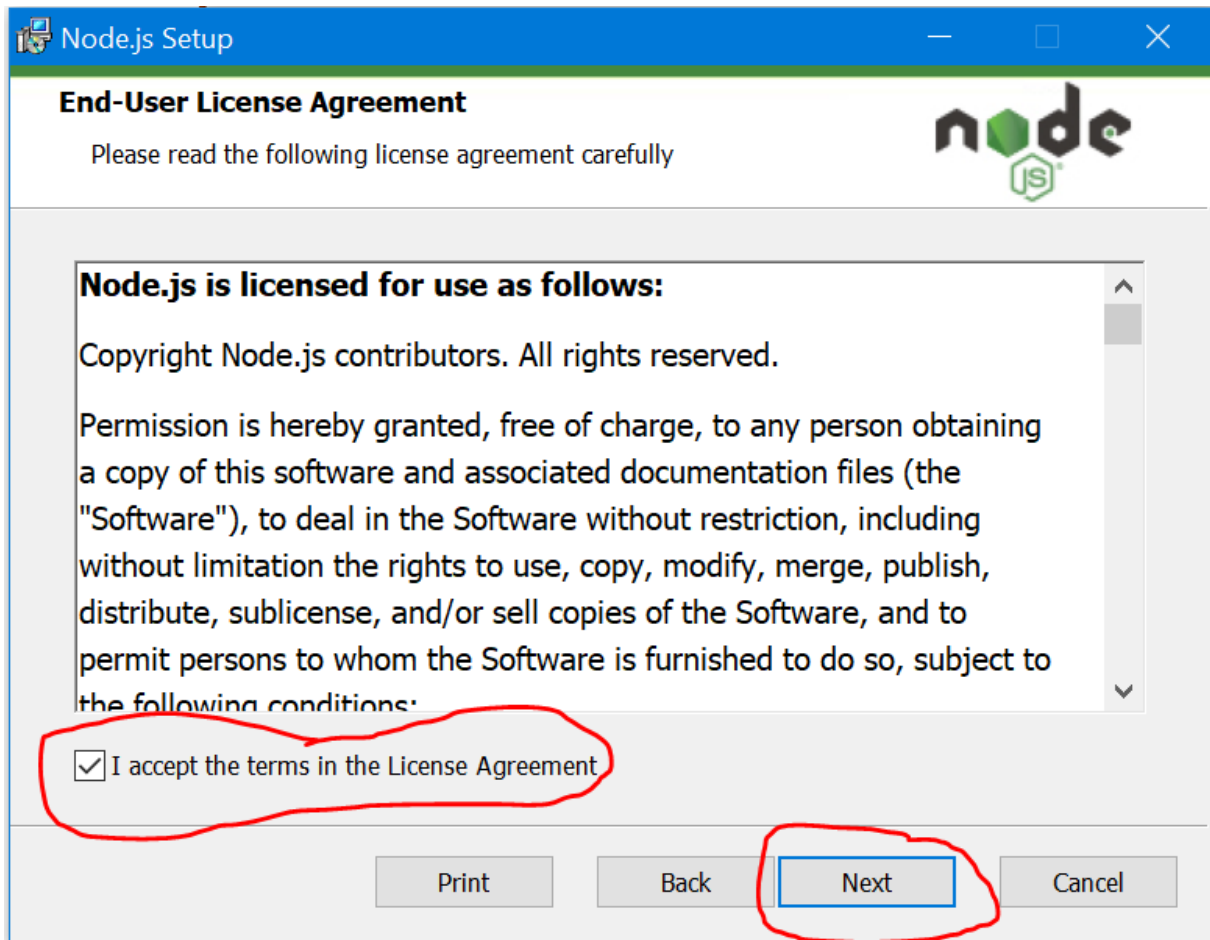
Cliquez sur exécuter comme suite :



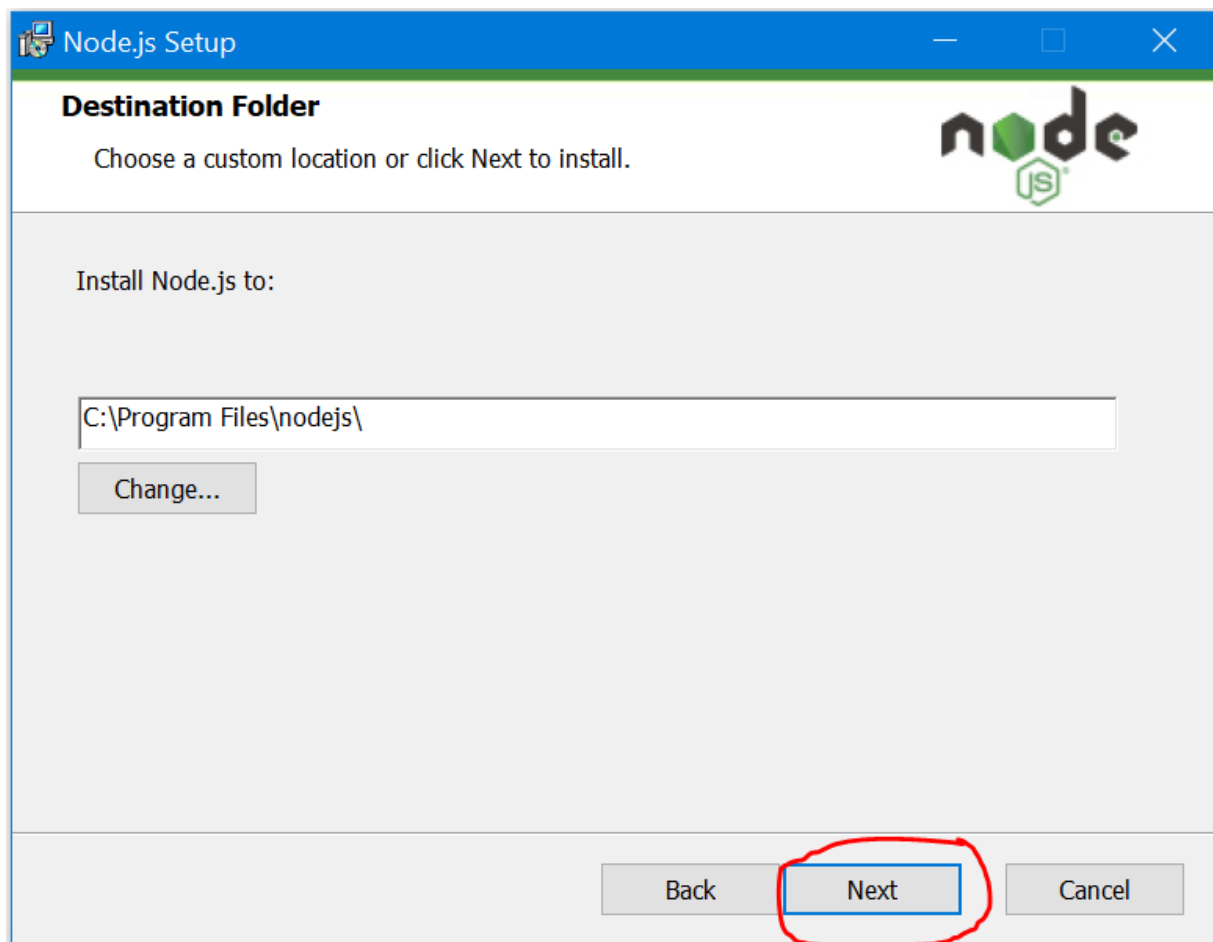
Cliquez sur next :



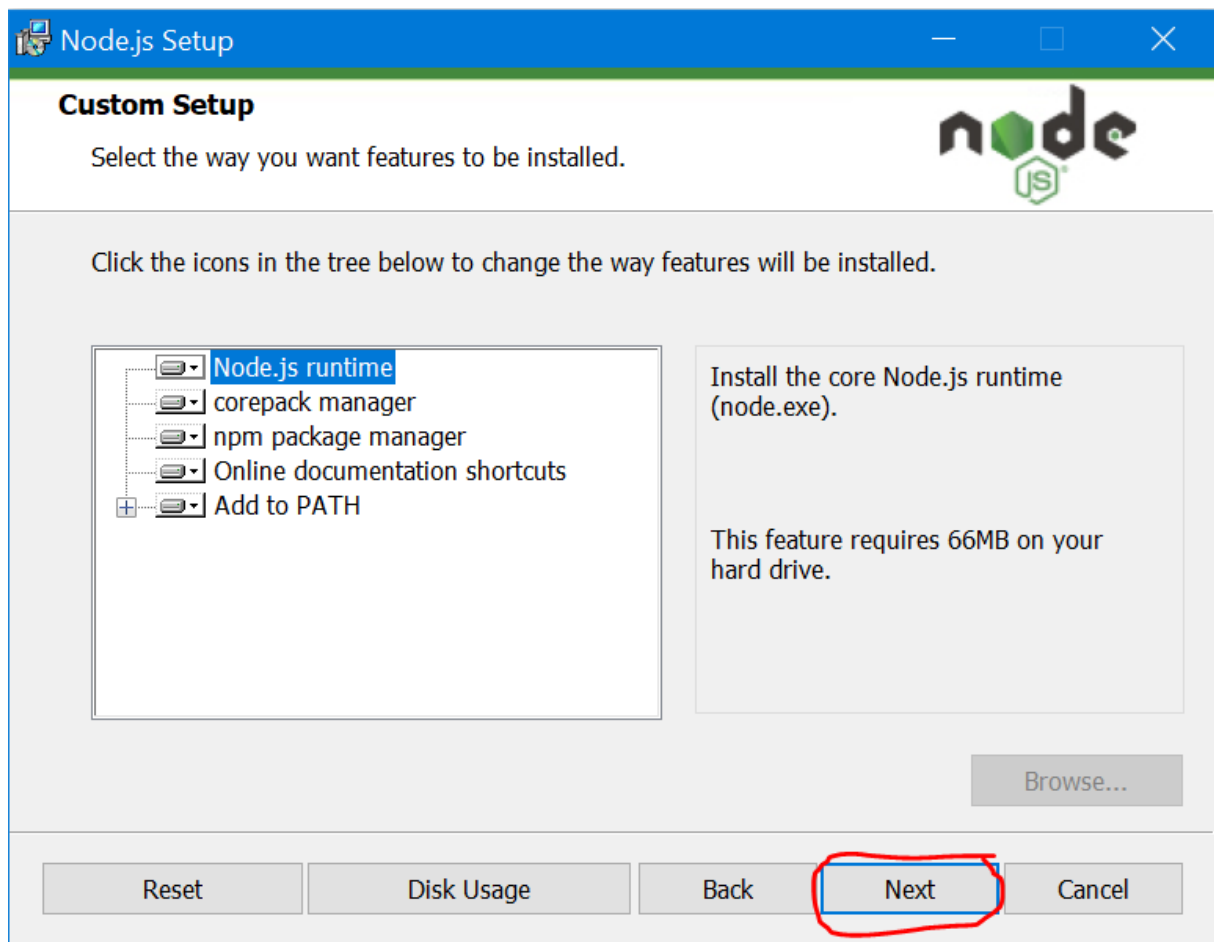
On coche I accept the terms in the licence agreement, puis cliquez sur next comme suite :



Cliquez sur next :

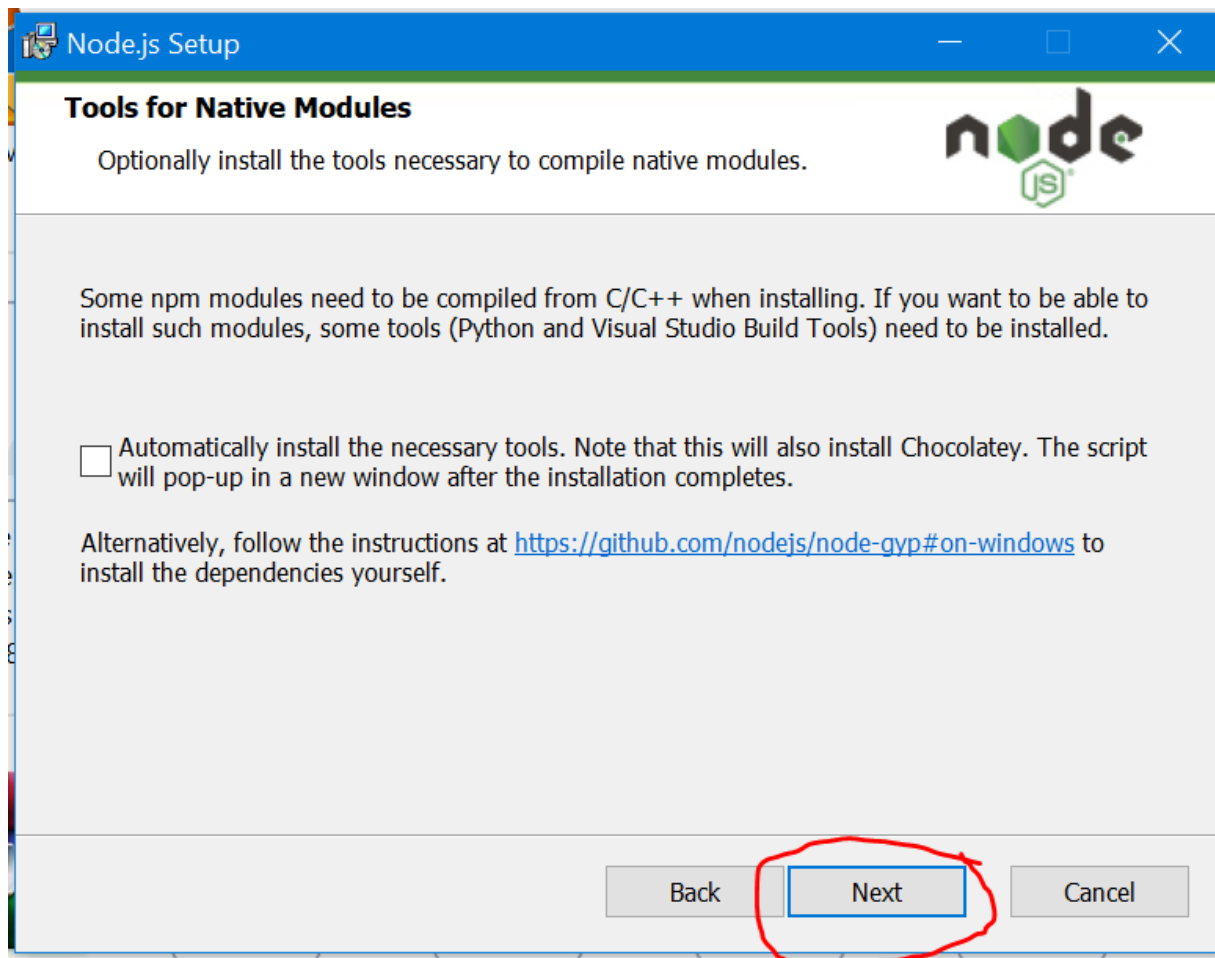


Cliquez sur next :

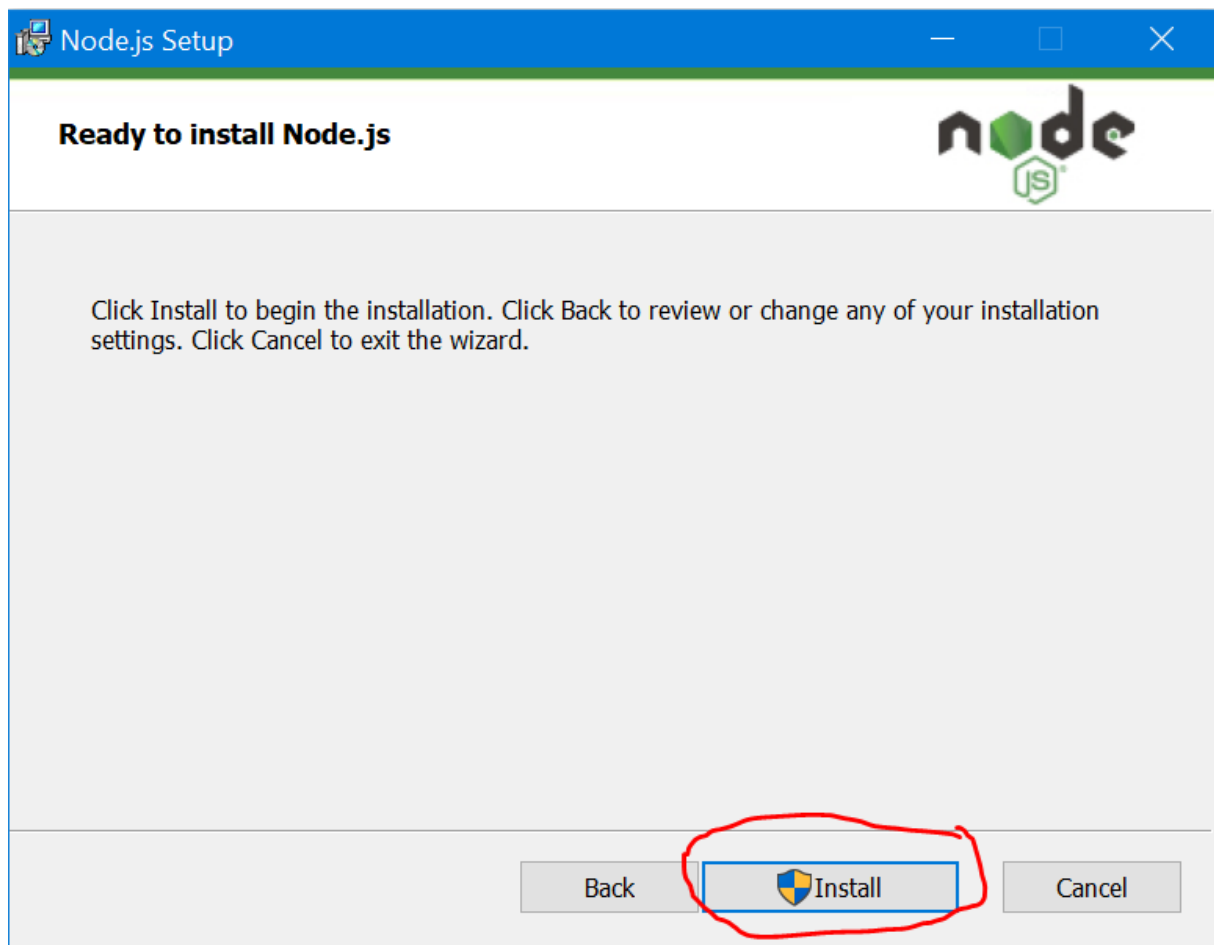


Cliquez sur next :

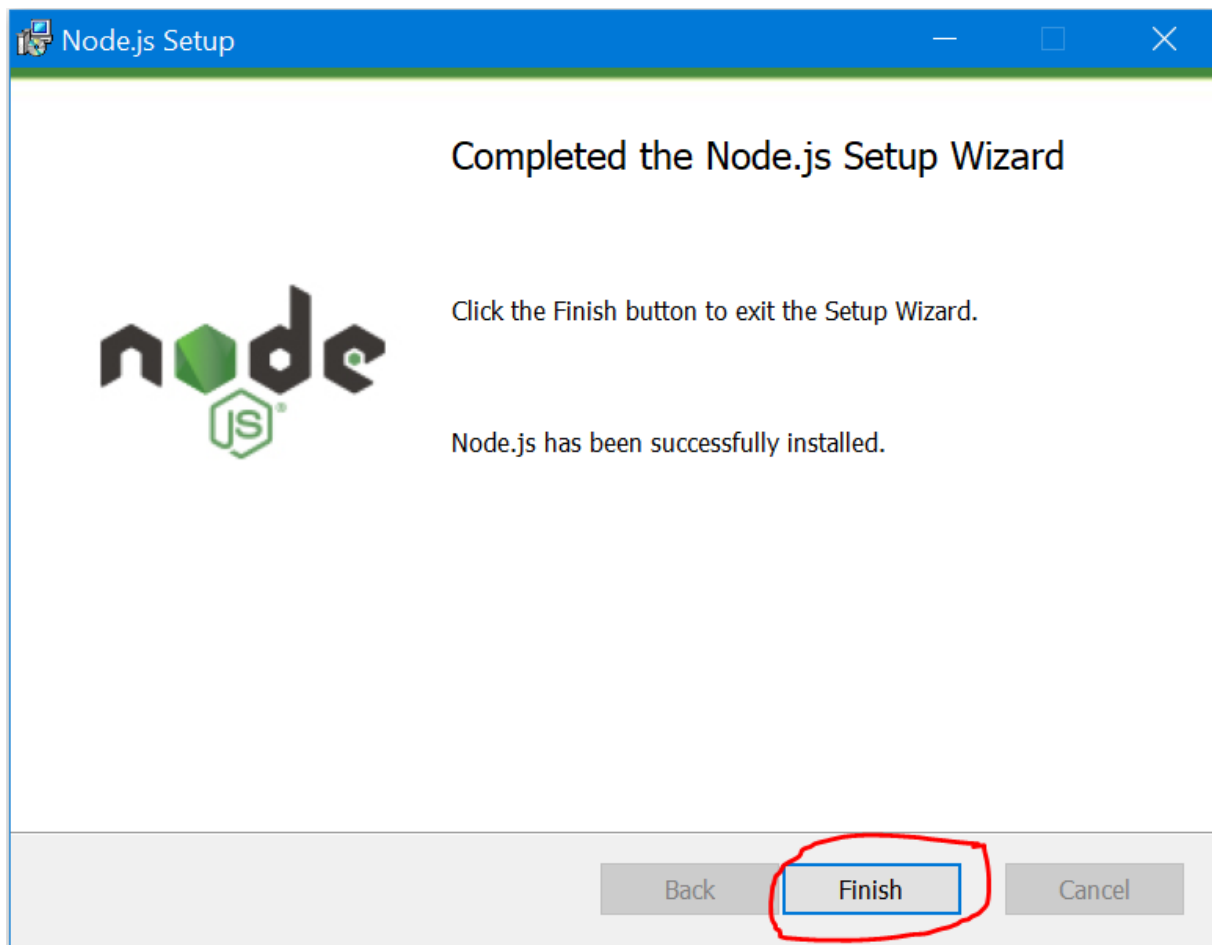




Cliquez sur install :



Cliquez sur finish :



### 1.6.3 Vérification de la version de nodejs sous Windows.

Pour vérifier la version de nodejs, on ouvre le terminal, après on tape la commande suivante :

```
C:\Users\nasry>node --version
```

The image shows a Windows command prompt window with a blue title bar. The title bar text is 'C:\Windows\system32\cmd.exe'. The command prompt displays the following text: 'Microsoft Windows [version 10.0.19045.5011]', '(c) Microsoft Corporation. Tous droits réservés.', 'C:\Users\nasry>node --version', 'v20.18.1', and 'C:\Users\nasry>'. The text is white on a black background.

# Chapitre 2 : Installation et Gestion des Versions avec NVM

## 2.1 Présentation de NVM

Le Node Version Manager (NVM) est un outil essentiel pour les développeurs qui travaillent avec Node.js. Dans cet article, nous explorerons en détail ce qu'est le NVM, comment il fonctionne, et comment l'installer sur Windows et Linux. De plus, nous aborderons l'installation de NPM, la mise à jour de Node sur Ubuntu, et fournirons des astuces utiles pour gérer efficacement votre environnement de développement.

## 2.2 Qu'est-ce que NVM ?

NVM (Node Version Manager) est un outil en ligne de commande qui permet de gérer plusieurs versions de Node.js sur un même système. Il est particulièrement utile pour les développeurs travaillant sur des projets nécessitant différentes versions de Node.js.

- ✚ **Fonction principale :** Installer, utiliser, et basculer facilement entre différentes versions de Node.js.
- ✚ **Fonctionnement :** Chaque version de Node.js installée via NVM est isolée, permettant de passer d'une version à une autre sans conflit.

## 2.3 Comment fonctionne NVM ?

- ✚ **Isolation des versions :**

NVM installe chaque version de Node.js dans un répertoire spécifique, et ajuste les chemins d'exécution pour activer la version souhaitée.

- ✚ **Commandes simples :**

Avec des commandes comme `nvm install` ou `nvm use`, vous pouvez gérer les versions de Node.js sans avoir à désinstaller ou réinstaller manuellement.

- ✚ **Configuration indépendante :**

Chaque projet peut utiliser une version spécifique de Node.js définie dans un fichier `.nvmrc`.

## 2.4 Avantages et Inconvénients de NVM

### 2.4.1 Avantages

- + **Facilité d'installation et de gestion** : Simplifie la gestion de plusieurs versions.
- + **Isolation des environnements** : Pas de conflits entre projets utilisant des versions différentes.
- + **Flexibilité** : Permet de basculer rapidement entre les versions pour tester ou développer.
- + **Compatibilité** : Fonctionne sur la majorité des systèmes d'exploitation (Linux, macOS, Windows via WSL).

### 2.4.2 Inconvénients

- + **Performance** : Peut être légèrement plus lent que des installations directes de Node.js.
- + **Non natif sous Windows** : Nécessite Windows Subsystem for Linux (WSL) ou une alternative comme nvm-windows.

## 2.5 Avantages d'utiliser NVM pour l'installation de Node.js

- + **Conformité aux projets** :

Certains projets nécessitent des versions spécifiques de Node.js. Avec NVM, vous pouvez facilement respecter ces exigences.

- + **Mises à jour simplifiées** :

Installez les dernières versions de Node.js sans risque d'altérer les projets existants.

- + **Tests rétrocompatibles** :

Tester des projets avec des versions plus anciennes ou plus récentes pour vérifier leur compatibilité.

## 2.6 Installation de NVM sur différents systèmes d'exploitation

### 2.6.1 Installation sur Linux (Ubuntu)

Téléchargement et installation de NVM

Exécutez la commande suivante dans le terminal :

```
root@nasry-ahamadi:/home/nasry# apt install curl
```

```

root@nasry-ahamadi:/home/nasry# apt install curl
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances
Lecture des informations d'état... Fait
Le paquet suivant a été installé automatiquement et n'est plus nécessaire :
  gir1.2-goa-1.0
Veuillez utiliser « apt autoremove » pour le supprimer.
Les NOUVEAUX paquets suivants seront installés :
  curl
0 mis à jour, 1 nouvellement installés, 0 à enlever et 0 non mis à jour.
Il est nécessaire de prendre 162 ko dans les archives.
Après cette opération, 414 ko d'espace disque supplémentaires seront utilisés.
Réception de :1 http://sn.archive.ubuntu.com/ubuntu focal-updates/main amd64 curl amd64 7.68.0-1ubunt
u2.25 [162 kB]
162 ko réceptionnés en 1s (121 ko/s)
Sélection du paquet curl précédemment désélectionné.
(Lecture de la base de données... 184275 fichiers et répertoires déjà installés.)
Préparation du dépaquetage de .../curl_7.68.0-1ubuntu2.25_amd64.deb ...
Dépaquetage de curl (7.68.0-1ubuntu2.25) ...
Paramétrage de curl (7.68.0-1ubuntu2.25) ...
Traitement des actions différées (« triggers ») pour man-db (2.9.1-1) ...
root@nasry-ahamadi:/home/nasry#

```

```

root@nasry-ahamadi:/home/nasry# curl -o- https://raw.githubusercontent.com/nvm-
sh/nvm/v0.39.5/install.sh | bash

```

```

root@nasry-ahamadi:/home/nasry# curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.5/install
.sh | bash
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100 15916  100 15916    0     0  29973      0 --:--:-- --:--:-- --:--:-- 29973
=> Downloading nvm as script to '/root/.nvm'

=> Appending nvm source string to /root/.bashrc
=> Appending bash_completion source string to /root/.bashrc
=> Close and reopen your terminal to start using nvm or run the following to use it now:

export NVM_DIR="$HOME/.nvm"
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh" # This loads nvm
[ -s "$NVM_DIR/bash_completion" ] && \. "$NVM_DIR/bash_completion" # This loads nvm bash_completion
root@nasry-ahamadi:/home/nasry#

```

```

root@nasry-ahamadi:/home/nasry# source ~/.bashrc

```

```

root@nasry-ahamadi:/home/nasry# source ~/.bashrc
root@nasry-ahamadi:/home/nasry#

```

```

root@nasry-ahamadi:/home/nasry# nvm --version

```

```

root@nasry-ahamadi:/home/nasry# nvm --version
0.39.5
root@nasry-ahamadi:/home/nasry#

```

## 2.7 Liste de toutes les commandes de NVM

| Commande                                       | Description                                                   |
|------------------------------------------------|---------------------------------------------------------------|
| <code>nvm install &lt;version&gt;</code>       | Installe une version spécifique de Node.js.                   |
| <code>nvm use &lt;version&gt;</code>           | Utilise une version installée de Node.js.                     |
| <code>nvm ls</code>                            | Liste les versions installées localement.                     |
| <code>nvm ls-remote</code>                     | Liste toutes les versions disponibles en ligne.               |
| <code>nvm uninstall &lt;version&gt;</code>     | Désinstalle une version de Node.js.                           |
| <code>nvm current</code>                       | Affiche la version actuellement utilisée.                     |
| <code>nvm alias default &lt;version&gt;</code> | Définit une version par défaut pour tous les nouveaux shells. |

## 2.8 Gestion des versions de Node.js avec NVM

### 2.8.1 Commandes principales de NVM

 **Pour Lister les versions disponibles en ligne**

```
root@nasry-ahamadi:/home/nasry# nvm ls-remote
v0.1.14
v0.1.15
v0.1.16
v0.1.17
v0.1.18
v0.1.19
v0.1.20
v0.1.21
v0.1.22
v0.1.23
v0.1.24
v0.1.25
v0.1.26
v0.1.27
v0.1.28
v0.1.29
v0.1.30
v0.1.31
v0.1.32
v0.1.33
v0.1.90
v0.1.91
```

 **Installation d'une version spécifique (la version de 20).**

```
root@nasry-ahamadi:/home/nasry# nvm install 20
```

```

root@nasry-ahamadi:/home/nasry# nvm install 20
Downloading and installing node v20.18.1...
Downloading https://nodejs.org/dist/v20.18.1/node-v20.18.1-linux-x64.tar.xz...
##### 100,0%
Computing checksum with sha256sum
Checksums matched!
Now using node v20.18.1 (npm v10.8.2)
Creating default alias: default -> 20 (-> v20.18.1)
root@nasry-ahamadi:/home/nasry#

```

### Pour lister les versions installées localement

root@nasry-ahamadi:/home/nasry# nvm ls

```

root@nasry-ahamadi:/home/nasry# nvm ls
->      v20.18.1
default -> 20 (-> v20.18.1)
iojs -> N/A (default)
unstable -> N/A (default)
node -> stable (-> v20.18.1) (default)
stable -> 20.18 (-> v20.18.1) (default)
lts/* -> lts/jod (-> N/A)
lts/argon -> v4.9.1 (-> N/A)
lts/boron -> v6.17.1 (-> N/A)
lts/carbon -> v8.17.0 (-> N/A)
lts/dubnium -> v10.24.1 (-> N/A)
lts/erbium -> v12.22.12 (-> N/A)
lts/fermium -> v14.21.3 (-> N/A)
lts/gallium -> v16.20.2 (-> N/A)
lts/hydrogen -> v18.20.5 (-> N/A)
lts/iron -> v20.18.1
lts/jod -> v22.12.0 (-> N/A)
root@nasry-ahamadi:/home/nasry#

```

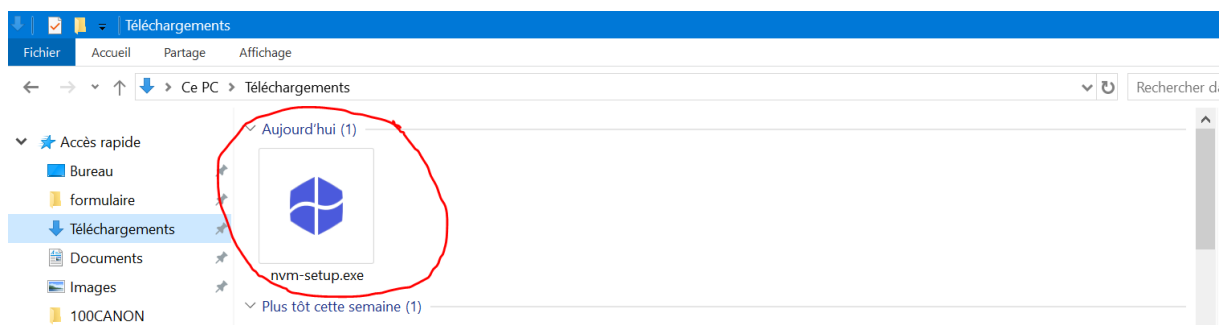
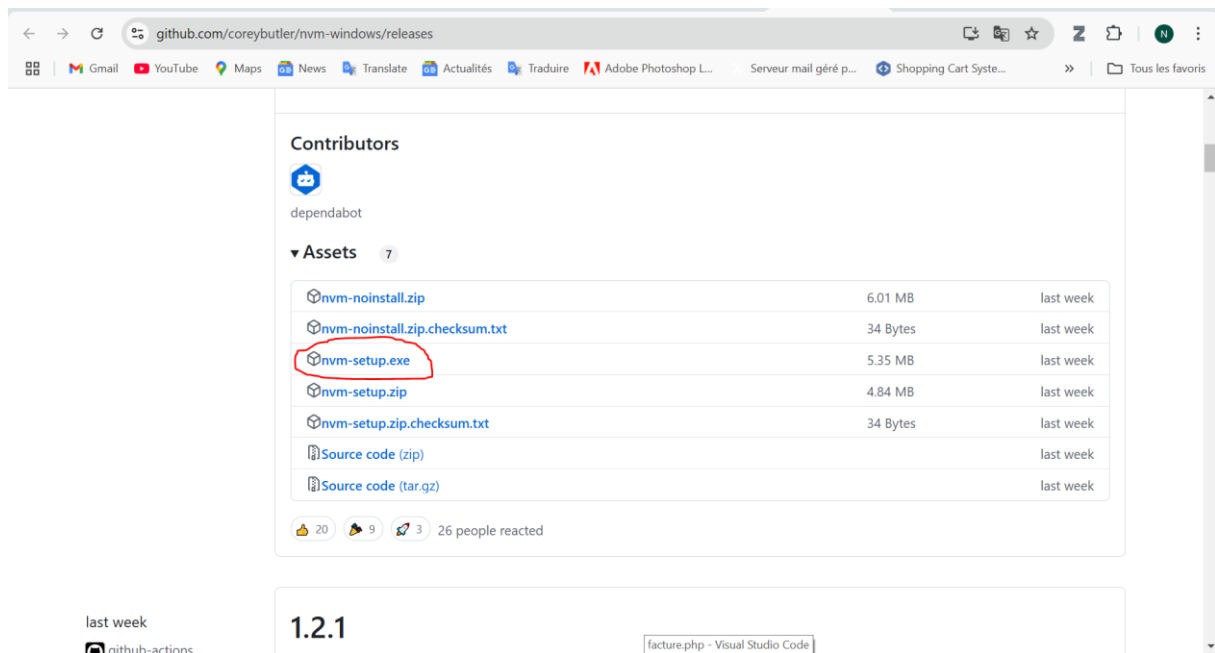
## 2.8.2 Installation de nvm sous Windows

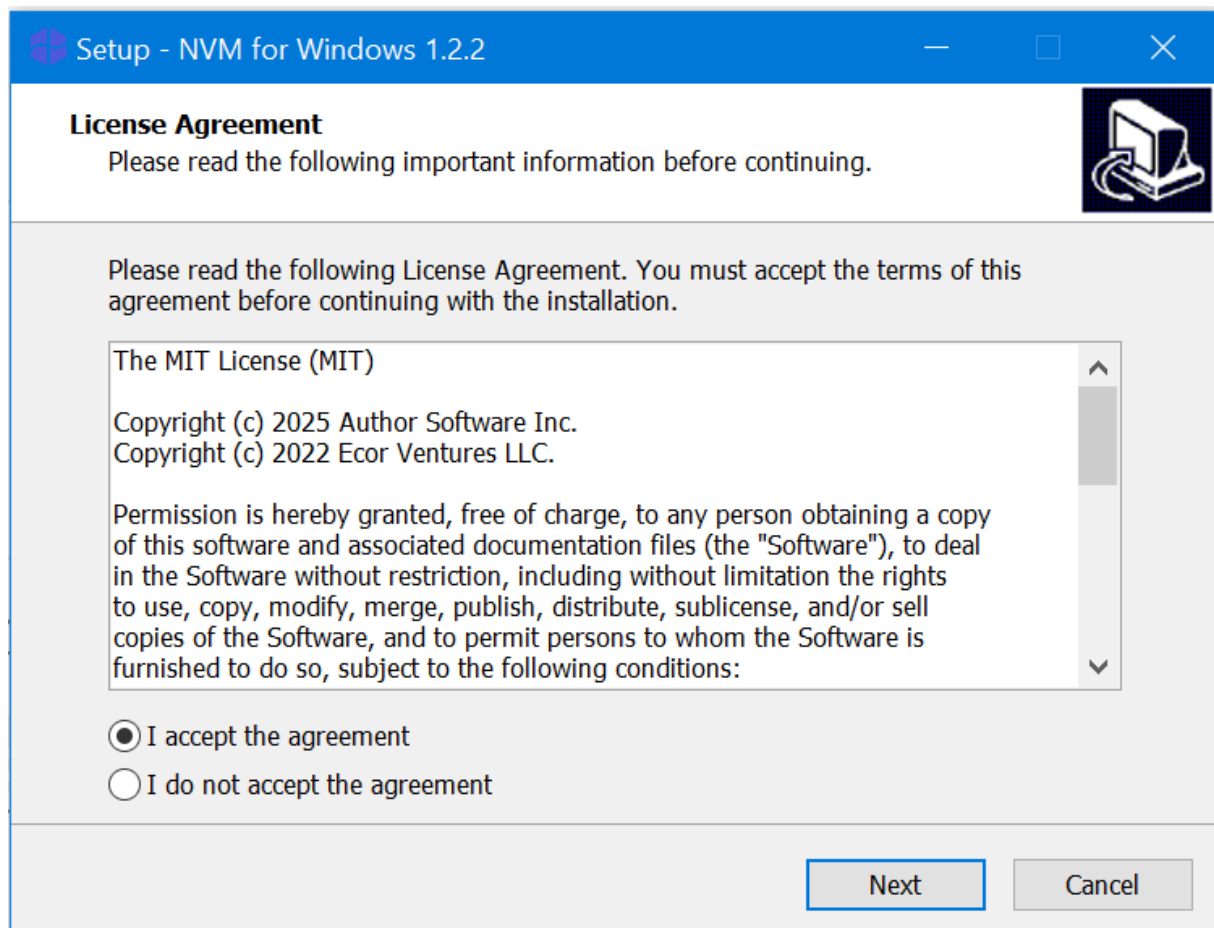
### Téléchargement de l'installateur NVM

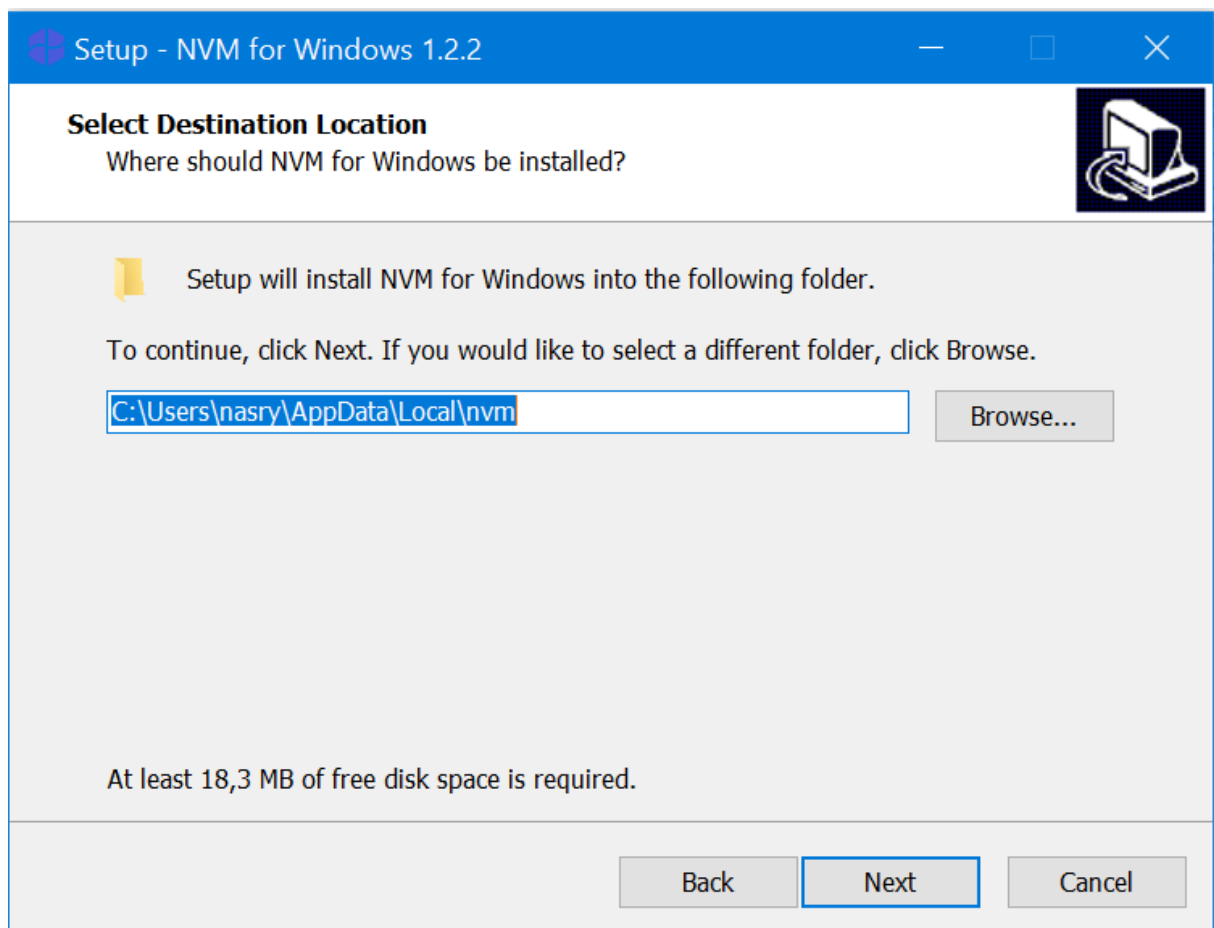
Rendez-vous sur le site officiel du NVM pour Windows et téléchargez l'installateur.

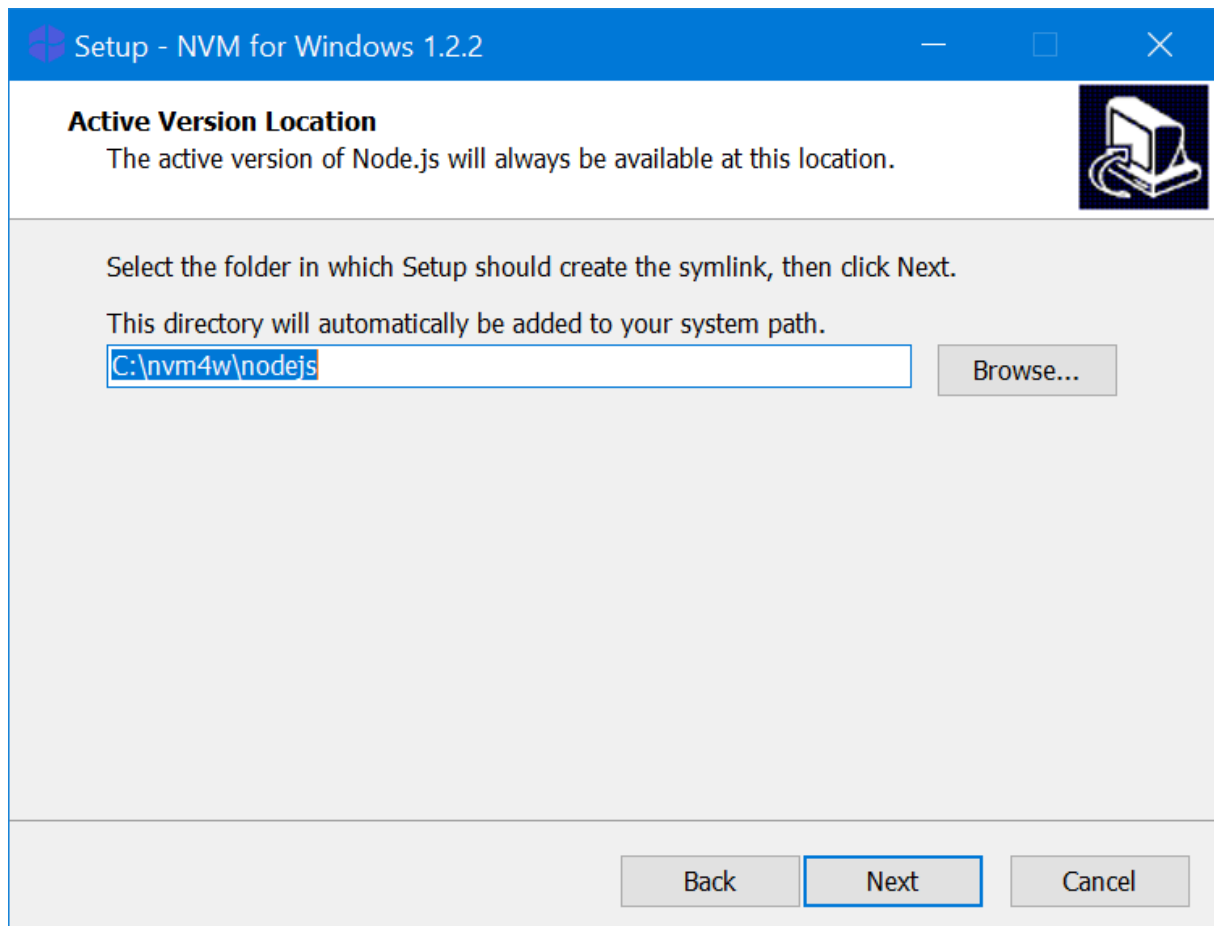
<https://github.com/coreybutler/nvm-windows/releases>

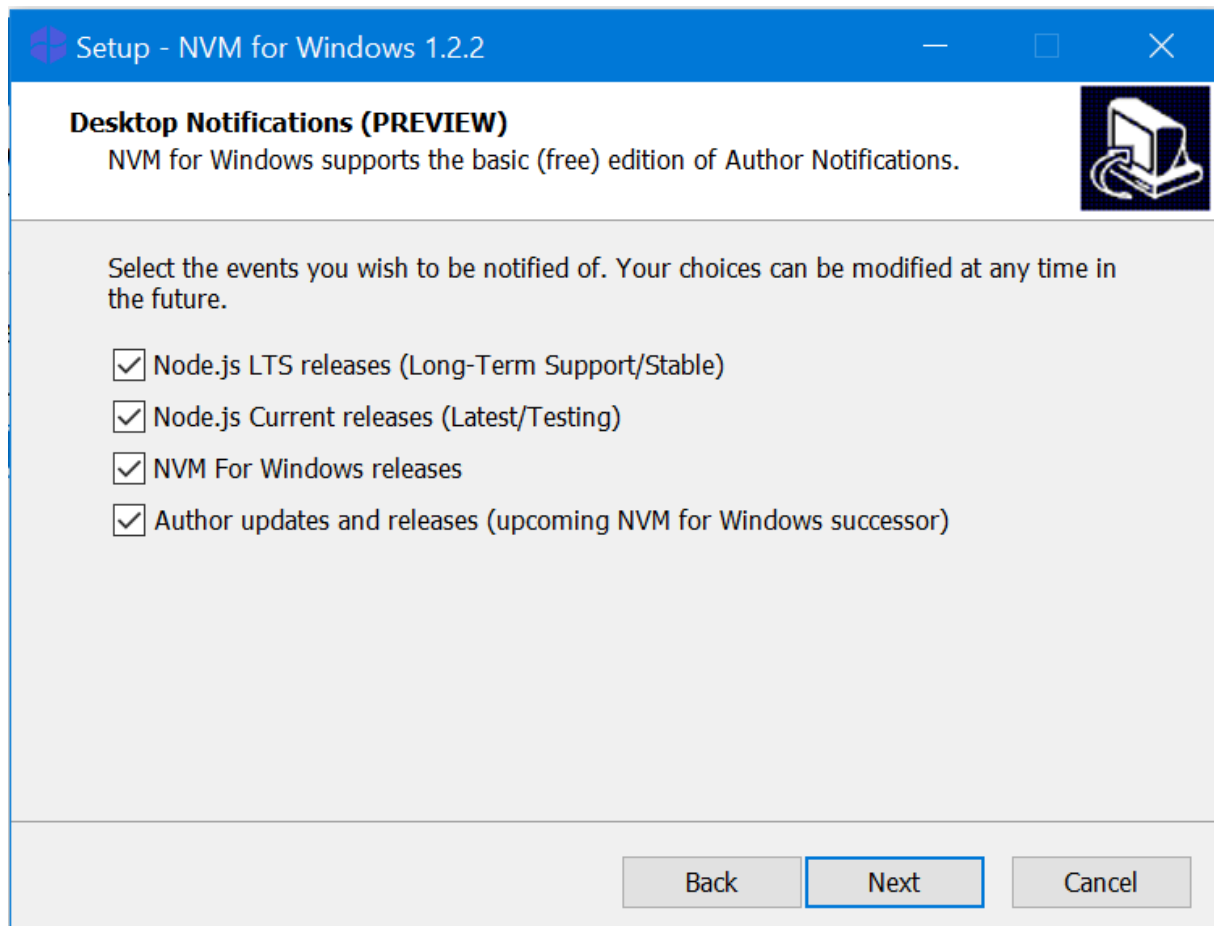












Setup - NVM for Windows 1.2.2

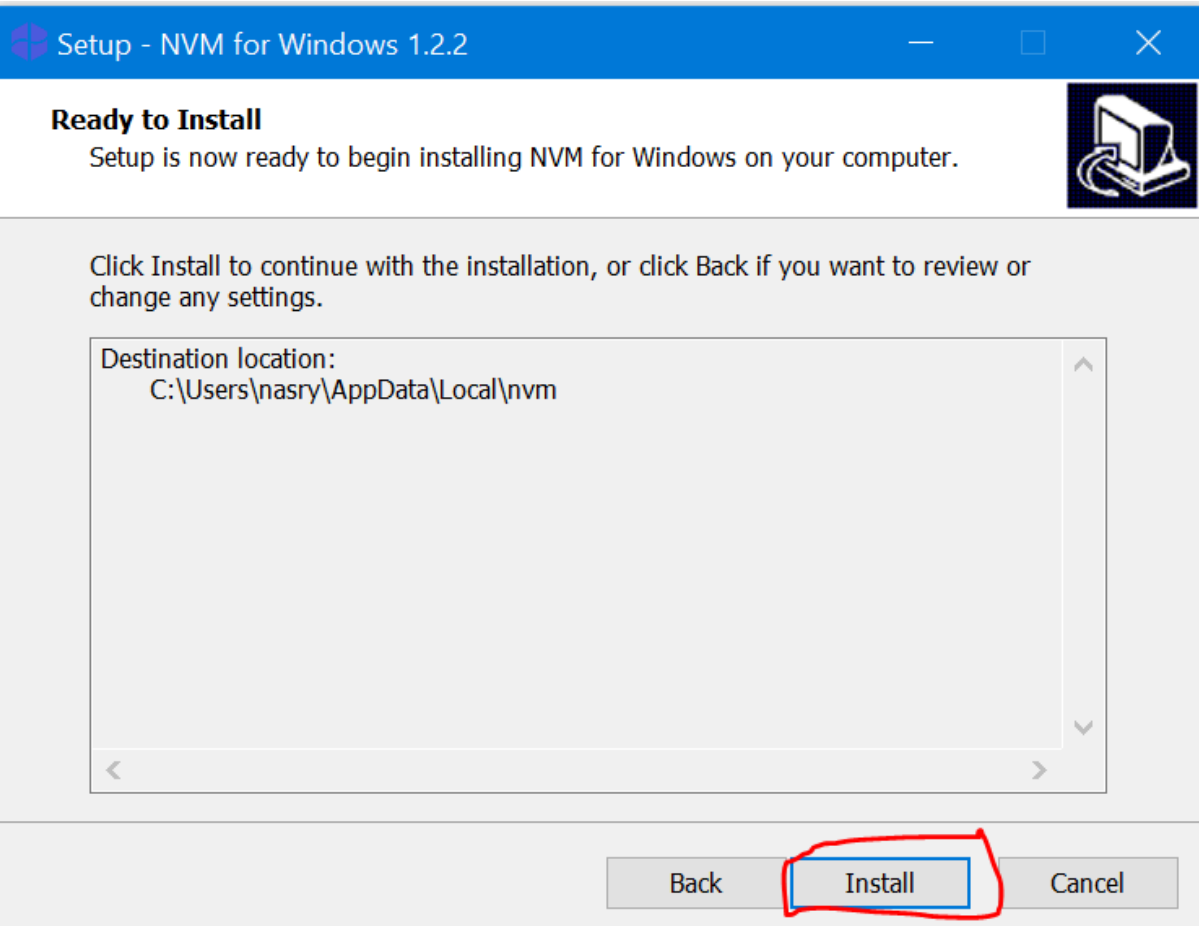
### Author Progress Email Signup

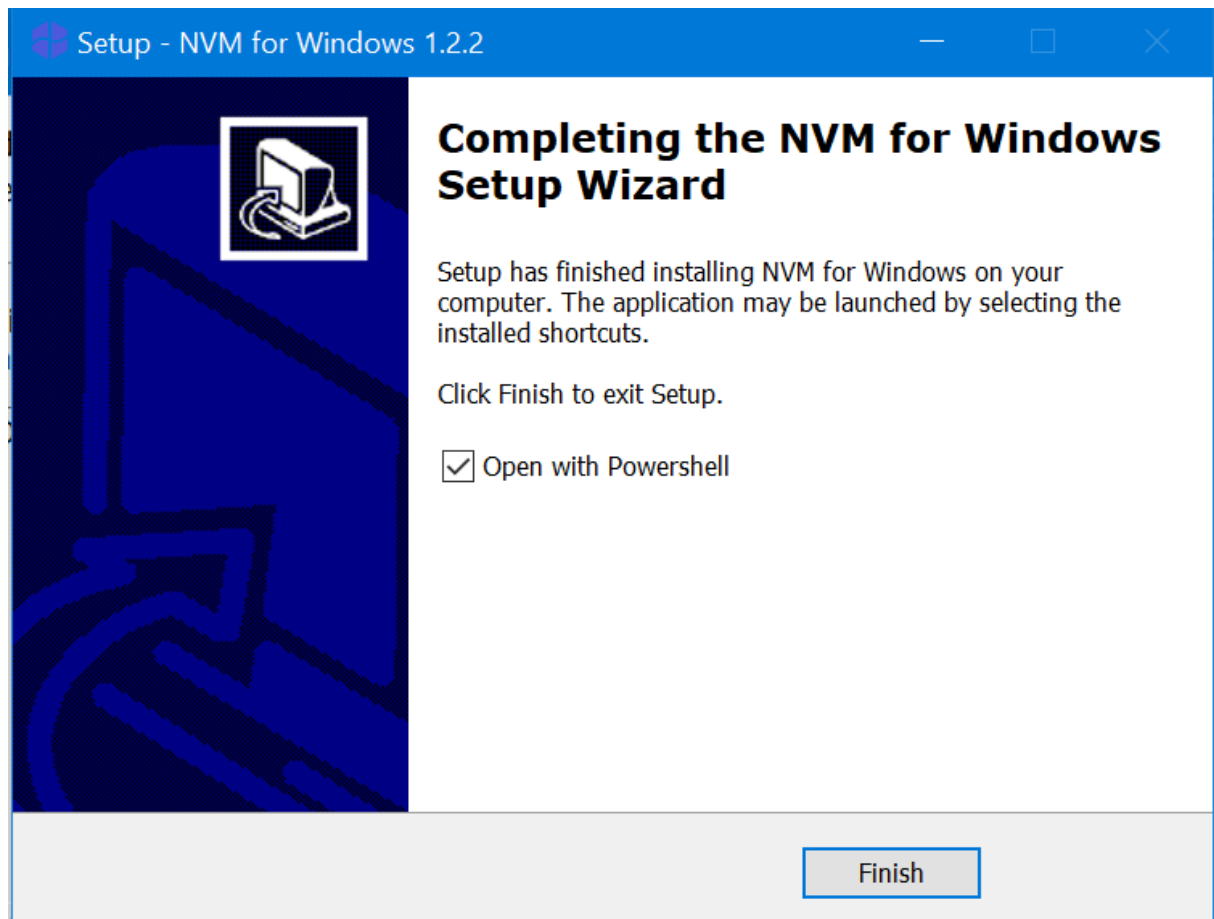
Get details about Author development milestones in your inbox.

Author is the upcoming successor to NVM for Windows. Provide your email address to be informed of development milestones, release timelines, and enterprise capabilities. Leave it blank if you do not wish to receive notifications.

**Email Address: (Optional)**

Back Next Cancel





```
PS C:\Users\nasry> nvm -v
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. Tous droits réservés.

Testez le nouveau système multiplateforme PowerShell https://aka.ms/pscore6

PS C:\Users\nasry> nvm -v
1.2.2
PS C:\Users\nasry> |
```



# Chapitre 3 : Les Fondamentaux de JavaScript pour Node.js

## 3.1 Présentation

JavaScript est un langage de programmation polyvalent qui constitue la base de Node.js. Pour maîtriser Node.js, il est essentiel de comprendre en profondeur les concepts fondamentaux de JavaScript et ses spécificités côté serveur.

## 3.2 Concepts de base JavaScript

### 3.2.1 Variables et Types de Données

#### 3.2.1.1 Définition des Variables

Une variable est un conteneur pour stocker des données dans la mémoire de l'ordinateur. En JavaScript, les variables sont faiblement typées (dynamically typed), ce qui signifie que le type d'une variable peut changer au cours de l'exécution du programme.

#### 3.2.1.2 Déclaration de Variables

Il existe trois mots-clés pour déclarer des variables :

- ✚ **let** : Permet de déclarer des variables dont la valeur peut être modifiée
- ✚ **const** : Permet de déclarer des constantes dont la valeur ne peut pas être modifiée
- ✚ **var** : Ancienne façon de déclarer des variables (à éviter dans le code moderne)

```
let age = 25;      // Mutable
const PI = 3.14159; // Immutable
var nom = "John";  // À éviter
```

### 3.2.2 Types de Données Primitifs

#### 3.2.2.1 Number

Représente à la fois les nombres entiers et décimaux. JavaScript n'a qu'un seul type numérique basé sur le standard IEEE 754 (nombres à virgule flottante double précision).

- ✚ **Caractéristiques :**
  - Peut représenter des entiers jusqu'à  $\pm 2^{53}$
  - Inclut les valeurs spéciales : Infinity, -Infinity, NaN

```
const entier = 42;

const decimal = 3.14;

const scientifique = 2.998e8;
```

### 3.2.2.2 String

Représente une séquence de caractères. Les chaînes sont immuables en JavaScript.

#### **Caractéristiques :**

- Peut utiliser des guillemets simples ou doubles
- Support des template literals avec backticks (`)
- Possède de nombreuses méthodes intégrées

```
const simple = 'Hello';

const template = `${simple} World`;
```

### 3.2.2.3 Boolean

Représente une valeur logique qui ne peut être que true ou false.

#### **Utilisations :**

- Contrôle de flux
- Conditions
- Opérations logiques

```
const estActif = true;

const estConnecte = false;
```

### 3.2.2.4 Null et Undefined

-  null : Représente l'absence délibérée de valeur
-  undefined : Représente une valeur non définie

## 3.2.3 Types de Données Complexes

### 3.2.3.1 Object

Collection de paires clé-valeur qui permet de stocker des données structurées.

#### **Caractéristiques :**

- Mutable

- Peut contenir n'importe quel type de données
- Supporte l'héritage prototypal

```
const personne = {
  nom: "Dupont",
  age: 30
};
```

### 3.2.3.2 Array

Collection ordonnée de valeurs accessibles par index.

#### **Caractéristiques :**

- Index basé sur zéro
- Peut contenir différents types de données
- Longueur dynamique

```
const nombres = [1, 2, 3];
```

## 3.2.4 Fonctions et Portée

### 3.2.4.1 Définition des Fonctions

Une fonction est un bloc de code réutilisable qui effectue une tâche spécifique. Les fonctions peuvent accepter des paramètres et retourner des valeurs.

### 3.2.4.2 Types de Fonctions

### 3.2.4.3 Déclaration de Fonction

Méthode traditionnelle de création de fonction avec le mot-clé function.

#### **Caractéristiques :**

- Hoisting (remontée de déclaration)
- Peut être nommée
- Crée son propre contexte this

```
function additionner(a, b) {
  return a + b;
}
```

### 3.2.4.4 Expression de Fonction

Fonction définie dans une expression, souvent assignée à une variable.

#### **Caractéristiques :**

- Pas de hoisting
- Peut être anonyme
- Plus flexible pour la programmation fonctionnelle

```
const multiplier = function(a, b) {  
  
    return a * b;  
  
};
```

### 3.2.4.5 Fonction Fléchée

Syntaxe plus concise introduite en ES6.

#### **Caractéristiques :**

- Pas de liaison this
- Retour implicite pour les expressions simples
- Idéale pour les callbacks courts

```
const diviser = (a, b) => a / b;
```

### 3.2.4.6 Portée (Scope)

La portée détermine l'accessibilité des variables dans différentes parties du code.

### 3.2.4.7 Types de Portée

#### **Portée Globale**

-  Variables accessibles partout dans le programme
-  Déclarées hors de toute fonction ou bloc

#### **Portée Locale (Function Scope)**

-  Variables accessibles uniquement dans la fonction où elles sont déclarées
-  Créée par les fonctions

#### **Portée de Bloc (Block Scope)**

-  Variables accessibles uniquement dans le bloc où elles sont déclarées

- ✚ Créée par les structures comme if, for, while

### 3.3 Spécificités JavaScript côté Serveur

#### 3.3.1 Différences entre JS Navigateur et Node.js

Node.js est utilisé pour exécuter du code JavaScript côté serveur, permettant de construire des applications web et de réseau performantes et évolutives. JavaScript est un langage de programmation (script) interprété, orienté objet et fonctionnel.

##### 3.3.1.1 Environnement d'Exécution

Contexte dans lequel le code JavaScript s'exécute.

##### Dans le Navigateur :

- ✚ Objet global : window
- ✚ Focus sur le DOM et les APIs Web
- ✚ Sécurité basée sur le même origine (same-origin policy)

##### Dans Node.js :

- ✚ Objet global : global
- ✚ Focus sur le système de fichiers et le réseau
- ✚ Accès complet au système d'exploitation

##### 3.3.1.2 APIs Disponibles

APIs Exclusives au Navigateur : Interfaces spécifiques à l'environnement navigateur.

```
// Non disponibles dans Node.js
```

```
window.localStorage
```

```
document.querySelector()
```

```
window.fetch
```

APIs Exclusives à Node.js : Modules et fonctionnalités spécifiques à l'environnement serveur.

```
// Disponibles uniquement dans Node.js
```

```
const fs = require('fs');
```

```
const os = require('os');
```

### 3.3.1.3 Gestion des Modules

#### Dans le Navigateur :

- ✚ Historiquement utilisait les balises <script>
- ✚ Support moderne des ES Modules avec type="module"
- ✚ Chargement asynchrone possible

#### Dans Node.js :

- ✚ Système de modules CommonJS natif
- ✚ Support des ES Modules avec extension .mjs
- ✚ Résolution de modules sophistiquée

## 3.3.2 CommonJS vs ES Modules

### 3.3.2.1 CommonJS

Système de modules original de Node.js, utilisant require() et module.exports.

#### ✚ Caractéristiques :

- Chargement synchrone
- Compatible avec tous les versions de Node.js
- Plus simple pour le debug

#### ✚ Syntaxe :

```
// Exportation
```

```
module.exports = {  
  maFonction: function() {},  
  maVariable: 42  
};
```

```
// Importation
```

```
const monModule = require('./monModule');
```

### 3.3.2.2 ES Modules

Standard moderne pour les modules JavaScript, utilisant import et export.

#### **Caractéristiques :**

- Chargement asynchrone
- Analyse statique possible
- Support natif dans les navigateurs modernes

#### **Syntaxe :**

```
// Exportation

export const maFonction = () => {};

export const maVariable = 42;


// Importation

import { maFonction, maVariable } from './monModule.js';
```

## 3.3.3 Le Système de Modules Node.js

### 3.3.3.1 Définition des Modules

**Un module en Node.js est :**

-  Une unité de code réutilisable
-  Un fichier JavaScript autonome
-  Un moyen d'encapsuler du code

### 3.3.3.2 Types de Modules

Modules Natifs (Core Modules) : Modules intégrés à Node.js.

```
const fs = require('fs');           // Système de fichiers

const http = require('http');       // Serveur HTTP

const path = require('path');       // Gestion des chemins

const events = require('events');   // Gestion des événements
```

Modules Locaux : Modules créés par le développeur.

```
// utils.js

module.exports = {

  formatDate: (date) => new Date(date).toLocaleDateString(),

  calculateAge: (birthDate) => {

    const today = new Date();

    const birth = new Date(birthDate);

    return today.getFullYear() - birth.getFullYear();

  }

};
```

Modules Tiers (npm) : Modules installés via npm.

```
const express = require('express'); // Framework web

const lodash = require('lodash'); // Utilitaires
```

### 3.3.3.3 Résolution des Modules

Processus par lequel Node.js trouve et charge les modules.

#### 3.3.3.4 Ordre de résolution :

-  Modules natifs
-  Chemins absolus
-  Chemins relatifs
-  node\_modules



# Chapitre 4 : Le Système de Modules Node.js

## 4.1 Présentation

Le système de modules est l'un des aspects fondamentaux de Node.js. Il permet d'organiser le code en unités réutilisables et maintenues séparément. Cette modularité est essentielle pour créer des applications scalables et maintenables.

## 4.2 Modules Natifs (Core Modules)

### 4.2.1 Qu'est-ce qu'un Module Natif ?

Les modules natifs sont des modules intégrés à Node.js, fournis avec l'installation de base. Ils constituent la bibliothèque standard de Node.js.

#### Caractéristiques :

- Préinstallés avec Node.js
- Optimisés pour les performances
- Stables et bien testés
- Documentation officielle complète

### 4.2.2 Principaux Modules Natifs

#### 4.2.2.1 Module fs (File System)

Fournit une API pour interagir avec le système de fichiers.

#### Fonctionnalités principales :

- Lecture/écriture de fichiers
- Manipulation de répertoires
- Gestion des permissions
- Surveillance de fichiers

```
const fs = require('fs');

// Version synchrone

try {

    const data = fs.readFileSync('fichier.txt', 'utf8');

    console.log(data);
```

```

} catch (err) {

    console.error('Erreur de lecture:', err);

}

```

```
// Version asynchrone

fs.readFile('fichier.txt', 'utf8', (err, data) => {

  if (err) {

    console.error('Erreur de lecture:', err);

    return;

  }

  console.log(data);

});
```

#### 4.2.2.2 Module path

Utilitaire pour manipuler les chemins de fichiers de manière cross-platform.

## Fonctionnalités principales :

- Résolution de chemins
- Manipulation de chemins
- Normalisation de chemins

```
const path = require('path');

// Manipulation de chemins

const cheminComplet = path.join(__dirname, 'dossier', 'fichier.txt');

console.log(cheminComplet);

// Extraction d'informations
```

```
console.log(path.basename('/chemin/vers/fichier.txt')); // fichier.txt

console.log(path.extname('fichier.txt')); // .txt

console.log(path.dirname('/chemin/vers/fichier.txt')); // /chemin/vers
```

#### 4.2.2.3 Module http/https

Permet de créer des serveurs HTTP et d'effectuer des requêtes HTTP.

##### **Fonctionnalités principales :**

- Création de serveurs web
- Gestion des requêtes et réponses
- Client HTTP

```
const http = require('http');

// Création d'un serveur simple

const server = http.createServer((req, res) => {

  res.writeHead(200, {'Content-Type': 'text/plain'});

  res.end('Hello World\n');

});

server.listen(3000, () => {

  console.log('Serveur démarré sur le port 3000');

});
```

### 4.2.3 Utilisation des Modules Built-in

#### 4.2.3.1 Bonnes Pratiques

##### **Importation en début de fichier**

```
const fs = require('fs');

const path = require('path');
```

```
const http = require('http');
```

#### **Gestion des erreurs**

```
// Utilisation de try-catch pour les opérations synchrones

try {

    // Opérations

} catch (err) {

    // Gestion des erreurs

}
```

```
// Gestion des erreurs pour les opérations asynchrones

operation((err, resultat) => {

    if (err) {

        // Gestion des erreurs

        return;

    }

    // Traitement du résultat

});
```

## 4.3 Gestion des Packages

### 4.3.1 npm

npm (Node Package Manager) est le gestionnaire de packages officiel de Node.js.

#### **Fonctionnalités principales :**

- Installation de packages
- Gestion des dépendances
- Publication de packages
- Scripts de développement

### 4.3.2 Installation et Configuration

# Installation globale d'un package

```
npm install -g package-name
```

# Installation locale (projet)

```
npm install package-name
```

# Installation avec sauvegarde dans dependencies

```
npm install package-name --save
```

# Installation avec sauvegarde dans devDependencies

```
npm install package-name --save-dev
```

### 4.3.3 package.json et ses Composants

Fichier de configuration qui décrit le projet et ses dépendances.

### 4.3.4 Structure de base

```
{  
  
  "name": "mon-projet",  
  
  "version": "1.0.0",  
  
  "description": "Description du projet",  
  
  "main": "index.js",  
  
  "scripts": {  
  
    "start": "node index.js",  
  
    "test": "jest",  
  
    "dev": "nodemon index.js"  
  
  },  
}
```

```
"dependencies": {  
  "express": "^4.17.1"  
},  
"devDependencies": {  
  "nodemon": "^2.0.7",  
  "jest": "^27.0.6"  
}  
}
```

## 4.3.5 Composants Principaux

### 4.3.5.1 Métadonnées

- ✚ name: Nom du package
- ✚ version: Version suivant SemVer
- ✚ description: Description du projet
- ✚ author: Auteur du package
- ✚ license: Licence du projet

### 4.3.5.2 Dépendances

- ✚ dependencies: Dépendances de production
- ✚ devDependencies: Dépendances de développement
- ✚ peerDependencies: Dépendances requises

### 4.3.5.3 Scripts

- ✚ Commandes personnalisées
- ✚ Automatisation des tâches

## 4.3.6 Gestion des Dépendances

### 4.3.6.1 Versions et SemVer

Système de versionnage sémantique (MAJOR.MINOR.PATCH)

```
"dependencies": {  
  "package1": "1.0.0", // Version exacte
```

```
"package2": "^1.0.0", // Compatible avec 1.x.x  
"package3": "~1.0.0", // Compatible avec 1.0.x  
"package4": "*"      // Dernière version  
}
```

### 4.3.7 Commandes Importantes

# Mise à jour des dépendances

```
npm update
```

# Vérification des mises à jour disponibles

```
npm outdated
```

# Installation à partir du package.json

```
npm install
```

# Nettoyage du cache

```
npm cache clean --force
```

### 4.3.8 Création et Publication de Packages

#### 4.3.8.1 Création d'un Package

 **Initialisation**

```
npm init -y
```

# Chapitre 5 : Gestionnaire de Paquets et Initialisation d'un Projet

## 5.1 Introduction à npm

## 5.2 Qu'est-ce qu'un gestionnaire de paquets ?

Un gestionnaire de paquets est un outil qui aide à gérer les bibliothèques, modules, et dépendances nécessaires pour un projet. Il permet :

- ✚ D'installer des paquets tiers facilement.
- ✚ De maintenir des versions cohérentes des dépendances.
- ✚ D'automatiser les scripts de construction et de test d'un projet.

## 5.3 npm (Node Package Manager)

### 5.3.1 Présentation

- ✚ npm est le gestionnaire de paquets officiel livré avec Node.js.
- ✚ Permet de publier, partager, installer et gérer des dépendances dans un projet Node.js.
- ✚ Associé au fichier package.json pour documenter les dépendances d'un projet.

### 5.3.2 Fonctionnalités Principales

- ✚ Installation de Packages: Installation locale ou globale.
- ✚ Gestion des Dépendances: via le fichier package.json.
- ✚ Exécution de Scripts: Scripts définis dans package.json.

### 5.3.3 Commandes essentielles de npm

### 5.3.4 Initialisation d'un projet :

```
root@nasry-ahamadi:/home/nasry# mkdir projet
```

```
root@nasry-ahamadi:/home/nasry# cd projet/
```

```
root@nasry-ahamadi:/home/nasry# mkdir projet
root@nasry-ahamadi:/home/nasry# cd projet/
root@nasry-ahamadi:/home/nasry/projet#
```

```
root@nasry-ahamadi:/home/nasry/projet# npm init -y
```



```

root@nasry-ahamadi:/home/nasry/projet# npm init -y
Wrote to /home/nasry/projet/package.json:

{
  "name": "projet",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": ""
}

root@nasry-ahamadi:/home/nasry/projet#

```

### 5.3.5 Installation d'un paquet :

Dépendances du projet (locales)

 **Syntaxe de base :**

```
npm install <nom_du_paquet>
```

L'option **--save** lors de l'installation d'un package avec `npm install` était utilisée pour ajouter le package à la section dependencies du fichier package.json. Cependant, depuis npm 5 (sorti en 2017), cette option est devenue implicite et n'est plus nécessaire. Par défaut, les packages installés avec `npm install` sont automatiquement ajoutés à la section dependencies.

 **Exemple d'installation de paquet :**

```
root@nasry-ahamadi:/home/nasry/projet# npm install express
```

```

root@nasry-ahamadi:/home/nasry/projet# npm install express
up to date, audited 70 packages in 996ms

14 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
root@nasry-ahamadi:/home/nasry/projet#

```

Dépendances globales

 **Syntaxe de base :**

```
npm install -g <nom_du_paquet>
```

✚ Exemple d'installation de paquet :

```
root@nasry-ahamadi:/home/nasry/projet# npm install -g express
added 69 packages in 1s
14 packages are looking for funding
  run `npm fund` for details
root@nasry-ahamadi:/home/nasry/projet#
```

### 5.3.6 Pour lister les paquets installés :

✚ Pour les paquets locaux

```
root@nasry-ahamadi:/home/nasry/projet# npm list
```

```
root@nasry-ahamadi:/home/nasry/projet# npm list
projet@1.0.0 /home/nasry/projet
└─ express@4.21.2

root@nasry-ahamadi:/home/nasry/projet#
```

✚ Pour les paquets globaux

```
root@nasry-ahamadi:/home/nasry/projet# npm list -g
```

```
root@nasry-ahamadi:/home/nasry/projet# npm list -g
/root/.npm/versions/node/v20.18.1/lib
├─ corepack@0.29.4
├─ express@4.21.2
└─ npm@10.8.2

root@nasry-ahamadi:/home/nasry/projet#
```

### 5.3.7 Suppression un paquet :

✚ Syntaxe de base :

```
npm uninstall <nom_du_paquet>
```

✚ Exemple de désinstallation de paquet :

```
root@nasry-ahamadi:/home/nasry/projet# npm uninstall express
```

```
root@nasry-ahamadi:/home/nasry/projet# npm uninstall express
removed 69 packages, and audited 1 package in 521ms
found 0 vulnerabilities
root@nasry-ahamadi:/home/nasry/projet#
```

root@nasry-ahamadi:/home/nasry/projet# npm list

```
root@nasry-ahamadi:/home/nasry/projet# npm list
projet@1.0.0 /home/nasry/projet
└─ (empty)

root@nasry-ahamadi:/home/nasry/projet#
```

### 5.3.8 Les scripts npm

Le fichier package.json peut inclure des scripts pour automatiser les tâches.

Dans un fichier package.json pour un projet Node.js, la section "scripts" permet de définir des commandes personnalisées que vous pouvez exécuter avec npm run.

 **Exemple :**

```
"scripts": {
  "start": "node app.js"
}
```

 **Explication :**

**start:**

- La commande associée à "start" est node app.js.
- Cela signifie qu'exécuter npm start lancera l'application en exécutant le fichier app.js avec Node.js.

 **Commande pour exécuter :**

```
npm run start
```

### 5.3.9 Avantages de npm

-  Large bibliothèque de modules disponibles (plus d'un million).
-  Facile à utiliser avec Node.js dès l'installation.
-  Grande communauté et support.

- ✚ Contrôle des Versions: Gestion précise des versions.
- ✚ Adoption Étendue: Largement utilisé dans la communauté JS.

### 5.3.10 Inconvénients de npm

- ✚ Gestion des Globales: Conflits potentiels entre projets.
- ✚ Taille du Node\_modules: Dossiers volumineux.

## 5.4 Initialisation d'un Projet Node.js

- ✚ Création d'un nouveau projet avec npm

```
root@nasry-ahamadi:/home/nasry# mkdir projet
```

```
root@nasry-ahamadi:/home/nasry# cd projet/
```

```
root@nasry-ahamadi:/home/nasry# mkdir projet
root@nasry-ahamadi:/home/nasry# cd projet/
root@nasry-ahamadi:/home/nasry/projet#
```

- ✚ Initialisation du projet :

```
root@nasry-ahamadi:/home/nasry/projet# npm init -y
```

```
root@nasry-ahamadi:/home/nasry/projet# npm init -y
Wrote to /home/nasry/projet/package.json:

{
  "name": "projet",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": ""
}

root@nasry-ahamadi:/home/nasry/projet#
```

- ✚ Contenu du fichier package.json généré :

```
root@nasry-ahamadi:/home/nasry/projet# cat package.json
```

```

root@nasry-ahamadi:/home/nasry/projet# cat package.json
{
  "name": "projet",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": ""
}
root@nasry-ahamadi:/home/nasry/projet# █

```

### 🔧 Configuration des scripts personnalisés

Ajout d'un script pour démarrer l'application :

```
root@nasry-ahamadi:/home/nasry/projet# nano package.json
```

```

root@nasry-ahamadi:/home/nasry/projet# nano package.json
root@nasry-ahamadi:/home/nasry/projet# █

```

```

GNU nano 4.8                                package.json
{
  "name": "projet",
  "version": "1.0.0",
  "main": "app.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "node app.js" ←
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": ""
}

```

## 5.5 Structure Complète d'un Fichier app.js Simplifié dans nodejs

### 5.5.1 Exemple de fichier app.js :

#### 1) Importation du module Express

```
const express = require('express');
```

## 2) Initialisation de l'application

```
const app = express();
```

## 3) Configuration des middlewares de base

```
app.use(express.json());           // Pour parser les requêtes JSON  
app.use(express.urlencoded({ extended: true })); // Pour parser les données encodées dans  
l'URL
```

## 4) Définition des routes

```
app.get('/', (req, res) => {  
    res.send('Bienvenue dans notre application Node.js simple !');  
});
```

```
app.get('/about', (req, res) => {  
    res.send('Ceci est une application de démonstration simple.');
```

## 5) Gestion des erreurs 404

```
app.use((req, res) => {  
    res.status(404).send('Erreur 404 : La ressource demandée est introuvable.');
```

## 6) Démarrage du serveur

```
const PORT = 3000; // Port défini pour l'application  
app.listen(PORT, () => {  
    console.log(`Le serveur fonctionne sur http://localhost:${PORT}`);  
});
```

## 5.5.2 Explications Détail par Section

### 1) Importation du Module Express

Pourquoi ? Express simplifie la création et la gestion des routes, et fournit des fonctionnalités prêtes à l'emploi pour les applications web.

## 2) Initialisation de l'Application

Créez une instance d'Express, qui servira de base pour définir les fonctionnalités et les routes de l'application.

## 3) Configuration des Middlewares

`express.json()` : Permet de traiter les données envoyées au format JSON.

`express.urlencoded({ extended: true })` : Permet de traiter les données encodées dans l'URL, comme celles des formulaires HTML.

## 4) Définition des Routes

Une route basique GET / retourne un message de bienvenue.

Une autre route GET /about fournit des informations sur l'application.

## 5) Gestion des Erreurs 404

Si aucune route ne correspond à la requête, une réponse 404 est renvoyée avec un message clair.

## 1) Démarrage du Serveur

Le serveur démarre sur le port 3000.

Une confirmation est affichée dans la console pour indiquer que le serveur est prêt.

### 5.5.3 Exemple pratique :

root@nasry-ahamadi:/home/nasry/projet# npm install express

```
root@nasry-ahamadi:/home/nasry/projet# npm install express
added 69 packages, and audited 70 packages in 9s

14 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
root@nasry-ahamadi:/home/nasry/projet#
```

La commande `npm install express` est utilisée pour installer le framework Express.js dans un projet Node.js. Voici une explication détaillée de cette commande et de son fonctionnement :

### 5.5.4 Décomposition de la commande

- ✚ **npm** : C'est le gestionnaire de paquets officiel de Node.js. Il permet d'installer, de gérer et de partager des packages (bibliothèques ou outils) pour des projets Node.js.
- ✚ **install** : C'est une commande de npm qui permet d'installer un package. Elle peut être abrégée en `i` (par exemple, `npm i express`).
- ✚ **express** : C'est le nom du package à installer. Ici, il s'agit du framework Express.js.

### 5.5.5 Où est installé Express.js ?

- ✚ **Dossier `node_modules`** : Tous les packages installés via npm sont stockés dans ce dossier. Express.js et ses dépendances seront donc disponibles dans `node_modules/express`.
- ✚ **Fichier `package-lock.json`** : Ce fichier est automatiquement généré ou mis à jour pour enregistrer les versions exactes des packages installés, garantissant la reproductibilité du projet.

root@nasry-ahamadi:/home/nasry/projet# nano app.js

```
root@nasry-ahamadi:/home/nasry/projet# nano app.js
root@nasry-ahamadi:/home/nasry/projet#
```

```
const express = require('express');

const app = express();

app.get('/', (req, res) => res.send('Bonjour Node.js !'));

app.listen(3000, () => console.log('Serveur lancé sur http://localhost:3000'));
```

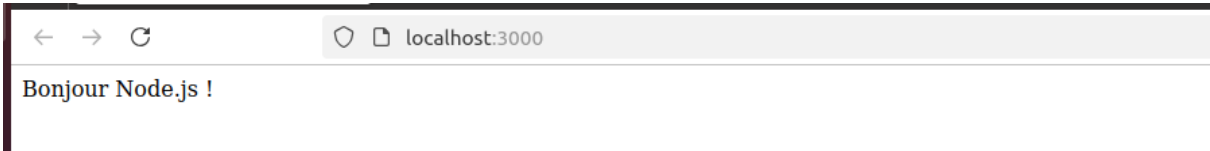
| GNU nano 4.8                                                                                                                                                                                                             | app.js | Modifié |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|---------|
| <pre>const express = require('express');  const app = express();  app.get('/', (req, res) =&gt; res.send('Bonjour Node.js !')); app.listen(3000, () =&gt; console.log('Serveur lancé sur http://localhost:3000'));</pre> |        |         |



```
root@nasry-ahamadi:/home/nasry/projet# node app.js
```

```
root@nasry-ahamadi:/home/nasry/projet# node app.js
Serveur lancé sur http://localhost:3000
```

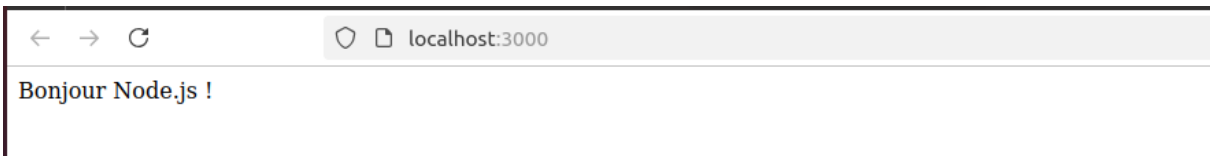
http://localhost:3000/



```
root@nasry-ahamadi:/home/nasry/projet# npm start
```

```
root@nasry-ahamadi:/home/nasry/projet# npm start
> projet@1.0.0 start
> node app.js
Serveur lancé sur http://localhost:3000
```

http://localhost:3000/



### 5.5.6 Exemple 2 :

```
root@nasry-ahamadi:/home/nasry/projet# nano app.js
```

```
root@nasry-ahamadi:/home/nasry/projet# nano app.js
root@nasry-ahamadi:/home/nasry/projet#
```

```

GNU nano 4.8 app.js
const express = require('express');

const app = express();

app.use(express.json()); // Pour parser les requêtes JSON
app.use(express.urlencoded({ extended: true })); // Pour parser les données encodées dans l'URL

app.get('/', (req, res) => {
  res.send('Bienvenue dans notre application Node.js simple !');
});

app.get('/about', (req, res) => {
  res.send('Ceci est une application de démonstration simple.');
```

### Lancer l'application :

root@nasry-ahamadi:/home/nasry/projet# node app.js

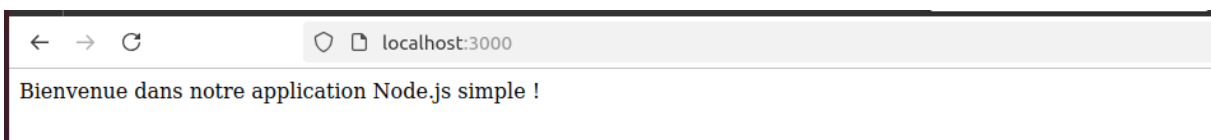
```

root@nasry-ahamadi:/home/nasry/projet# node app.js
Le serveur fonctionne sur http://localhost:3000
```

### Accéder au serveur :

Ouvrez votre navigateur et allez sur <http://localhost:3000>.

<http://localhost:3000/>



A screenshot of a web browser window. The address bar shows 'localhost:3000'. The page content displays the text: 'Bienvenue dans notre application Node.js simple !'.

<http://localhost:3000/about>



A screenshot of a web browser window. The address bar shows 'localhost:3000/about'. The page content displays the text: 'Ceci est une application de démonstration simple.'

<http://localhost:3000/test>



# Chapitre 6 : Framework Express.js

## 6.1 Présentation d'Express.js

Express.js est un framework web minimaliste et flexible pour Node.js, conçu pour faciliter le développement d'applications web et d'API. Il offre une structure légère et des fonctionnalités puissantes pour créer des serveurs web rapidement et efficacement.

## 6.2 Définition

Express.js est un framework web minimaliste et flexible conçu pour Node.js. Il est utilisé pour créer des applications web, des API REST, et des applications en temps réel. Il offre des outils et des fonctionnalités puissantes tout en laissant une grande liberté au développeur pour structurer son code.

## 6.3 Fonctionnalités principales :

- ✚ **Création de routes simples ou complexes** : Express permet de gérer facilement les différents chemins URL et méthodes HTTP (GET, POST, PUT, DELETE, etc.).
- ✚ **Gestion des middlewares** : Les middlewares permettent de traiter les requêtes et les réponses, ajoutant des fonctionnalités comme la journalisation, la validation, ou encore la gestion des erreurs.
- ✚ **Support des moteurs de templates** : Express peut s'intégrer à des moteurs comme EJS, Pug ou Handlebars pour générer des pages dynamiques.
- ✚ **Extensibilité** : Une vaste bibliothèque de middlewares tiers (comme body-parser, helmet, et cors) est disponible pour étendre les fonctionnalités.
- ✚ **Compatibilité** : Express est compatible avec toutes les versions modernes de Node.js.

Exemple minimal d'application avec Express.js :

Voici un exemple de code montrant la simplicité d'Express :

```
const express = require('express');

const app = express();

// Route principale
```

```

app.get('/', (req, res) => {

    res.send('Bienvenue sur mon serveur Express.js !');

});

// Lancement du serveur

app.listen(3000, () => {

    console.log('Serveur démarré sur http://localhost:3000');

});

```

## 6.4 Pourquoi utiliser Express.js ?

### ✚ Structure claire pour organiser le code :

- Express fournit une manière organisée de gérer les routes, les middlewares, et les fichiers statiques.
- Il facilite la séparation logique entre les différentes parties d'une application (routes, middlewares, contrôleurs, modèles).

### ✚ Exemple : Organisation du projet

```

mon-projet/
├── app.js                # Point d'entrée principal
├── routes/              # Fichiers pour gérer les routes
│   ├── utilisateurs.js
│   └── produits.js
├── middlewares/         # Fichiers pour les middlewares personnalisés
├── views/               # Modèles pour générer des pages HTML (avec EJS,
ejs, etc.)
├── public/              # Fichiers statiques (CSS, JS, images)
└── package.json

```

### ✚ Support des middlewares pour gérer les requêtes et les réponses :

Qu'est-ce qu'un middleware ?

Un middleware est une fonction qui s'exécute entre la réception d'une requête HTTP et l'envoi d'une réponse. Il peut être utilisé pour :

- ✚ Valider les données de la requête.
- ✚ Ajouter des en-têtes de sécurité.

- ✚ Gérer les erreurs globales.

### Exemple de middleware :

```
app.use((req, res, next) => {  
  
  console.log(`Requête reçue : ${req.method} ${req.url}`);  
  
  next(); // Passe au middleware suivant  
  
});
```

### Large écosystème et communauté active :

- ✚ Écosystème riche : Une multitude de modules et middlewares tiers sont disponibles pour répondre à des besoins spécifiques (par exemple, l'authentification, la gestion des fichiers ou la compression).
- ✚ Communauté active : Une grande communauté signifie des mises à jour régulières, une documentation complète, et de nombreux tutoriels ou forums pour apprendre.

### Adapté à toutes les tailles de projet :

- ✚ Petits projets : Express est minimaliste, ce qui permet de démarrer rapidement avec un projet simple.
- ✚ Grands projets : Grâce à sa flexibilité, Express peut être organisé en suivant les bonnes pratiques (MVC, microservices, etc.), ce qui le rend adapté aux applications complexes.

## 6.5 Pourquoi Express.js est essentiel

- ✚ Simplicité et rapidité : Express réduit la complexité de développement tout en offrant des performances optimales.
- ✚ Flexibilité : Que vous construisiez une application web interactive ou une API REST complexe, Express s'adapte facilement.
- ✚ Extensibilité : Avec des middlewares intégrés et tiers, vous pouvez personnaliser votre application selon vos besoins spécifiques.

## Chapitre 7 : Les Middlewares dans Node.js

Ce chapitre explore le concept des middlewares dans Node.js, leur rôle essentiel dans les applications web, et leur utilisation avec Express.js. Il inclut des exemples pratiques et des cas concrets pour mieux comprendre leur fonctionnement.

### 7.1 Concepts des Middlewares

#### 7.1.1 Définition et Rôle des Middlewares

Un middleware est une fonction qui intercepte les requêtes HTTP entrantes et/ou les réponses HTTP sortantes. Il peut :

- ✚ Modifier la requête ou la réponse.
- ✚ Effectuer une opération avant de passer au prochain middleware ou à la route cible.
- ✚ Terminer le cycle de requête-réponse.

#### 7.1.2 Métaphore pour Comprendre un Middleware : Le Contrôle de Sécurité à l'Entrée d'un Bâtiment

Imagine un grand immeuble d'entreprise avec plusieurs bureaux à l'intérieur. Chaque bureau représente une route dans ton application (par exemple : /public, /private). Les visiteurs veulent entrer dans le bâtiment pour accéder à ces bureaux.

#### 7.1.3 Les Éléments de la Métaphore :

- ✚ Le visiteur = la requête HTTP (request) venant du client.
- ✚ Les bureaux = les routes de ton application (ex : /home, /dashboard).
- ✚ Le garde de sécurité à l'entrée = le middleware.
- ✚ Le badge d'accès = le token d'authentification.
- ✚ Les employés dans les bureaux = les contrôleurs qui répondent à la requête.

#### 7.1.4 Comment ça Fonctionne ?

- ✚ **Un visiteur arrive à l'entrée (une requête HTTP est envoyée).**
- ✚ **Le garde de sécurité (le middleware) vérifie :**

A-t-il un badge ? (Vérification du token)

Est-ce que le badge est valide ? (Validation des données)

- ✚ **Si le badge est correct :**

Le garde dit : « Parfait, tu peux entrer » et il laisse passer le visiteur (appel de next()).

✚ Si le badge est incorrect ou absent :

Le garde bloque le visiteur et dit : « Désolé, tu ne peux pas entrer » (le middleware renvoie une erreur sans passer à la route).

### 7.1.5 Application au Code d'Exemple :

- ✚ La route /public = une zone d'accès libre (comme un hall d'entrée). Pas besoin de badge.
- ✚ La route /private = un bureau sécurisé. Tu dois présenter ton badge au garde.
- ✚ Le middleware authMiddleware = le garde de sécurité qui vérifie si tu es autorisé à entrer.

### 7.1.6 Structure de Base d'un Middleware :

```
function middlewareExample(req, res, next) {  
  
  // Logique du middleware  
  
  console.log('Middleware exécuté');  
  
  next(); // Passe au middleware suivant ou à la route  
  
}
```

### 7.1.7 Types de Middlewares

✚ **Général :**

- S'applique à toutes les requêtes.
- Exemple : journalisation des requêtes.

✚ **Spécifique :**

- S'applique uniquement à certaines routes.
- Exemple : vérification d'authentification.

✚ **Middleware d'erreur :**

- Gère les erreurs dans l'application.
- Exemple : capture des exceptions.

### 7.1.8 Exemple : Middleware d'erreur :

```
function errorHandler(err, req, res, next) {  
  
  console.error(err.stack);  
  
}
```



```
res.status(500).send('Quelque chose s'est mal passé !');  
}
```

## 7.2 Utilisation des Middlewares avec Express.js

Express.js est un framework Node.js populaire qui facilite la gestion des middlewares.

### 7.2.1 Middlewares Intégrés

-  **express.json** : Parse les corps des requêtes en JSON.
-  **express.urlencoded** : Parse les corps des requêtes encodés en URL.

### 7.2.2 Exemple : Utilisation des Middlewares Intégrés :

```
const express = require('express');  
  
const app = express();  
  
// Middleware intégré pour parser les requêtes JSON  
app.use(express.json());  
  
// Middleware intégré pour parser les données URL-encoded  
app.use(express.urlencoded({ extended: true }));  
  
app.post('/data', (req, res) => {  
  console.log(req.body); // Affiche les données reçues  
  res.send('Données reçues avec succès');  
});  
  
app.listen(3000, () => console.log('Serveur en écoute sur le port 3000'));
```

### 7.2.3 Middlewares Tiers

-  **morgan** : Logger HTTP.
-  **helmet** : Renforce la sécurité des en-têtes HTTP.

 **cors** : Gère les requêtes Cross-Origin.

Voici une liste des middlewares tiers les plus utilisés dans les projets Node.js avec Express.js, accompagnée d'une brève description et d'exemples d'utilisation :

### 7.2.3.1 Morgan

Middleware de journalisation HTTP. Il enregistre les détails des requêtes entrantes, utiles pour le débogage.

 **Installation :**

```
npm install morgan
```

 **Utilisation :**

```
const morgan = require('morgan');  
  
app.use(morgan('dev')); // Format 'dev' pour une sortie lisible
```

### 7.2.3.2 Helmet

Améliore la sécurité des applications en configurant des en-têtes HTTP sécurisés.

 **Installation :**

```
npm install helmet
```

 **Utilisation :**

```
const helmet = require('helmet');  
  
app.use(helmet());
```

### 7.2.3.3 CORS (Cross-Origin Resource Sharing)

Gère les problèmes liés aux requêtes entre origines (Cross-Origin).

 **Installation :**

```
npm install cors
```

 **Utilisation :**

```
const cors = require('cors');  
  
app.use(cors()); // Autorise toutes les origines
```

### 7.2.3.4 Body-Parser

Middleware pour analyser les corps des requêtes HTTP (deprecated mais souvent intégré dans Express depuis `express.json()` et `express.urlencoded()`).

 **Installation :**

```
npm install body-parser
```

 **Utilisation :**

```
const bodyParser = require('body-parser');  
  
app.use(bodyParser.json()); // Parse les requêtes JSON  
  
app.use(bodyParser.urlencoded({ extended: true })); // Parse les données URL-encoded
```

### 7.2.3.5 Compression

Middleware pour compresser les réponses HTTP, réduisant leur taille et améliorant les performances.

 **Installation :**

```
npm install compression
```

 **Utilisation :**

```
const compression = require('compression');  
  
app.use(compression());
```

### 7.2.3.6 Multer

Middleware pour gérer le téléchargement de fichiers.

 **Installation :**

```
npm install multer
```

 **Utilisation :**

```
const multer = require('multer');  
  
const upload = multer({ dest: 'uploads/' }); // Répertoire de stockage
```

```
app.post('/upload', upload.single('file'), (req, res) => {  
  
    res.send('Fichier téléchargé avec succès');  
  
});
```

### 7.2.3.7 Express-Rate-Limit

Limite le nombre de requêtes pour éviter les abus.

 **Installation :**

```
npm install express-rate-limit
```

 **Utilisation :**

```
const rateLimit = require('express-rate-limit');  
  
const limiter = rateLimit({  
  
    windowMs: 15 * 60 * 1000, // 15 minutes  
  
    max: 100 // Limite à 100 requêtes par IP  
  
});  
  
app.use(limiter);
```

### 7.2.3.8 Cookie-Parser

Analyse les cookies des requêtes HTTP.

 **Installation :**

```
npm install cookie-parser
```

 **Utilisation :**

```
const cookieParser = require('cookie-parser');  
  
app.use(cookieParser());
```

### 7.2.3.9 Express-Validator

Middleware pour valider et normaliser les entrées utilisateur.

 **Installation :**

```
npm install express-validator
```

 **Utilisation :**

```
const { body, validationResult } = require('express-validator');

app.post('/data',

  body('email').isEmail(),

  (req, res) => {

    const errors = validationResult(req);

    if (!errors.isEmpty()) {

      return res.status(400).json({ errors: errors.array() });

    }

    res.send('Données valides');

  }

);
```

#### 7.2.3.10 Passport

Middleware pour l'authentification, prend en charge de nombreuses stratégies (JWT, OAuth, etc.).

 **Installation :**

```
npm install passport
```

 **Utilisation :**

```
const passport = require('passport');

app.use(passport.initialize());
```

#### 7.2.3.11 Jsonwebtoken (JWT)

Gère les tokens JWT pour l'authentification sécurisée.

 **Installation :**

```
npm install jsonwebtoken
```

#### Utilisation :

```
const jwt = require('jsonwebtoken');

const secretKey = 'secret';

// Générer un token

const token = jwt.sign({ userId: 1 }, secretKey, { expiresIn: '1h' });

// Vérifier un token

jwt.verify(token, secretKey, (err, decoded) => {

  if (err) {

    console.log('Token invalide');

  } else {

    console.log('Token valide', decoded);

  }

});
```

#### 7.2.3.12 Winston

Middleware de gestion des logs, plus flexible et puissant que console.log.

#### Installation :

```
npm install winston
```

#### Utilisation :

```
const winston = require('winston');

const logger = winston.createLogger({

  level: 'info',

  transports: [
```

```

    new winston.transports.Console(),

    new winston.transports.File({ filename: 'app.log' })

  ]
});

logger.info('Message d\'information');

```

### 7.2.3.13 Session

Middleware pour gérer les sessions utilisateur.

 **Installation :**

```
npm install express-session
```

 **Utilisation :**

```

const session = require('express-session');

app.use(session({

  secret: 'secret-key',

  resave: false,

  saveUninitialized: true

}));

```

### 7.2.3.14 CSURF

Middleware pour protéger les applications contre les attaques CSRF (Cross-Site Request Forgery).

 **Installation :**

```
npm install csurf
```

 **Utilisation :**

```

const csurf = require('csurf');

const csrfProtection = csurf();

```

```

app.use(csrfProtection);

app.get('/form', (req, res) => {
  res.send(`<form action="/submit" method="POST">
    <input type="hidden" name="_csrf" value="${req.csrfToken()}">
    <button type="submit">Envoyer</button>
  </form>`);
});

```

#### 7.2.4 Exemple : Utilisation d'un Middleware Tiers (morgan) :

```

const morgan = require('morgan');

app.use(morgan('dev'));

```

### 7.3 Création de Middlewares Personnalisés

Les middlewares personnalisés permettent de répondre à des besoins spécifiques.

#### 7.3.1 Structure d'un Middleware

Un middleware personnalisé est une fonction prenant trois paramètres :

-  **req** : Objet de la requête.
-  **res** : Objet de la réponse.
-  **next** : Fonction pour passer au middleware suivant.

#### 7.3.2 Exemple : Middleware de Journalisation

```

function logger(req, res, next) {
  console.log(`${req.method} ${req.url}`);
  next(); // Passe au middleware suivant
}

app.use(logger);

```



### 7.3.3 Exemple : Middleware de Validation des Données

```
function validateData(req, res, next) {  
  
  const { name, age } = req.body;  
  
  if (!name || !age) {  
  
    return res.status(400).send('Les champs "name" et "age" sont requis.');  
  }  
  
  next();  
}  
  
app.post('/validate', validateData, (req, res) => {  
  
  res.send('Données valides');  
  
});
```

## 7.4 Cas Pratiques

### 7.4.1 Middleware de Gestion des Erreurs

Les middlewares de gestion des erreurs capturent les exceptions non traitées et renvoient une réponse appropriée.

**Exemple : Middleware d'Erreur Personnalisé :**

```
function errorHandler(err, req, res, next) {  
  
  console.error('Erreur détectée :', err.message);  
  
  res.status(500).json({ error: 'Erreur interne du serveur' });  
  
}  
  
// Exemple d'une route générant une erreur  
app.get('/error', (req, res, next) => {  
  
  const err = new Error('Erreur simulée');
```

```
next(err); // Passe au middleware d'erreur

});

// Middleware d'erreur

app.use(errorHandler);
```

root@nasry-ahamadi:/home/nasry/projet# npm install morgan

```
root@nasry-ahamadi:/home/nasry/projet# npm install morgan
added 5 packages, and audited 104 packages in 5s

18 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
root@nasry-ahamadi:/home/nasry/projet#
```

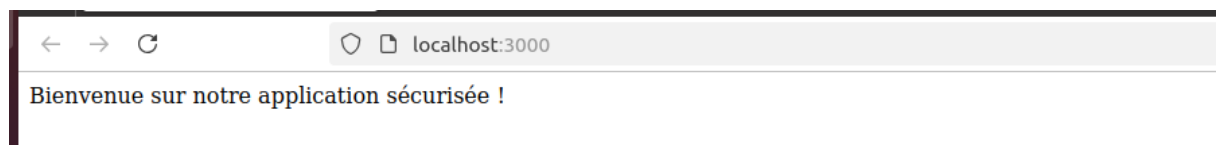
root@nasry-ahamadi:/home/nasry/projet# npm install helmet

```
root@nasry-ahamadi:/home/nasry/projet# npm install helmet
added 1 package, and audited 105 packages in 2s

18 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
root@nasry-ahamadi:/home/nasry/projet#
```

http://localhost:3000/



## Chapitre 8 : Les Moteurs de Templates

### 8.1 Présentation

Les moteurs de templates sont essentiels dans le développement d'applications web dynamiques. Ils permettent de générer des pages HTML dynamiques côté serveur en intégrant des données dans des modèles préconçus.

Ils jouent aussi un rôle crucial dans les applications web Node.js, facilitant la séparation entre la logique backend et la présentation frontend.

## 8.2 Définition

Un moteur de templates est un outil qui permet de créer des vues HTML dynamiques en intégrant des données provenant du serveur. Il transforme un modèle (template) avec des balises dynamiques en un document HTML final prêt à être envoyé au client.

## 8.3 Objectifs des moteurs de templates

- ✚ **Séparer la logique et la présentation** : Sépare la logique métier (backend) de la présentation (frontend).
- ✚ **Réutilisabilité** : Facilite la réutilisation de morceaux de code HTML à travers des fichiers partiels ou des composants.
- ✚ **Productivité** : Réduit le temps de développement en automatisant la création de vues dynamiques.
- ✚ **Générer des pages dynamiques** : Injecte facilement des données provenant du backend dans des modèles HTML.
- ✚ **Rendu côté serveur** : Produit des pages HTML complètes prêtes à être envoyées au client, ce qui est bénéfique pour le SEO.
- ✚ **Seamless integration** : Node.js prend en charge les moteurs de templates grâce à des frameworks comme Express.js.

## 8.4 Fonctionnement d'un moteur de templates

Modèle (template) : Un fichier contenant la structure HTML avec des balises dynamiques.  
Exemple avec EJS :

```
<h1>Bienvenue, <%= utilisateur.nom %> !</h1>
```

Données : Fournies par le serveur en tant qu'objets ou tableaux. Exemple :

```
const utilisateur = { nom: "Nasry" };
```

Rendu : Le moteur combine le modèle et les données pour générer un document HTML final :

```
<h1>Bienvenue, Nasry !</h1>
```

## 8.5 Les principaux moteurs de templates pour Node.js

Voici les moteurs les plus populaires pour Node.js avec leurs caractéristiques :

### 8.5.1 EJS (Embedded JavaScript)

#### Description :

Embedded JavaScript est l'un des moteurs de modèle les plus utilisés. Il permet d'incorporer du JavaScript directement dans le code HTML.

Simple et léger.

Utilise JavaScript pur pour intégrer des données dans les vues.

#### Installation :

```
npm install ejs
```

#### Syntaxe :

```
<h1>Bonjour, <%= nom %> !</h1>
```

#### Avantages :

Facile à apprendre.

Compatible avec Express.js.

Syntaxe proche de HTML pur.

#### Inconvénients :

Peu adapté pour des projets complexes.

Moins d'outils intégrés que certains autres moteurs.

### 8.5.2 Handlebars

Basé sur des balises {{ }} pour insérer des données.

Offre des helpers pour ajouter des fonctionnalités dans les templates.

#### **Installation :**

```
npm install express-handlebars
```

#### **Syntaxe :**

```
<h1>{{prenom}} {{nom}}</h1>
```

#### **Avantages :**

Syntaxe claire et intuitive.

Support des partials (morceaux de templates réutilisables).

#### **Inconvénients :**

Limitations en termes de logique métier dans les templates.

### 8.5.3 Pug (anciennement Jade)

Un moteur minimaliste avec une syntaxe simplifiée.

Génère des fichiers HTML à partir d'une indentation propre.

#### **Installation :**

```
npm install pug
```

#### **Syntaxe :**

```
h1 Bienvenue #{nom}
```

#### **Avantages :**

Syntaxe concise.

Réduction des erreurs grâce à l'indentation.

#### **Inconvénients :**

Courbe d'apprentissage plus élevée.

Peut être difficile à lire pour les développeurs novices.

### 8.5.4 Mustache

Moteur simple basé sur le principe de "logique-less templates" (pas de logique dans le template).

#### **Installation :**

```
npm install mustache
```

#### **Syntaxe :**

```
<h1>{{prenom}} {{nom}}</h1>
```

#### **Avantages :**

Léger et portable.

Disponible dans de nombreux langages.

#### **Inconvénients :**

Pas de logique avancée possible dans les templates.

### 8.5.5 Nunjucks

Inspiré de Jinja2 (Python), riche en fonctionnalités.

Idéal pour des templates avancés avec des blocs et des macros.

#### **Installation :**

```
npm install nunjucks
```

#### **Syntaxe :**

```
{% for utilisateur in utilisateurs %}  
  
  <h1>{{ utilisateur.nom }}</h1>  
  
{% endfor %}
```

#### **Avantages :**

Support des boucles, blocs, et inclusions.

Syntaxe puissante pour des projets complexes.

### 8.5.6 Liquid

Utilisé dans les CMS comme Shopify.

Syntaxe lisible et basée sur des blocs.

#### **Installation :**

```
npm install liquidjs
```

#### **Syntaxe :**

```
{% if utilisateur %}  
  
  <h1>{{ utilisateur.nom }}</h1>  
  
{% endif %}
```

#### **Avantages :**

Idéal pour des projets CMS-like ou générateurs statiques.

### 8.5.7 Dust.js

Conçu pour des applications nécessitant un rendu asynchrone.

#### **Installation :**

```
npm install dustjs-linkedin
```

#### **Syntaxe :**

```
{#utilisateurs}  
  
  <h1>{nom}</h1>  
  
{/utilisateurs}
```

#### **Avantages :**

Performant pour les pages rendues en streaming.

### 8.5.8 Marko

Optimisé pour le rendu côté serveur et le streaming haute performance.

#### **Installation :**

```
npm install marko
```

#### **Syntaxe :**

```
<h1>${utilisateur.nom}</h1>
```

### **Avantages :**

Performant pour des applications exigeantes.

Supporte le rendu incrémental.

## 8.6 Comparaison entre les moteurs de templates

| Moteur     | Avantages                                 | Inconvénients                     | Cas d'utilisation                         |
|------------|-------------------------------------------|-----------------------------------|-------------------------------------------|
| EJS        | Simple, syntaxe JS intégrée               | Peu adapté aux grands projets     | Projets simples                           |
| Pug        | Syntaxe concise, fonctionnalités avancées | Courbe d'apprentissage élevée     | Applications complexes                    |
| Handlebars | Syntaxe claire, helpers puissants         | Logique limitée                   | Projets intermédiaires                    |
| Mustache   | Léger, rapide                             | Pas de logique dans les templates | Applications simples                      |
| Nunjucks   | Riche en fonctionnalités                  | Syntaxe complexe                  | Projets avancés                           |
| Liquid     | CMS-ready                                 | Peu utilisé en dehors des CMS     | Applications CMS ou générateurs statiques |
| Dust.js    | Rendu asynchrone rapide                   | Syntaxe non standard              | Streaming HTML                            |
| Marko      | Performances élevées, streaming rapide    | Courbe d'apprentissage            | Applications haute performance            |

## 8.7 Choix du moteur selon le projet

-  Projets simples : EJS, Mustache.
-  Projets intermédiaires : Handlebars, Pug.
-  Projets complexes : Nunjucks, Liquid, Marko.
-  Streaming asynchrone : Dust.js, Marko.



## 8.8 Exemple d'utilisation avec EJS

```
root@nasry-ahamadi:/home/nasry# mkdir templates
```

```
root@nasry-ahamadi:/home/nasry# cd templates/
```

```
root@nasry-ahamadi:/home/nasry# mkdir templates
root@nasry-ahamadi:/home/nasry# cd templates/
root@nasry-ahamadi:/home/nasry/templates#
```

```
root@nasry-ahamadi:/home/nasry/templates# npm init -y
```

```
root@nasry-ahamadi:/home/nasry/templates# npm init -y
Wrote to /home/nasry/templates/package.json:

{
  "name": "templates",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": ""
}

root@nasry-ahamadi:/home/nasry/templates#
```

```
root@nasry-ahamadi:/home/nasry/templates# npm install express
```

```
root@nasry-ahamadi:/home/nasry/templates# npm install express
added 69 packages, and audited 70 packages in 13s

14 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
root@nasry-ahamadi:/home/nasry/templates#
```

### 8.8.1 Installation d'EJS :

```
npm install ejs
```

```
root@nasry-ahamadi:/home/nasry/templates# npm install ejs
added 16 packages, and audited 86 packages in 3s

16 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
root@nasry-ahamadi:/home/nasry/templates#
```

### 8.8.2 Configuration d'Express.js :

```
root@nasry-ahamadi:/home/nasry/templates# nano app.js
root@nasry-ahamadi:/home/nasry/templates#
```

```
GNU nano 4.8 app.js
const express = require('express');
const app = express();

app.set('view engine', 'ejs');
app.set('views', './views');

app.get('/', (req, res) => {
  const utilisateur = { nom: "Nasry", prenom: "Ahamadi" };
  res.render('index', { utilisateur });
});

app.listen(3000, () => {
  console.log('Serveur démarré sur http://localhost:3000');
});
```

### 8.8.3 Code complet :

```
root@nasry-ahamadi:/home/nasry/templates# nano app.js
root@nasry-ahamadi:/home/nasry/templates#
```

```

GNU nano 4.8                                app.js
const express = require('express');
const app = express();

app.set('view engine', 'ejs');
app.set('views', './views');

app.get('/', (req, res) => {
  const utilisateur = { nom: "Nasry", prenom: "Ahamadi" };
  res.render('index', { utilisateur });
});

app.listen(3000, () => {
  console.log('Serveur démarré sur http://localhost:3000');
});

```

#### 8.8.4 Création du html

```

root@nasry-ahamadi:/home/nasry/templates/views# nano index.ejs
root@nasry-ahamadi:/home/nasry/templates/views#

```

```

GNU nano 4.8                                index.ejs
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Bienvenue</title>
</head>
<body>
  <h1>Bonjour, <%= utilisateur.nom %> <%= utilisateur.prenom %>!</h1>
</body>
</html>

```

#### 8.8.5 Démarrage du serveur

```
root@nasry-ahamadi:/home/nasry/templates# node app.js
```

```

root@nasry-ahamadi:/home/nasry/templates# node app.js
Serveur démarré sur http://localhost:3000

```

#### 8.8.6 Résultat :

http://localhost:3000/



# Chapitre 9 : Interaction avec les Bases de Données

## 9.1 Présentations des bases de données

Une base de données est un ensemble organisé de données, stockées et accessibles de manière électronique. Elle permet de gérer, manipuler et interroger des données de manière efficace.

Rôle fondamental : faciliter le stockage, la récupération et la manipulation des données dans divers systèmes d'information (SI).

## 9.2 Pourquoi interagir avec une base de données ?

Une base de données est essentielle pour stocker, gérer, et récupérer les données nécessaires au fonctionnement des applications. Les interactions avec les bases de données permettent :

- + De manipuler des données dynamiques (utilisateurs, produits, etc.).
- + De persister les informations entre plusieurs sessions utilisateur.
- + D'assurer la gestion des relations entre les entités.

## 9.3 Types de bases de données

Bases relationnelles (SQL) :

- + Stockent les données sous forme de tables avec des relations claires entre elles.
- + Exemples : MySQL, PostgreSQL, SQLite.

Bases non relationnelles (NoSQL) :

- + Stockent les données sous forme de documents, graphes, ou colonnes.
- + Exemples : MongoDB, Cassandra, DynamoDB.

## 9.4 Les Requêtes directes avec Node.js

### 9.4.1 Définition

Les requêtes directes consistent à écrire et exécuter des requêtes SQL ou NoSQL pour interagir avec la base de données, sans utiliser d'abstraction supplémentaire.

### 9.4.2 Requêtes SQL (avec MySQL)

Pour interagir avec une base relationnelle comme MySQL, nous utilisons un client MySQL comme mysql2.

### 9.4.3 Exemple d'installation :

```
npm install mysql2
```

### 9.4.4 Configuration de la connexion :

```
const mysql = require('mysql2');

// Configuration

const db = mysql.createConnection({

  host: 'localhost',

  user: 'root',

  password: 'votre_mot_de_passe',

  database: 'nom_de_la_base'

});

// Connexion

db.connect((err) => {

  if (err) throw err;

  console.log('Connecté à la base de données.');
```

### 9.4.5 Requêtes CRUD :

Création une table :

```
CREATE TABLE clients (

  id INT AUTO_INCREMENT PRIMARY KEY,

  prenom VARCHAR(50),

  nom VARCHAR(50),

  numcompte VARCHAR(50),

  code VARCHAR(50),
```

```
);
```

Insérer des données :

```
const query = 'INSERT INTO clients (prenom, nom, numcompte, code,solde) VALUES  
( ?, ?, ?, ?, ?)';  
  
db.query(query, ['Ahamadi', 'Nasry', '1001', '1111', '100000'], (err, result) => {  
  
  if (err) throw err;  
  
  console.log('client ajouté avec ID :', result.insertId);  
  
});
```

Lire les données :

```
db.query('SELECT * FROM clients', (err, results) => {  
  
  if (err) throw err;  
  
  console.log('Clients :', results);  
  
});
```

Mettre à jour des données :

```
const query = 'UPDATE clients SET email = ? WHERE id = ?';  
  
db.query(query, ['prenom', 1], (err, result) => {  
  
  if (err) throw err;  
  
  console.log('Client mis à jour.');
```

Supprimer des données :

```
const query = 'DELETE FROM clients WHERE id = ?';  
  
db.query(query, [1], (err, result) => {  
  
  if (err) throw err;  
  
  console.log('Client supprimé.');
```

```
});
```

## 9.4.6 Requêtes NoSQL (avec MongoDB)

### 9.4.6.1 Connexion avec MongoDB :

Installez le client MongoDB :

```
npm install mongodb
```

Configuration de la connexion :

```
const { MongoClient } = require('mongodb');

// URL et nom de la base

const url = 'mongodb://localhost:27017';

const dbName = 'nom_de_la_base';

MongoClient.connect(url, { useUnifiedTopology: true })

  .then((client) => {

    console.log('Connecté à MongoDB.');

const db = client.db(dbName);



// Opérations MongoDB ici



})



.catch((err) => console.error('Erreur de connexion à MongoDB :', err));


```

### 9.4.6.2 Exemples de requêtes CRUD :

Insérer des données :

```
db.collection('clients').insertOne({ nom: 'Nasry', prenom: 'Ahamadi', solde: 3000000 })

  .then((result) => console.log('Document inséré avec ID :', result.insertedId))

  .catch((err) => console.error('Erreur :', err));
```

Lire les données :

```
db.collection('clients').find().toArray()

  .then((docs) => console.log('Documents trouvés :', docs))
```

```
.catch((err) => console.error('Erreur :', err));
```

Mettre à jour des données :

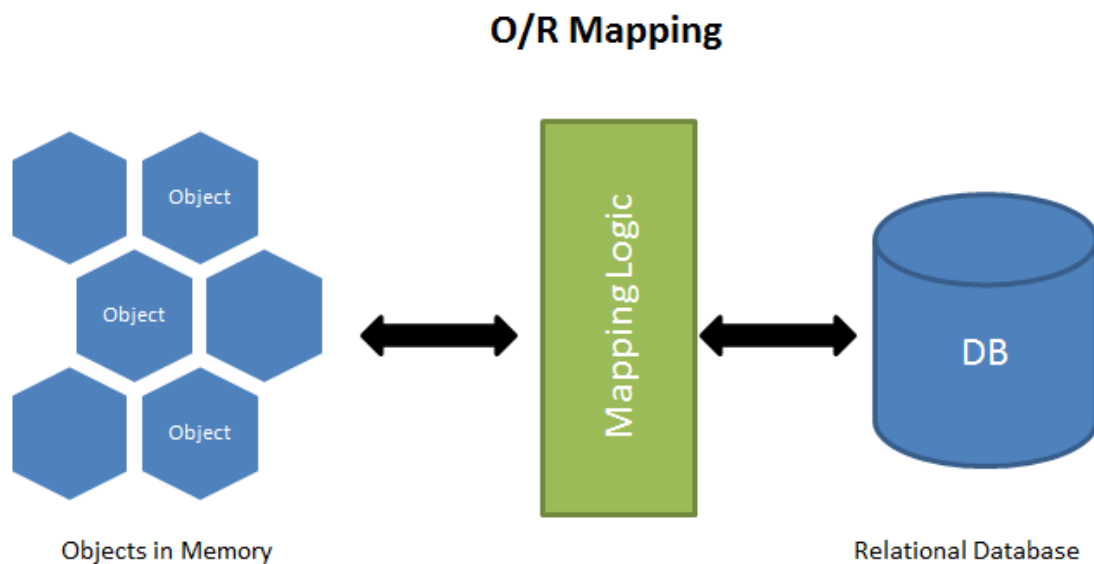
```
db.collection('clients').updateOne({ nom: 'Nasry' }, { $set: { solde: 90000000 } })  
  
.then((result) => console.log('Documents mis à jour :', result.modifiedCount))  
  
.catch((err) => console.error('Erreur :', err));
```

Supprimer des données :

```
db.collection('clients').deleteOne({ nom: 'Nasry' })  
  
.then((result) => console.log('Documents supprimés :', result.deletedCount))  
  
.catch((err) => console.error('Erreur :', err));
```

## 9.5 Les ORM (Object-Relational Mapping)

L'ORM, ou Object-Relational Mapping, est une technique utilisée en programmation pour faire le lien entre les objets de ton code et les données de ta Bases de données relationnelles. Voici une explication détaillée de ce concept :





### 9.5.1 Qu'est-ce qu'un ORM ?

Un ORM est une bibliothèque qui permet de manipuler une base de données à l'aide de classes et d'objets, sans écrire directement de requêtes SQL. L'ORM gère la traduction entre le code JavaScript et la base relationnelle.

## 9.6 Pourquoi utiliser un ORM ?

### 9.6.1 Avantages :

- + Abstraction : Masque la complexité des requêtes SQL.
- + Productivité accrue : Réduit le temps de développement.
- + Portabilité : Permet de changer de base de données facilement (par ex. MySQL à PostgreSQL).
- + Validation intégrée : Vérifie les données avant de les insérer ou de les modifier.

### 9.6.2 Inconvénients :

- + Performance : Les requêtes générées peuvent être moins optimisées.
- + Courbe d'apprentissage : Nécessite de comprendre la logique de l'ORM.
- + Complexité : Peut compliquer les projets simples.

## 9.7 ORM populaires dans l'écosystème Node.js

| ORM       | Bases de données supportées        | Avantages                                                    | Inconvénients                      |
|-----------|------------------------------------|--------------------------------------------------------------|------------------------------------|
| Sequelize | MySQL, PostgreSQL, SQLite, MariaDB | Riche en fonctionnalités, support des associations complexes | Plus lourd pour les petits projets |
| TypeORM   | MySQL, PostgreSQL, SQLite, MongoDB | Basé sur TypeScript, très modulable                          | Courbe d'apprentissage plus élevée |
| Mongoose  | MongoDB                            | Validation avancée, intégration facile avec MongoDB          | Exclusivement pour MongoDB         |

|              |                              |                               |                                   |
|--------------|------------------------------|-------------------------------|-----------------------------------|
| Objection.js | MySQL,<br>PostgreSQL, SQLite | Basé sur Knex.js,<br>flexible | Nécessite de<br>connaître Knex.js |
|--------------|------------------------------|-------------------------------|-----------------------------------|

### 9.7.1 Exemple avec Sequelize

Installation :

```
npm install sequelize mysql2
```

Configuration de la connexion :

```
const { Sequelize, DataTypes } = require('sequelize');

const sequelize = new Sequelize('nom_de_la_base', 'root', 'votre_mot_de_passe', {
  host: 'localhost',
  dialect: 'mysql',
});

// Tester la connexion

sequelize.authenticate()
  .then(() => console.log('Connecté à la base de données via Sequelize.'))
  .catch((err) => console.error('Erreur de connexion :', err));
```

Définition d'un modèle :

```
const Client = sequelize.define('Client', {
  nom: { type: DataTypes.STRING, allowNull: false },
  prenom: { type: DataTypes.STRING, allowNull: false },
  solde: { type: DataTypes.INTEGER },
});
```

Opérations CRUD avec Sequelize :

Insérer des données :

```
Client.create({ nom: 'Nasry', prenom: 'Ahamadi'});
```

Lire des données :

```
Client.findAll().then((users) => console.log(users));
```

Mettre à jour des données :

```
Client.update({ prenom: 'Toto' }, { where: { nom: 'Nasry' } });
```

Supprimer des données :

```
Client.destroy({ where: { nom: 'Nasry' } });
```

# Chapitre 10 : Développement d'APIs RESTful

## 10.1 Présentation d'API

Une API est un ensemble de définitions et de protocoles qui facilite la création et l'intégration de logiciels d'applications. On compare parfois les API à un contrat entre un utilisateur d'informations et un fournisseur d'informations, qui permet de définir le contenu demandé par le consommateur (l'appel) et le contenu demandé au producteur (la réponse). Par exemple, l'API d'un service de météo peut demander à l'utilisateur de fournir un code postal et au producteur de renvoyer une réponse en deux parties : la première concernant la température maximale et la seconde concernant la température minimale.

En d'autres termes, lorsqu'un utilisateur interagit avec un ordinateur ou un système pour obtenir des informations ou exécuter une fonction, l'API va l'aider à expliquer ce qu'il souhaite au système afin que ce dernier puisse comprendre et répondre à sa demande.

Les API servent de médiateur entre les utilisateurs ou les clients et les ressources ou services web qu'ils souhaitent obtenir. Ce sont aussi des outils qui permettent à une entreprise de partager des ressources et des informations de manière sécurisée et contrôlée, en déterminant qui peut accéder à quoi.

Autre avantage des API : il n'est pas nécessaire de connaître les détails de la mise en cache, c'est-à-dire comment les ressources sont récupérées ou d'où elles proviennent.

## 10.2 Évolution des API

### 10.2.1 Premières générations d'API :

- ✚ RPC (Remote Procedure Call) : Basé sur l'exécution de procédures sur des serveurs distants. Utilise des protocoles comme XML-RPC ou JSON-RPC.
- ✚ SOAP (Simple Object Access Protocol) : API complexe utilisant XML. Orienté sur la sécurité et les transactions. Principalement utilisé dans les systèmes d'entreprise.

### 10.2.2 Apparition de REST (Representational State Transfer) :

- ✚ Concept introduit en 2000 par Roy Fielding dans sa thèse.
- ✚ REST a révolutionné la manière de construire des API en adoptant les protocoles HTTP comme fondation.

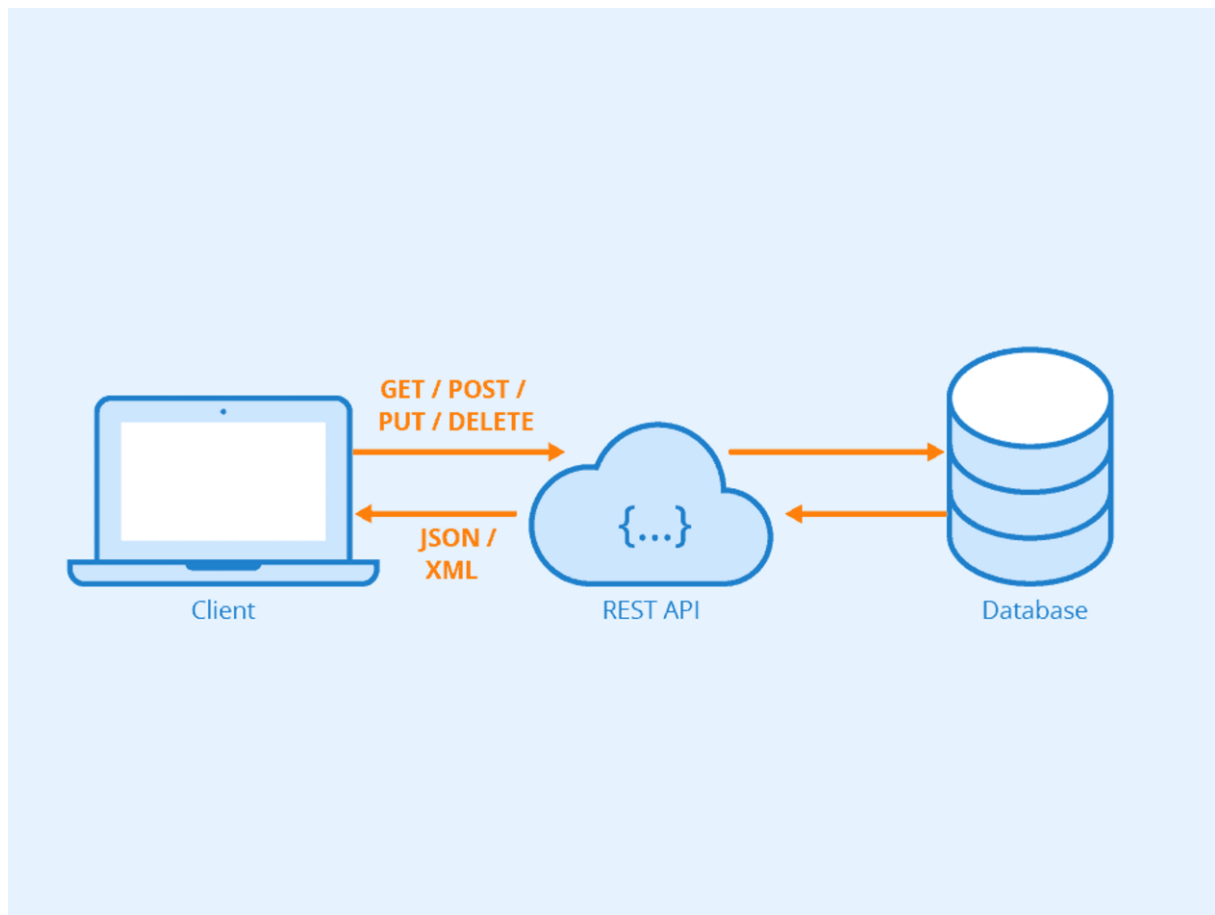
### 10.2.3 APIs modernes :

- ✚ GraphQL : Langage de requête flexible pour des besoins spécifiques.
- ✚ gRPC : Basé sur Protocol Buffers pour une communication rapide et efficace.

## 10.3 Concepts théoriques des APIs RESTful

### 10.3.1 Définition de REST

REST est une architecture qui repose sur l'utilisation des méthodes HTTP pour manipuler des ressources identifiées par des URI (Uniform Resource Identifiers).



### 10.3.2 Comment fonctionne une API REST ?

Dans la pratique, l'API REST fonctionne sur le principe de l'environnement client-serveur. L'API RESTful récupère et transmet d'un côté les requêtes d'un utilisateur ou d'une application, et de l'autre les informations rendues par le serveur (application ou base de données).

Quelques notions clés permettent de saisir le fonctionnement d'une API RESTful. Le client est l'entité effectuant une demande. C'est le cas par exemple d'un utilisateur effectuant une recherche au sein d'un catalogue de produits sur son navigateur. L'API se charge de

communiquer au serveur la requête, et de renvoyer vers le client les informations demandées. Les informations qui transitent par l'API sont les ressources. Le serveur traite les requêtes. Dans le cas présent, il renverra la liste des produits correspondant aux critères de recherche.

Les requêtes du client sont formulées à travers le protocole HTTP (HyperText Transfer Protocol). Voici les principales méthodes et tâches qu'il permet d'accomplir :

- ✚ **GET** : récupérer des données envoyées par le serveur.
- ✚ **POST** : envoyer et publier des informations vers le serveur (les données d'un formulaire d'inscription par exemple).
- ✚ **PUT** : mettre à jour les informations du serveur.
- ✚ **PATCH** : modifier partiellement une ressource existante.
- ✚ **DELETE** : supprimer des informations du serveur.

Les formats de données pour utiliser une API REST sont divers. Le format JSON (JavaScript Object Notation) est un format léger, facile à comprendre, et utilisable par de nombreux langages de programmation. XML (Extensible Markup Language) permet de gérer des structures de données complexes, et est compatible avec d'autres standards comme le RSS. YAML et HTML sont d'autres formats souvent utilisés pour communiquer les ressources.

### 10.3.3 Quels sont les principes de l'API REST ?

Une API REST suit les principes de REST en termes d'architecture logicielle. Ces principes créent une ligne directrice permettant de créer des APIs flexibles et légères, parfaitement adaptées à la transmission de données sur internet.

Voici les six principes architecturaux qui régissent une interface REST :

- ✚ **Découplage client-serveur** : Le client connaît seulement l'URI (Uniform Resource Identifier) de la ressource à récupérer. Le serveur interagit uniquement en transmettant ses données via HTTP.
- ✚ **Interface uniforme** : L'architecture REST uniformise la façon dont les informations sont identifiées, gérées et transmises, et fait appel aux hyperliens pour apporter des ressources supplémentaires au client (Les URL, les méthodes, et les codes de statut HTTP sont standardisés).
- ✚ **Code à la demande** : Le serveur peut transmettre du code au client pour étendre ses fonctionnalités, par exemple pour aider à repérer les erreurs d'un formulaire.

- ✚ **Système en couches** : Une API RESTful peut s'exécuter sur plusieurs serveurs organisés de façon hiérarchique, pour fournir un service plus stable et performant au client.
- ✚ **Mise en cache** : Le serveur REST peut mettre en cache des données pour mieux servir le client, en stockant par exemple les images d'un site pour les resservir ensuite (Les réponses peuvent être mises en cache pour optimiser les performances).
- ✚ **Absence d'état (Stateless)** : Chaque requête du client est isolée et traitée indépendamment par le serveur. Ainsi, chaque appel doit contenir tous les éléments nécessaires à son traitement (Le serveur ne conserve pas l'état des sessions entre les requêtes).

### 10.3.4 Méthodes HTTP et leurs usages

- ✚ **GET** : Lire des données.
- ✚ **POST** : Ajouter de nouvelles données.
- ✚ **PUT** : Modifier ou remplacer des données existantes.
- ✚ **DELETE** : Supprimer une ressource.

## 10.4 Création d'API avec Node.js

### 10.4.1 Création du projet

On commence par créer un nouveau dossier pour notre projet :

```
root@nasry-ahamadi:/home/nasry# mkdir api-clients
```

```
root@nasry-ahamadi:/home/nasry# cd api-clients/
```

```
root@nasry-ahamadi:/home/nasry# mkdir api-clients
root@nasry-ahamadi:/home/nasry# cd api-clients/
root@nasry-ahamadi:/home/nasry/api-clients#
```

### 10.4.2 Initialisation du projet

Pour initialiser notre projet, on utilise la commande suivante :

```
root@nasry-ahamadi:/home/nasry/api-clients# npm init -y
```

```

root@nasry-ahamadi:/home/nasry/api-clients# npm init -y
Wrote to /home/nasry/api-clients/package.json:

{
  "name": "api-clients",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": ""
}

root@nasry-ahamadi:/home/nasry/api-clients# █

```

La commande `npm init -y` est utilisée pour initialiser un projet Node.js et générer rapidement un fichier `package.json` avec les valeurs par défaut. Ce fichier est essentiel pour gérer les dépendances et les métadonnées du projet.

### 10.4.3 Installation des dépendances nécessaires :

Une fois initialiser notre projet, on va maintenant installer les modules nécessaires pour le fonctionnement de notre projet.

```
root@nasry-ahamadi:/home/nasry/api-clients# npm install express
```

```

root@nasry-ahamadi:/home/nasry/api-clients# npm install express

up to date, audited 70 packages in 1s

14 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
root@nasry-ahamadi:/home/nasry/api-clients# █

```

Installation du module Express, qui est un framework minimaliste et flexible pour Node.js. Express facilite le développement d'applications web et d'API en fournissant des outils robustes pour gérer les requêtes HTTP, les réponses, le routage, et plus encore.

### 10.4.4 Importation des Modules

On commence par l'importation des modules dans notre projet :



```
const express = require('express');
```

```
GNU nano 4.8 app.js  
const express = require('express');
```

### 10.4.5 Initialisation de l'Application Express

On initialise notre application en mettant la ligne suivante :

```
const app = express();
```

```
const app = express();
```

#### Explication

- ✚ Cette ligne initialise une instance d'Express pour créer l'application API.
- ✚ Toutes les routes et middlewares sont ajoutés à cette instance.

```
const PORT = 3000;
```

```
const PORT = 3000;
```

### 10.4.6 Configuration des Middlewares

Ajout du middleware bodyParser.json :

```
app.use(express.json());
```

```
app.use(express.json());
```

#### Explication :

express.json() est un middleware intégré à Express. Il permet à Express de comprendre et de traiter les données JSON envoyées dans le corps (body) des requêtes HTTP.

### 10.4.7 Déclaration de la liste de clients

```
let clients = [  
  { id: 1, prenom: "Macky", nom: "SALL", numcompte: "1001", code: "1111", solde:  
100000 },
```

```
{ id: 2, prenom: "Ousmane", nom: "SONKO", numcompte: "1002", code: "2222",  
solde: 200000 }  
];
```

```
// Tableau pour stocker les clients en mémoire  
let clients = [  
  { id: 1, prenom: "Macky", nom: "SALL", numcompte: "1001", code: "1111", solde: 100000 },  
  { id: 2, prenom: "Ousmane", nom: "SONKO", numcompte: "1002", code: "2222", solde: 200000 }  
];
```

#### Explication :

- ✚ let clients = [] : Crée un tableau de clients.
- ✚ Chaque {} : Un objet avec les informations du client.
- ✚ Chaque client a :
  - id : Identifiant unique
  - prenom, nom : Nom complet
  - numcompte : Numéro de compte
  - code : Code PIN (secret)
  - solde : Solde du compte

#### 10.4.8 Route GET pour afficher tous les clients

```
// Récupérer tous les clients
```

```
app.get('/clients', (req, res) => {  
  
  res.json(clients);  
});
```

```
// Récupérer tous les clients  
app.get('/clients', (req, res) => {  
  res.json(clients);  
});
```

#### Explication :

- ✚ app.get('/clients', ...) : Route pour une requête GET à /clients.
- ✚ res.json(clients) : Renvoie la liste complète des clients au format JSON.

### 10.4.9 Route GET pour afficher un client par ID

```
// Récupérer un client par ID
```

```
app.get('/clients/:id', (req, res) => {  
  
  const id = parseInt(req.params.id);  
  
  const client = clients.find(c => c.id === id);  
  
  if (!client) {  
  
    return res.status(404).json({ message: 'Client non trouvé' });  
  
  }  
  
  res.json(client);  
  
});
```

```
// Récupérer un client par ID  
app.get('/clients/:id', (req, res) => {  
  const id = parseInt(req.params.id);  
  const client = clients.find(c => c.id === id);  
  if (!client) {  
    return res.status(404).json({ message: 'Client non trouvé' });  
  }  
  res.json(client);  
});
```

#### Explication :

- ✚ `app.get('/clients/:id', ...)` : Route avec un paramètre dynamique `:id`.
- ✚ `const id = parseInt(req.params.id);` : Récupère l'id depuis l'URL (`req.params`).
- ✚ `const client = clients.find(...)` : Recherche du client par id.
- ✚ `if (!client) return res.status(404)` : Si pas trouvé, renvoie une erreur 404 ("Non trouvé").
- ✚ `res.json(client)` : Sinon, renvoie les infos du client.

### 10.4.10 Route POST pour ajouter un nouveau client

```
// Ajouter un nouveau client
```

```
app.post('/clients', (req, res) => {  
  
  const { prenom, nom, numcompte, code, solde } = req.body;
```

```

if (!prenom || !nom || !numcompte || !code || solde === undefined) {

    return res.status(400).json({ message: 'Tous les champs sont obligatoires' });

}

const id = clients.length ? clients[clients.length - 1].id + 1 : 1;

const nouveauClient = { id, prenom, nom, numcompte, code, solde };

clients.push(nouveauClient);

res.status(201).json(nouveauClient);

});

```

```

// Ajouter un nouveau client
app.post('/clients', (req, res) => {
    const { prenom, nom, numcompte, code, solde } = req.body;

    if (!prenom || !nom || !numcompte || !code || solde === undefined) {
        return res.status(400).json({ message: 'Tous les champs sont obligatoires' });
    }

    const id = clients.length ? clients[clients.length - 1].id + 1 : 1;
    const nouveauClient = { id, prenom, nom, numcompte, code, solde };
    clients.push(nouveauClient);
    res.status(201).json(nouveauClient);
});

```

### Explication :

- ✚ app.post('/clients', ...) : Route pour une requête POST.
- ✚ const { prenom, nom, ... } = req.body : Récupère les données envoyées (JSON) dans req.body (nécessite app.use(express.json())).
- ✚ Vérification : Si un champ manque → Erreur 400 ("Bad Request").
- ✚ Création de l'ID automatiquement (dernier ID + 1).
- ✚ clients.push(nouveauClient) : Ajoute le client au tableau.
- ✚ res.status(201).json(nouveauClient) : Renvoie le client créé avec statut 201 ("Created").

#### 10.4.11 Route PUT pour modifier un client existant par ID

```

app.put('/clients/:id', (req, res) => {

    const id = parseInt(req.params.id);

```

```
const { prenom, nom, numcompte, code, solde } = req.body;

const client = clients.find(c => c.id === id);

if (!client) {

    return res.status(404).json({ message: 'Client non trouvé' });

}

if (!prenom || !nom || !numcompte || !code || solde === undefined) {

    return res.status(400).json({ message: 'Tous les champs sont obligatoires' });

}

client.prenom = prenom;

client.nom = nom;

client.numcompte = numcompte;

client.code = code;

client.solde = solde;

res.json(client);

});
```

```
// Mettre à jour un client
app.put('/clients/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const { prenom, nom, numcompte, code, solde } = req.body;
  const client = clients.find(c => c.id === id);

  if (!client) {
    return res.status(404).json({ message: 'Client non trouvé' });
  }

  if (!prenom || !nom || !numcompte || !code || solde === undefined) {
    return res.status(400).json({ message: 'Tous les champs sont obligatoires' });
  }

  client.prenom = prenom;
  client.nom = nom;
  client.numcompte = numcompte;
  client.code = code;
  client.solde = solde;
  res.json(client);
});
```

### Explication :

- ✚ app.put('/clients/:id', ...) : Route pour PUT (mise à jour).
- ✚ Recherche du client avec .find() comme dans la route GET.
- ✚ Vérification : Erreurs 404 si client non trouvé, 400 si champs manquants.
- ✚ Mise à jour des propriétés de l'objet existant.
- ✚ res.json(client) : Renvoie les infos mises à jour.

### 10.4.12 Route DELETE pour supprimer un client par ID

```
app.delete('/clients/:id', (req, res) => {

  const id = parseInt(req.params.id);

  const index = clients.findIndex(c => c.id === id);

  if (index === -1) {

    return res.status(404).json({ message: 'Client non trouvé' });

  }

  clients.splice(index, 1);

  res.status(204).send();
});
```

```
});
```

```
app.delete('/clients/:id', (req, res) => {  
  const id = parseInt(req.params.id);  
  const index = clients.findIndex(c => c.id === id);  
  
  if (index === -1) {  
    return res.status(404).json({ message: 'Client non trouvé' });  
  }  
  
  clients.splice(index, 1);  
  res.status(204).send();  
});
```

#### Explication :

- ✚ app.delete('/clients/:id', ...) : Route pour DELETE.
- ✚ const index = clients.findIndex() : Recherche de l'index du client.
- ✚ Si non trouvé (-1) → Erreur 404.
- ✚ .splice(index, 1) : Supprime l'élément du tableau.
- ✚ res.status(204).send() : Pas de contenu (204 = Suppression réussie sans réponse).

#### 10.4.13 Lancement du serveur

```
// Lancement du serveur
```

```
app.listen(PORT, () => {  
  
  console.log(`Serveur démarré sur http://localhost:${PORT}`);  
  
});
```

```
app.listen(PORT, () => {  
  console.log(`Serveur démarré sur http://localhost:${PORT}`);  
});
```

#### Explication :

- ✚ const PORT = process.env.PORT || 3000; : Utilise le port défini dans l'environnement ou 3000 par défaut.
- ✚ app.listen() : Démarre le serveur.
- ✚ console.log() : Affiche un message dans la console avec l'URL.

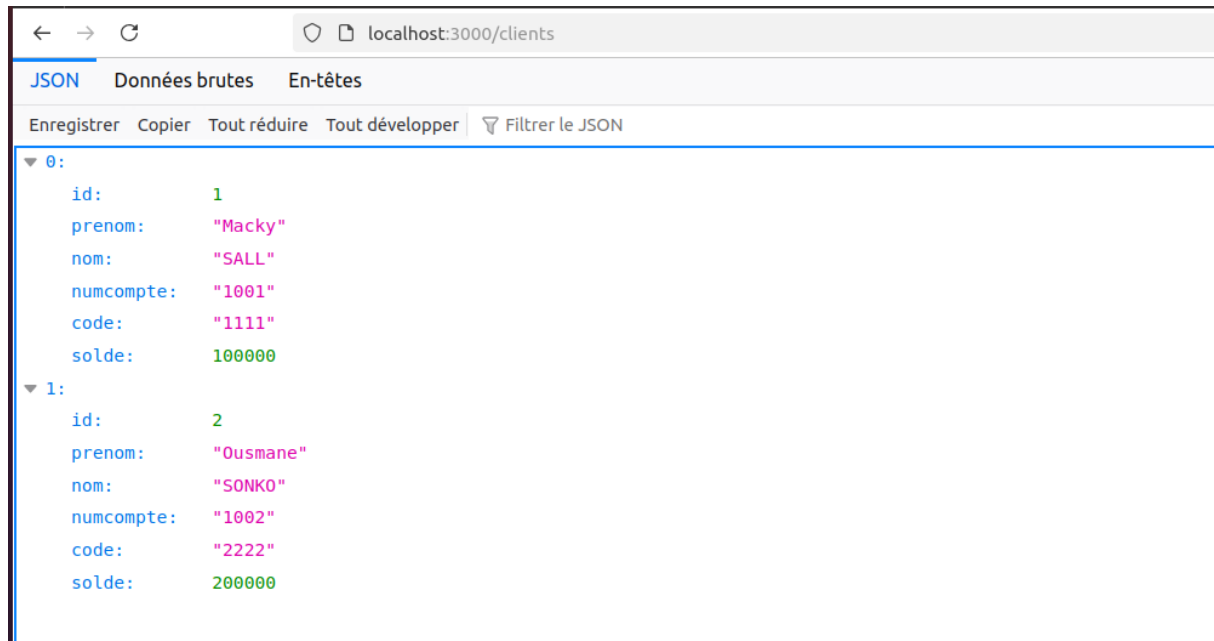
#### 10.4.14 Démarrage de l'api

```
root@nasry-ahamadi:/home/nasry/api-clients# node app.js
```

```
root@nasry-ahamadi:/home/nasry/api-clients# node app.js
Serveur démarré sur http://localhost:3000
```

### 10.4.15 Test de l'api

#### Affichage



The screenshot shows a web browser window with the address bar set to `localhost:3000/clients`. The browser is displaying the JSON response of a GET request. The response is an array of two objects, indexed 0 and 1. Each object contains the following fields: `id`, `prenom`, `nom`, `numcompte`, `code`, and `solde`.

| Index | id | prenom  | nom   | numcompte | code | solde  |
|-------|----|---------|-------|-----------|------|--------|
| 0     | 1  | Macky   | SALL  | 1001      | 1111 | 100000 |
| 1     | 2  | Ousmane | SONKO | 1002      | 2222 | 200000 |

#### Affichage des clients

```
root@nasry-ahamadi:/home/nasry# curl -X GET http://localhost:3000/clients
```

```
root@nasry-ahamadi:/home/nasry# curl -X GET http://localhost:3000/clients
[{"id":1,"prenom":"Macky","nom":"SALL","numcompte":"1001","code":"1111","solde":100000}, {"id":2,"prenom":"Ousmane","nom":"SONKO","numcompte":"1002","code":"2222","solde":200000}]root@nasry-ahamadi:/home/nasry#
root@nasry-ahamadi:/home/nasry#
```

#### Affichage d'un client à travers son id

```
root@nasry-ahamadi:/home/nasry# curl -X GET http://localhost:3000/clients/1
```

```
root@nasry-ahamadi:/home/nasry# curl -X GET http://localhost:3000/clients/1
{"id":1,"prenom":"Macky","nom":"SALL","numcompte":"1001","code":"1111","solde":100000}root@nasry-ahamadi:/home/nasry#
```

#### Affichage d'un client à travers un id qui n'existe pas

```
root@nasry-ahamadi:/home/nasry# curl -X GET http://localhost:3000/clients/999
```



```
root@nasry-ahamadi:/home/nasry# curl -X GET http://localhost:3000/clients/999
{"message":"Client non trouvé"}root@nasry-ahamadi:/home/nasry#
root@nasry-ahamadi:/home/nasry#
```

## 10.4.16 Ajouter un nouveau client

### 🚦 Requête POST :

```
root@nasry-ahamadi:/home/nasry# curl -X POST http://localhost:3000/clients -H "Content-Type: application/json" -d '{"prenom": "Diomaye","nom": "FAYE","numcompte": "1003","code": "3333","solde": 500000}'
```

```
root@nasry-ahamadi:/home/nasry# curl -X POST http://localhost:3000/clients -H "Content-Type: application/json" -d '{"prenom": "Diomaye","nom": "FAYE","numcompte": "1003","code": "3333","solde": 500000}'
{"id":3,"prenom":"Diomaye","nom":"FAYE","numcompte":"1003","code":"3333","solde":500000}root@nasry-ahamadi:/home/nasry#
```

### 🚦 Vérification de l'insertion

```
root@nasry-ahamadi:/home/nasry# curl -X GET http://localhost:3000/clients
```

```
root@nasry-ahamadi:/home/nasry# curl -X GET http://localhost:3000/clients
[{"id":1,"prenom":"Macky","nom":"SALL","numcompte":"1001","code":"1111","solde":100000},{"id":2,"prenom":"Ousmane","nom":"SONKO","numcompte":"1002","code":"2222","solde":200000},{"id":3,"prenom":"Diomaye","nom":"FAYE","numcompte":"1003","code":"3333","solde":500000}]root@nasry-ahamadi:/home/nasry#
```

## 10.4.17 Mettre à jour un client

### 🚦 Requête PUT (ID = 1) :

```
root@nasry-ahamadi:/home/nasry# curl -X PUT http://localhost:3000/clients/1 -H "Content-Type: application/json" -d '{"prenom": "Barack","nom": "OBAMA","numcompte": "1004","code": "4444","solde": 600000}'
```

```
root@nasry-ahamadi:/home/nasry# curl -X PUT http://localhost:3000/clients/1 -H "Content-Type: application/json" -d '{"prenom": "Barack","nom": "OBAMA","numcompte": "1004","code": "4444","solde": 600000}'
{"id":1,"prenom":"Barack","nom":"OBAMA","numcompte":"1004","code":"4444","solde":600000}root@nasry-ahamadi:/home/nasry#
```

### 🚦 Vérification de la modification

```
root@nasry-ahamadi:/home/nasry# curl -X GET http://localhost:3000/clients
```

```
root@nasry-ahamadi:/home/nasry# curl -X GET http://localhost:3000/clients
[{"id":1,"prenom":"Barack","nom":"OBAMA","numcompte":"1004","code":"4444","solde":600000},{"id":2,"prenom":"Ousmane","nom":"SONKO","numcompte":"1002","code":"2222","solde":200000},{"id":3,"prenom":"Diomaye","nom":"FAYE","numcompte":"1003","code":"3333","solde":500000}]root@nasry-ahamadi:/home/nasry#
```

### 10.4.18 Supprimer un client

🚦 **Requête DELETE (ID = 1) :**

```
root@nasry-ahamadi:/home/nasry# curl -X DELETE http://localhost:3000/clients/1
```

```
root@nasry-ahamadi:/home/nasry# curl -X DELETE http://localhost:3000/clients/1
root@nasry-ahamadi:/home/nasry#
```

### 10.4.19 Commandes cURL

| Action                     | Méthode | cURL Command                                                                                           |
|----------------------------|---------|--------------------------------------------------------------------------------------------------------|
| Récupérer tous les clients | GET     | curl -X GET http://localhost:3000/clients                                                              |
| Récupérer un client par ID | GET     | curl -X GET http://localhost:3000/clients/1                                                            |
| Ajouter un nouveau client  | POST    | curl -X POST http://localhost:3000/clients -H "Content-Type: application/json" -d '{"prenom": "..."}'  |
| Mettre à jour un client    | PUT     | curl -X PUT http://localhost:3000/clients/1 -H "Content-Type: application/json" -d '{"prenom": "..."}' |
| Supprimer un client        | DELETE  | curl -X DELETE http://localhost:3000/clients/1                                                         |

## 10.5 Postman

Postman est un outil populaire pour tester, documenter et automatiser les API. Il offre une interface graphique conviviale pour interagir avec les API, que ce soit pour des tests manuels ou automatisés.

### 10.5.1 Qu'est-ce que Postman ?

Postman est un outil collaboratif utilisé pour tester, développer, et documenter des APIs. Il permet aux développeurs d'interagir avec des endpoints d'API de manière intuitive en simulant des requêtes HTTP.

Initialement conçu comme une extension de navigateur, il est maintenant disponible sous forme d'application de bureau.

### 10.5.2 Rôle de Postman dans le développement d'API :

- ✚ **Tester des endpoints** : Vérifiez que votre API fonctionne correctement en envoyant des requêtes et en analysant les réponses.
- ✚ **Déboguer facilement** : Identifiez les problèmes dans les réponses de l'API.
- ✚ **Documenter l'API** : Créez automatiquement des documentations interactives basées sur vos tests.
- ✚ **Automatiser les tests** : Établissez des collections de tests qui peuvent être exécutées de manière séquentielle ou planifiée.

### 10.5.3 Pourquoi utiliser Postman ?

- ✚ **Facilité d'utilisation** : Interface intuitive pour tester rapidement des requêtes HTTP.
- ✚ **Support de toutes les méthodes HTTP** : GET, POST, PUT, DELETE, PATCH, etc.
- ✚ **Simulation d'API** : Permet d'envoyer des données au serveur via des corps de requêtes.
- ✚ **Documentation intégrée** : Génère automatiquement des collections pour décrire les API.

### 10.5.4 Méthodes HTTP prises en charge

GET : Récupérer des données depuis le serveur.

POST : Envoyer des données pour créer une ressource.

PUT : Mettre à jour une ressource existante.

DELETE : Supprimer une ressource.

### 10.5.5 Les Requêtes http

#### 10.5.5.1 Composantes principales d'une requête :

Requêtes :

- ✚ Une URL cible : L'adresse de l'API.

- ✚ Une méthode HTTP : (GET, POST, PUT, DELETE).
- ✚ En-têtes (Headers) : Métadonnées pour la requête, comme le type de contenu (Content-Type).
- ✚ Un corps de requête (pour POST ou PUT).
- ✚ Un corps de requête (Body) : Données envoyées pour les requêtes comme POST ou PUT.

#### 10.5.5.2 Exemple de requête avec Postman :

- ✚ URL : `http://localhost:3000/clients`
- ✚ Méthode : POST
- ✚ Headers : `{"Content-Type": "application/json"}`
- ✚ Body : `{"prenom": "Macky", "nom": "SALL", "numcompte": "1001", "code": "1111", "solde": 450000}`

#### 10.5.6 Collections

Une collection est un regroupement de requêtes organisées pour un projet ou une API.

Utile pour gérer les tests de projets complexes.

Avantages :

Partager des requêtes avec l'équipe.

Gérer les tests de manière centralisée.

#### 10.5.7 Environnements

Les environnements permettent de définir des variables globales ou locales (comme des URLs ou des clés d'API) pour simplifier les tests.

Exemple :

- ✚ Définir une variable `{{base_url}}` pour changer facilement l'URL de base. (Variable `{{base_url}}` définie comme `http://localhost:3000`).
- ✚ Requête : `{{base_url}}/clients`.

#### 10.5.8 Tests Automatisés

Postman permet d'écrire des scripts pour valider les réponses des API.

Exemple : Tester si une réponse retourne un statut 200.

### 10.5.9 Scripts de Pré-Requête et Tests :

Pré-requête : Scripts exécutés avant qu'une requête ne soit envoyée. Par exemple, pour ajouter un jeton d'authentification.

Tests : Vérifiez les réponses de l'API en utilisant JavaScript.

### 10.5.10 Historique des Requêtes

Postman garde un historique des requêtes effectuées, ce qui permet de les réutiliser ou de les analyser.

## 10.6 Avantages et Limites de Postman

### 10.6.1 Avantages

- ✚ Test rapide des API : Idéal pour les développeurs et testeurs.
- ✚ Interface utilisateur intuitive : Peu d'apprentissage requis.
- ✚ Gestion des scénarios complexes : Grâce aux environnements et collections.
- ✚ Exécution automatisée : Intégration avec Newman (CLI de Postman) pour l'automatisation.

### 10.6.2 Limites

- ✚ Apprentissage des scripts avancés : Les tests automatisés nécessitent des connaissances en JavaScript.
- ✚ Non conçu pour la production : Postman est un outil de développement et de test, pas une solution de déploiement.

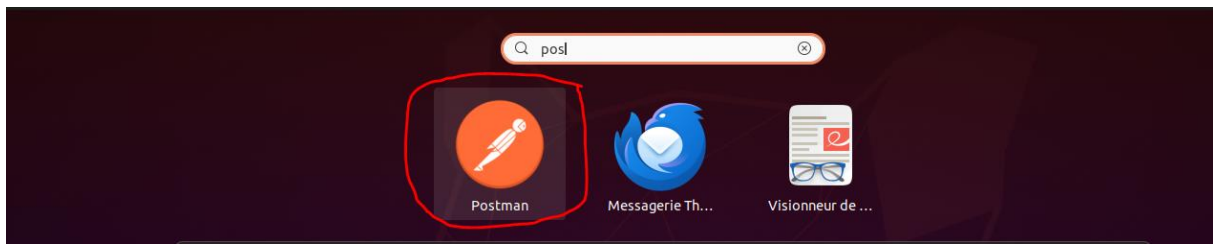
### 10.6.3 Installation de postman

Pour installer postman, on tape la commande suivante :

```
root@nasry-ahamadi:/home/nasry# snap install postman
```

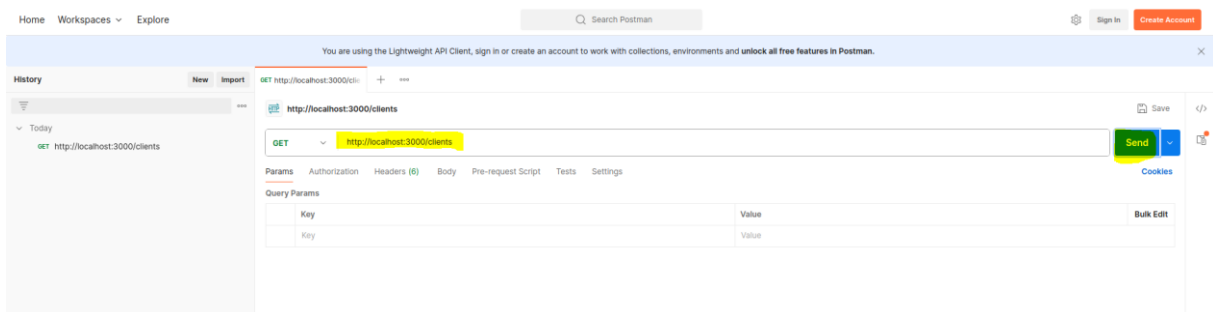
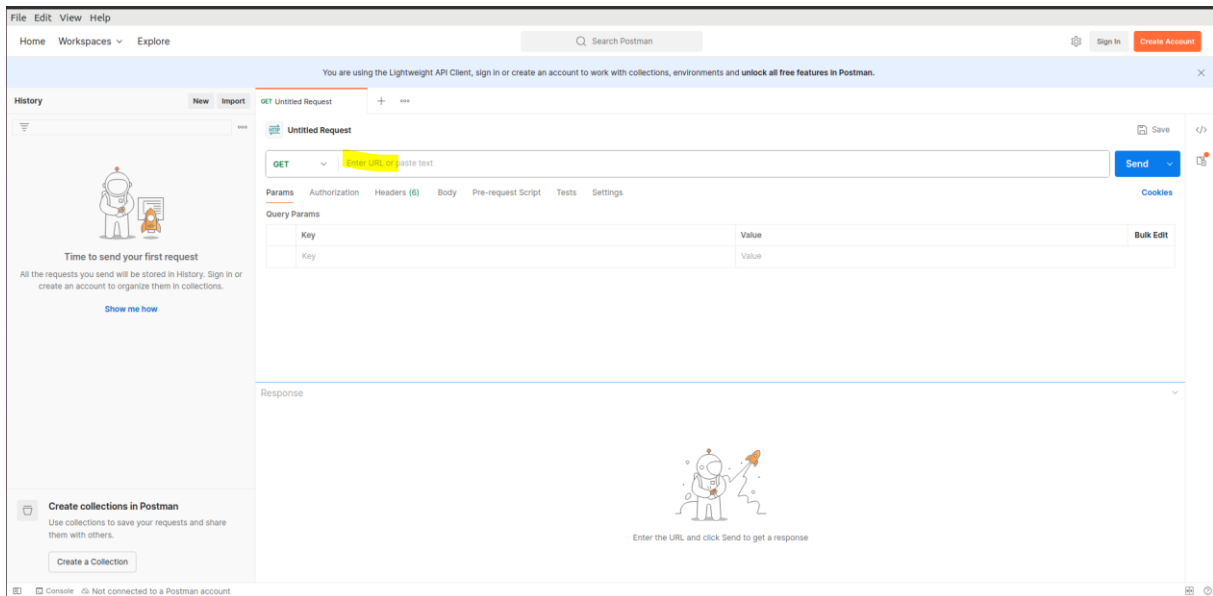
```
root@nasry-ahamadi:/home/nasry# snap install postman
postman (v11/stable) 11.23.3 from Postman, Inc. (postman-inc✓) installed
root@nasry-ahamadi:/home/nasry#
```

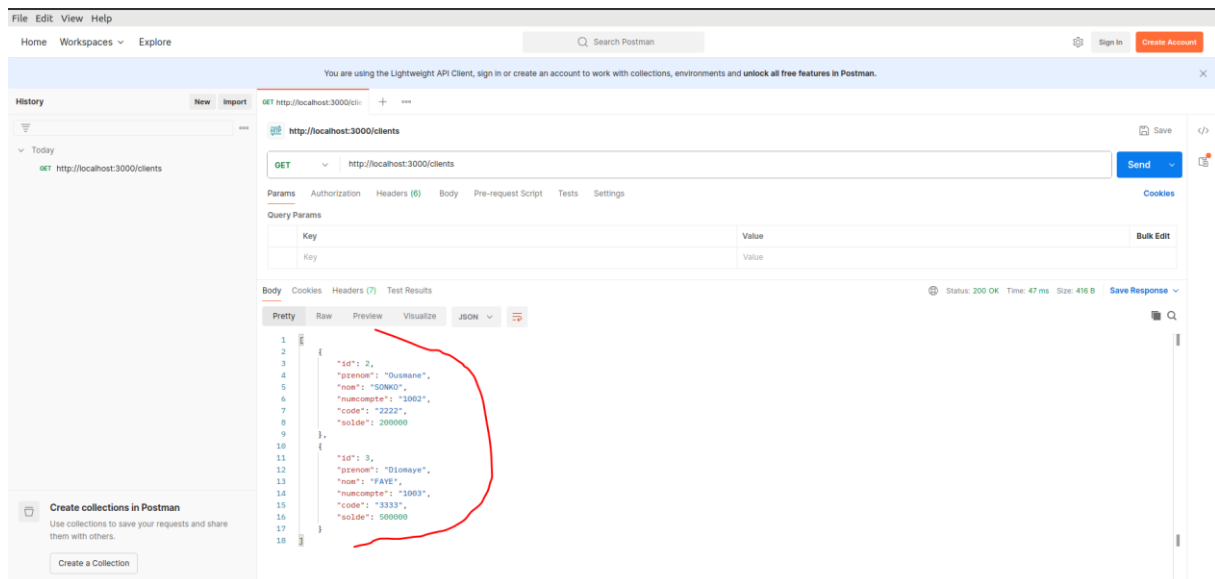
## 10.6.4 Démarrage de postman



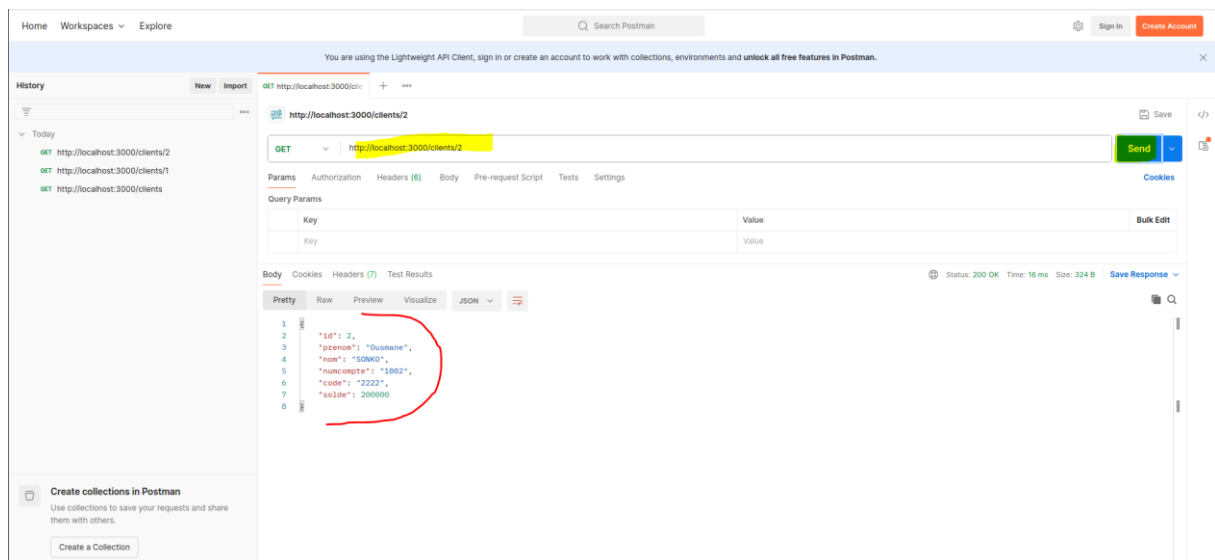
## 10.6.5 Affichage de la liste des clients

On choisit le verbe GET, après on renseigne l'url.

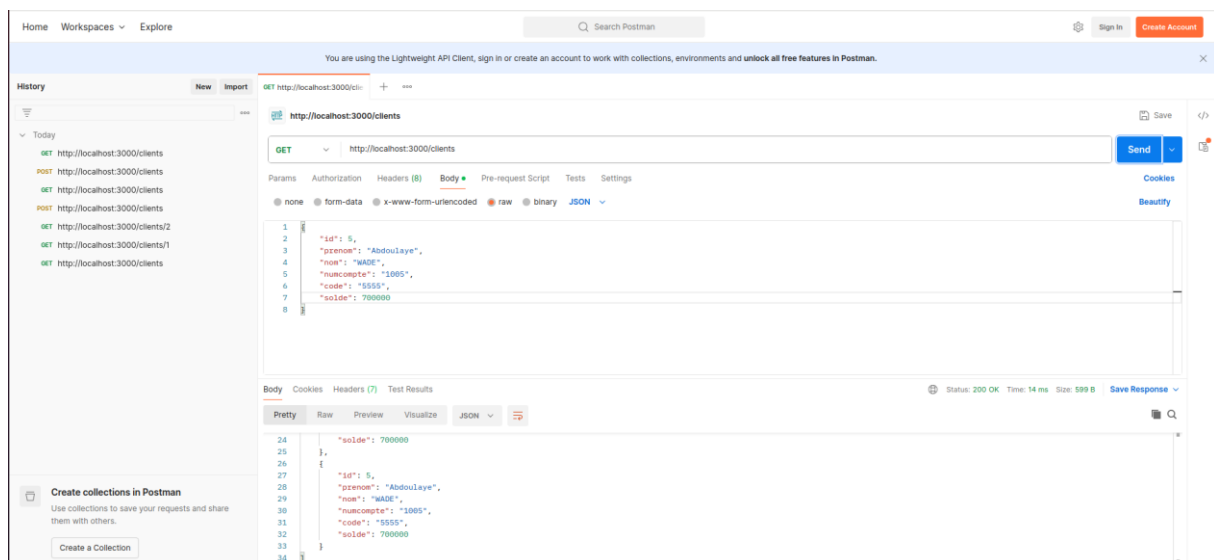
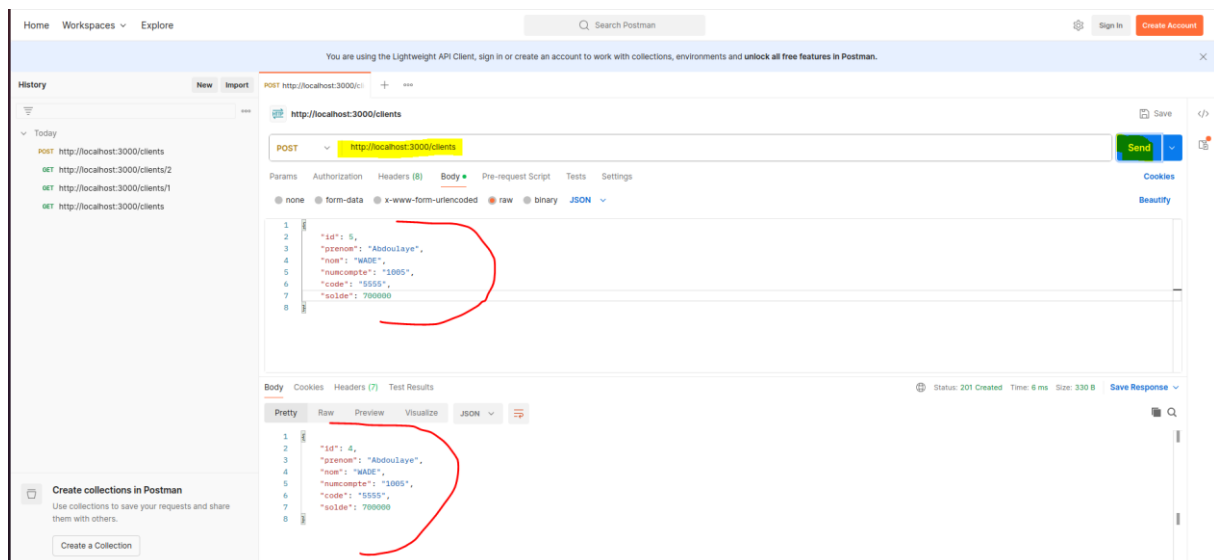
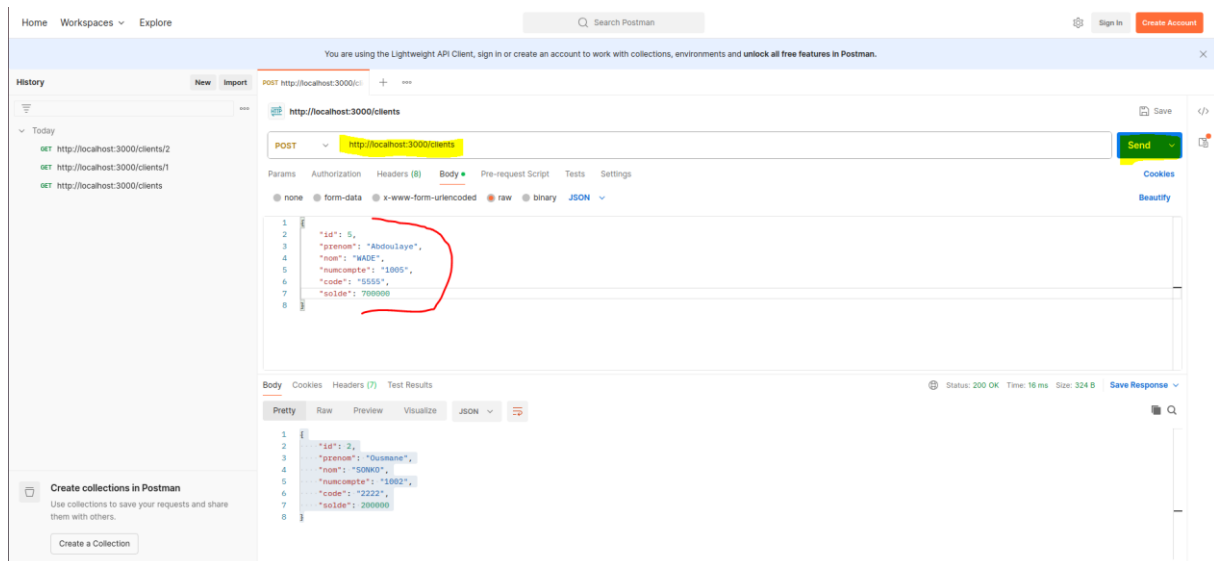




## 10.6.6 Affichage d'un client en utilisant son id

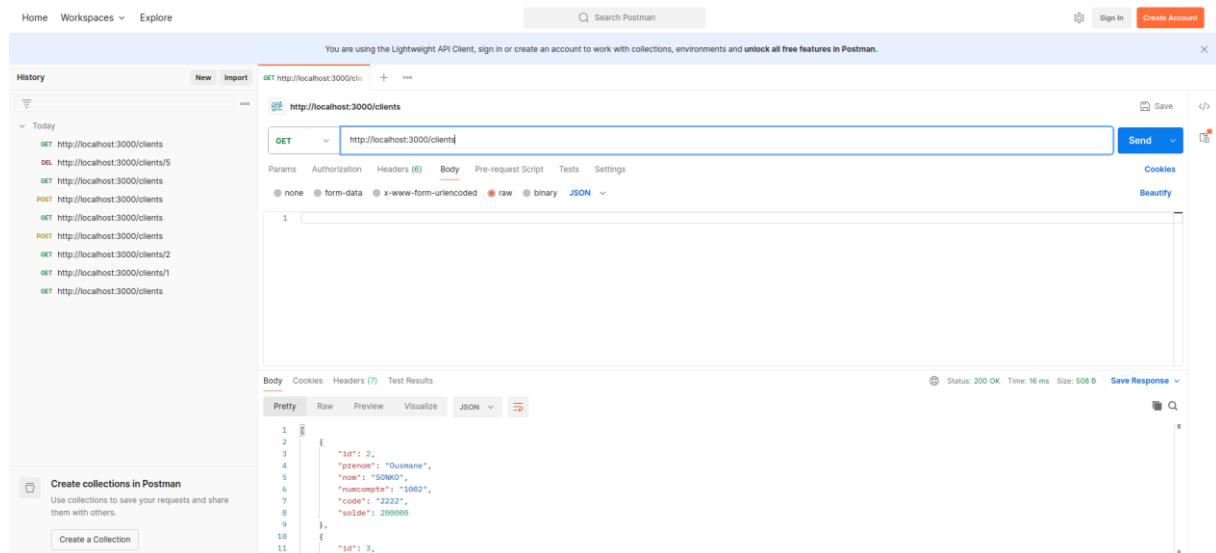
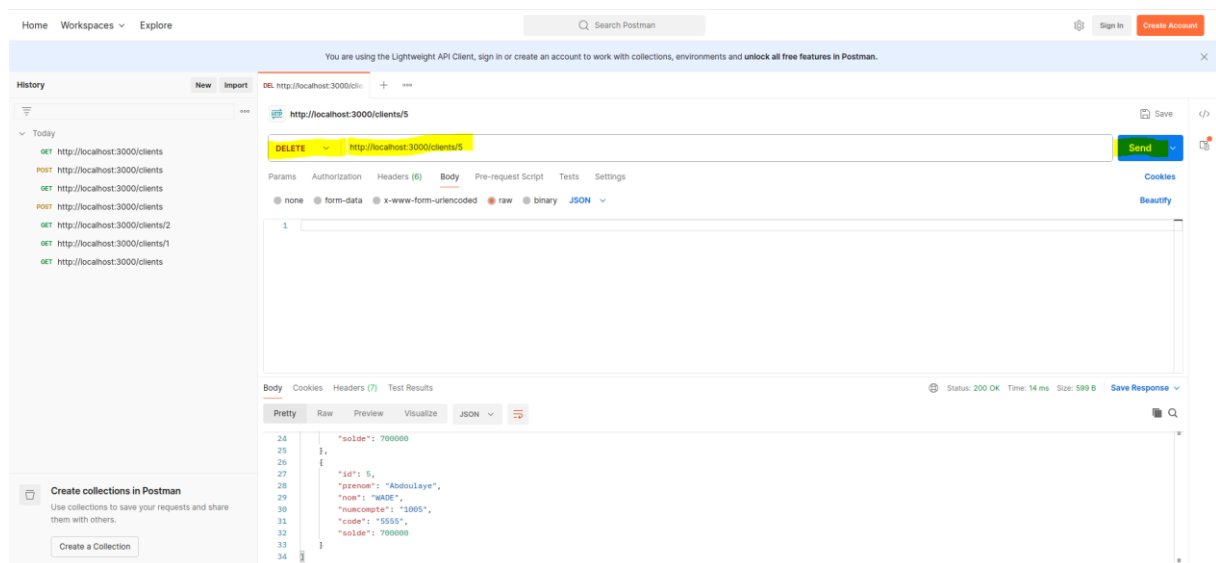


## 10.6.7 Ajout d'un client





## 10.6.8 Suppression d'un client



## 10.6.9 Comparaison entre Postman et d'autres outils similaires

| Fonctionnalité          | Postman                  | Insomnia | cURL                        |
|-------------------------|--------------------------|----------|-----------------------------|
| Interface graphique     | Oui                      | Oui      | Non                         |
| Support des collections | Oui                      | Oui      | Non                         |
| Tests automatisés       | Oui (intégration facile) | Oui      | Script à créer manuellement |
| Facilité d'utilisation  | Très simple              | Simple   | Complexe                    |

## 10.7 Insomnia

Insomnia est un outil open-source léger et convivial pour tester les APIs REST et GraphQL. Il offre une alternative à Postman avec une interface simplifiée et des fonctionnalités puissantes pour le développement et la gestion des APIs.

### 10.7.1 Qu'est-ce qu'Insomnia ?

- ✚ Insomnia est un client HTTP/API conçu pour aider les développeurs à tester, documenter, et automatiser les appels d'API.
- ✚ Supporte les requêtes REST et GraphQL.
- ✚ Open-source et disponible sur plusieurs plateformes : Windows, macOS, Linux.

### 10.7.2 Pourquoi utiliser Insomnia ?

- ✚ Interface intuitive : Design minimaliste, facile à prendre en main.
- ✚ Support des environnements : Gestion avancée des environnements avec des variables.
- ✚ Prise en charge des requêtes complexes : Authentification OAuth, JWT, Bearer Token, etc.
- ✚ Léger et performant : Moins gourmand en ressources comparé à d'autres outils.

### 10.7.3 Concepts clés d'Insomnia

#### 10.7.3.1 Requêtes HTTP

Insomnia permet de configurer des requêtes HTTP pour interagir avec les APIs.

Éléments d'une requête :

- ✚ URL : L'adresse de l'endpoint.
- ✚ Méthode HTTP : GET, POST, PUT, DELETE, PATCH.
- ✚ Headers : Métadonnées pour la requête (ex. Content-Type: application/json).
- ✚ Query Parameters : Paramètres ajoutés à l'URL (?key=value).
- ✚ Body : Contenu envoyé dans les requêtes POST, PUT, PATCH.

#### 10.7.3.2 Collections et dossiers

Insomnia organise les requêtes dans des Collections et Dossiers pour une gestion simplifiée.

Collection : Regroupe des requêtes liées à un projet ou une API.

Dossier : Structure interne à une collection pour organiser les endpoints.

### 10.7.3.3 Environnements

Insomnia supporte les environnements pour gérer des variables dynamiques (URLs, clés API, etc.).

Exemple d'utilisation :

- ✚ Définir une variable `{{base_url}}` avec la valeur `http://localhost:3000`.
- ✚ Utiliser `{{base_url}}/clients` dans vos requêtes pour éviter de répéter l'URL.

### 10.7.3.4 Authentification

Insomnia simplifie la configuration des méthodes d'authentification :

- ✚ Bearer Token : Pour envoyer un token dans l'en-tête.
- ✚ Basic Auth : Pour les systèmes nécessitant un nom d'utilisateur et un mot de passe.
- ✚ OAuth 2.0 : Gestion des tokens d'accès pour des APIs sécurisées.

### 10.7.3.5 Documentation intégrée

Insomnia génère automatiquement des détails sur les requêtes et réponses.

Permet d'exporter les collections en format JSON pour les partager ou les sauvegarder.

### 10.7.4 Avantages spécifiques d'Insomnia

- ✚ Prise en charge de GraphQL :

Insomnia permet de créer des requêtes GraphQL avec un éditeur intégré pour les schémas.

Facilité d'exploration et de test des endpoints GraphQL.

- ✚ Extensions personnalisées :

Insomnia permet d'ajouter des plugins pour étendre ses fonctionnalités, comme la génération de tests ou l'intégration avec des outils CI/CD.

- ✚ Export et import de collections :

Permet de partager vos requêtes ou de sauvegarder votre travail sous forme de fichiers JSON.

### 10.7.5 Comparaison Insomnia vs Postman

| Fonctionnalité | Insomnia | Postman |
|----------------|----------|---------|
|----------------|----------|---------|

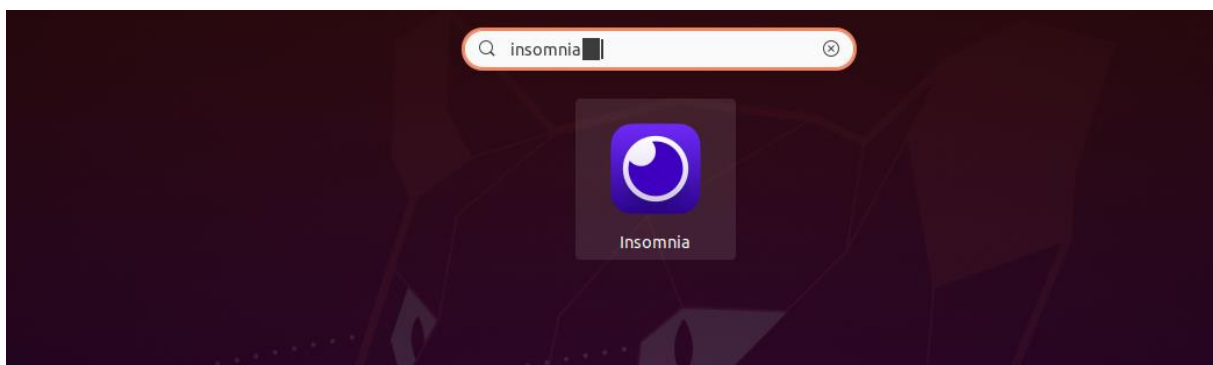
|                            |                                 |                               |
|----------------------------|---------------------------------|-------------------------------|
| Interface utilisateur      | Minimaliste, rapide             | Plus riche mais plus complexe |
| Requêtes prises en charge  | REST, GraphQL                   | REST, GraphQL, SOAP           |
| Gestion des environnements | Variables dynamiques et simples | Plus avancée                  |
| Tests automatisés          | Scripts intégrés simples        | Plus personnalisable          |
| Ressources système         | Léger                           | Plus gourmand                 |
| Licence                    | Open-source (licence MIT)       | Freemium                      |

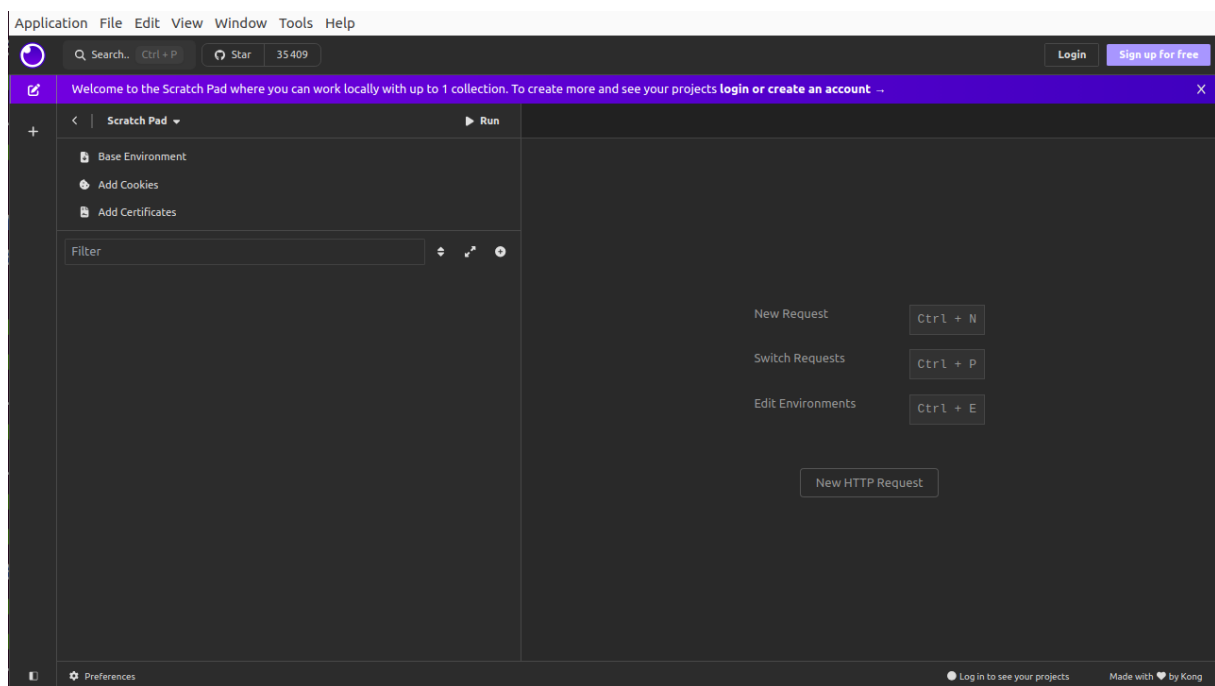
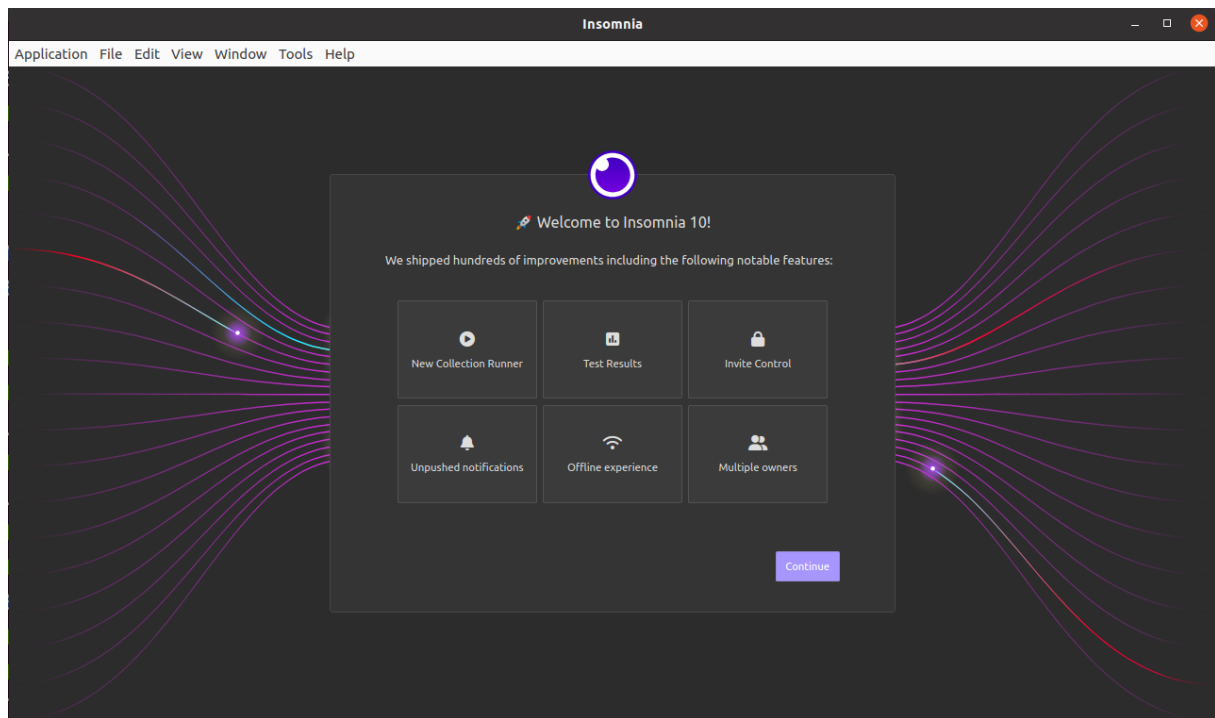
### 10.7.6 Installation et configuration

Téléchargez Insomnia en utilisant le gestionnaire de paquet snap.

```
root@nasry-ahamadi:/home/nasry# snap install insomnia
```

```
root@nasry-ahamadi:/home/nasry# snap install insomnia
insomnia 10.3.0 from Insomnia (getinsomnia) installed
root@nasry-ahamadi:/home/nasry#
```





## 10.7.7 Cas pratique : Tester une API REST simple

### 10.7.7.1 Exemple d'API

Endpoint : `http://localhost:3000/clients`.

### 10.7.7.2 Requêtes à tester :

GET `/clients` : Récupère tous les clients.

POST /clients : Ajoute un nouveau client :

Body :

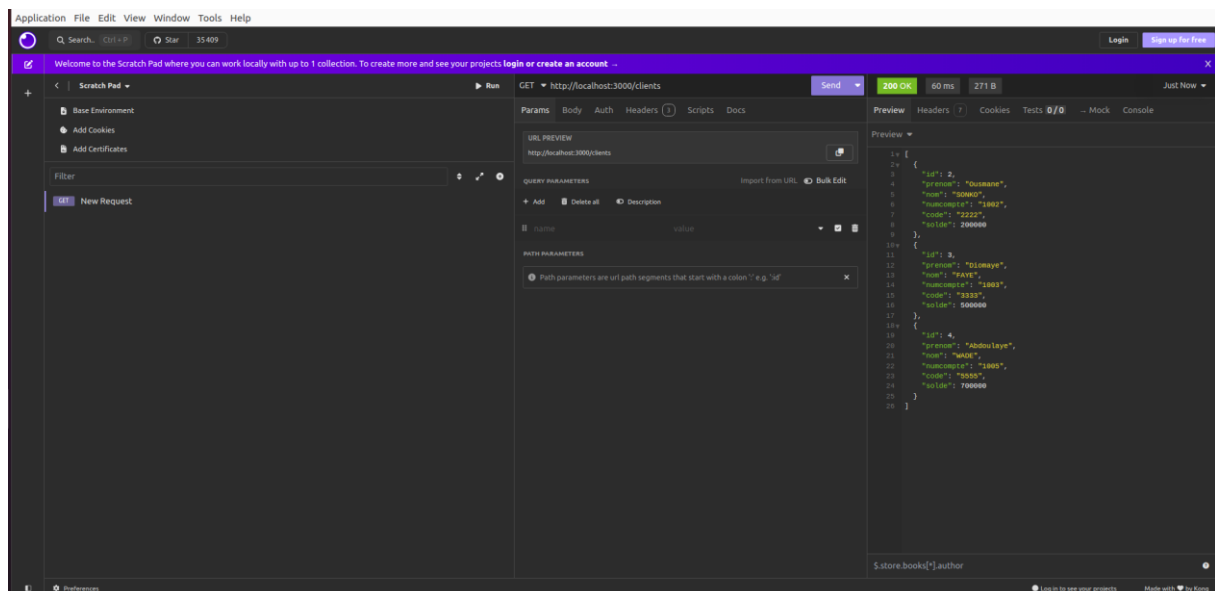
PUT /clients/1 : Modifie les informations du client ayant l'ID 1.

DELETE /clients/1 : Supprime le client ayant l'ID 1.

## 10.7.8 Étapes pour tester une API avec Insomnia

### 10.7.8.1 Tester un endpoint GET

- Créez une nouvelle requête GET.
- Entrez l'URL : `http://localhost:3000/clients`.
- Cliquez sur Send pour envoyer la requête.
- Observez la réponse JSON retournée par l'API.



### 10.7.8.2 Tester un endpoint POST

- Créez une nouvelle requête POST.
- Configurez l'URL : `http://localhost:3000/clients`.
- Ajoutez des headers :

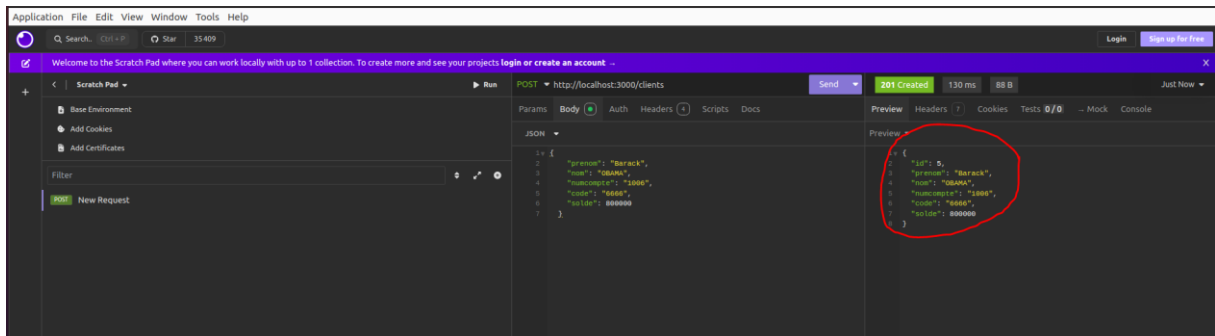
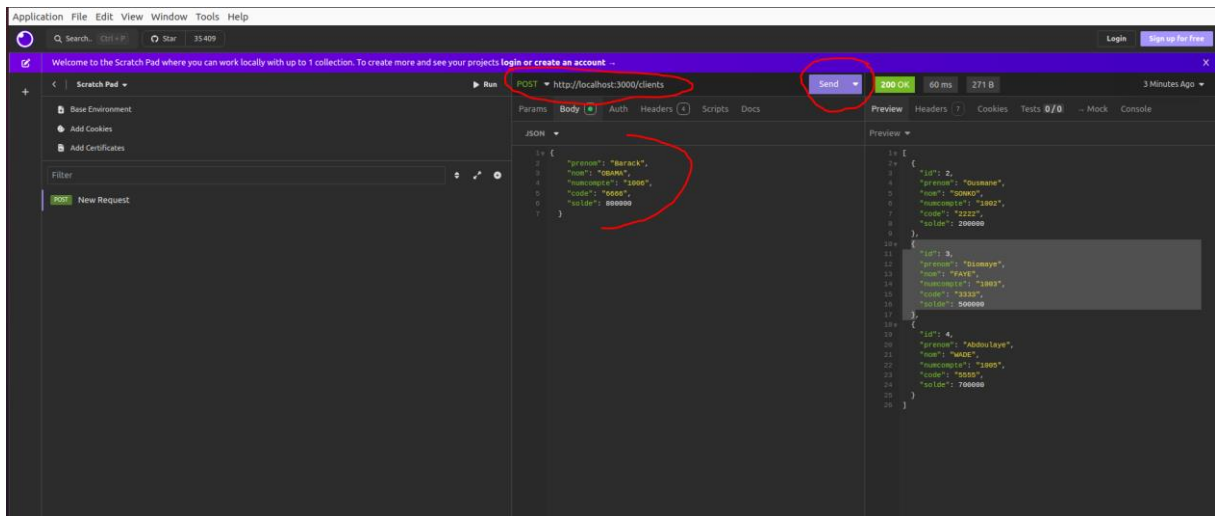
Content-Type: application/json.

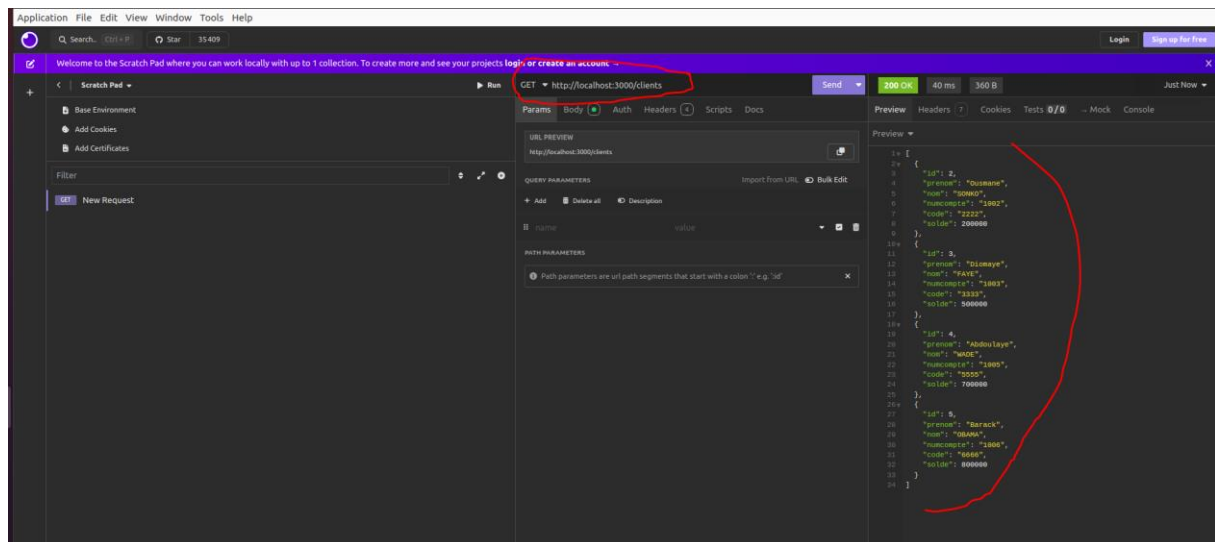
- Ajoutez un body :

```
{  
  
  "prenom": "Barack",  
}
```

```
"nom": "Obama",  
  
"numcompte": "1006",  
  
"code": "6666",  
  
"solde": 600000  
}
```

 Cliquez sur Send et vérifiez la réponse.







# Chapitre 11 : Création d'une api complet en utilisant la base de données MySQL.

## 11.1 Création de la base de données MySQL

### 11.1.1 Démarrer le serveur MySQL

On se connecte sur le serveur MySQL en tapant la commande suivante :

```
root@nasry-ahamadi:/home/nasry# mysql -u root -p
```

```
root@nasry-ahamadi:/home/nasry# mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 8.0.40-0ubuntu0.20.04.1 (Ubuntu)

Copyright (c) 2000, 2024, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> █
```

### 11.1.2 Création de la base de données :

On utilise la commande suivante, pour créer la base de données banque :

```
mysql> create database banque;
```

```
mysql> create database banque;
Query OK, 1 row affected (0,03 sec)

mysql> █
```

### 11.1.3 Sélection de la base de données :

Pour se sélectionner la base de données, on tape la commande suivante :

```
mysql> use banque;
```

```
mysql> use banque;
Database changed
mysql>
```

#### 11.1.4 Créer une table pour les clients

Pour créer la table clients avec ses champs suivant pour stocker les informations, on utilise la requête suivante :

```
mysql> create table clients(id int primary key auto_increment, prenom varchar(50),nom  
varchar(50),numcompte varchar(30),code varchar(20),solde varchar(30));
```

```
mysql> create table clients(id int primary key auto_increment, prenom varchar(50),n  
om varchar(50),numcompte varchar(30),code varchar(20),solde varchar(30));  
Query OK, 0 rows affected (0,05 sec)
```

```
mysql> █
```

#### 11.1.5 Création d'un utilisateur

Pour créer un utilisateur, on tape la requête suivante :

```
mysql> create user nasry identified by "passer";
```

```
mysql> create user nasry identified by "passer";  
Query OK, 0 rows affected (0,02 sec)
```

```
mysql>
```

#### 11.1.6 Attribution et application des privilèges

On tape les commandes les requêtes suivantes, pour attribuer et appliquer les privilèges :

```
mysql> grant all privileges on banque.* to nasry;
```

```
mysql> flush privileges;
```

```
mysql> grant all privileges on banque.* to nasry;  
Query OK, 0 rows affected (0,00 sec)
```

```
mysql> flush privileges;  
Query OK, 0 rows affected (0,01 sec)
```

```
mysql> █
```

#### 11.1.7 Insertion des données

Pour insérer des données, on tape la requête suivante :

```
mysql> insert into clients(prenom,nom,numcompte,code,solde)  
values("Macky","SALL","1001","1111","3000000");
```

```
mysql> insert into clients(prenom,nom,numcompte,code,solde) values("Macky","SALL","1001","1111","3000000");
Query OK, 1 row affected (0,02 sec)

mysql>
```

## 11.2 Création de l'api

### 11.3 Création du projet Node.js

Création d'un répertoire pour le projet :

```
root@nasry-ahamadi:/home/nasry# mkdir api-mysql
```

```
root@nasry-ahamadi:/home/nasry# cd api-mysql/
```

```
root@nasry-ahamadi:/home/nasry# mkdir api-mysql
root@nasry-ahamadi:/home/nasry# cd api-mysql/
root@nasry-ahamadi:/home/nasry/api-mysql#
```

### 11.4 Initialisation du projet Node.js :

Pour initialiser le projet, on tape la commande suivante :

```
root@nasry-ahamadi:/home/nasry/api-mysql# npm init -y
```

```
root@nasry-ahamadi:/home/nasry/api-mysql# npm init -y
Wrote to /home/nasry/api-mysql/package.json:

{
  "name": "api-mysql",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": ""
}

root@nasry-ahamadi:/home/nasry/api-mysql#
```

### 11.5 Installation des dépendances nécessaires :

Pour le bon fonctionnement de notre projet, on va les dépendances nécessaires comme suite :

### 11.5.1 Installation d'express

Pour installer express, on tape la commande suivante :

```
root@nasry-ahamadi:/home/nasry/api-mysql# npm install express
```

```
root@nasry-ahamadi:/home/nasry/api-mysql# npm install express
added 69 packages, and audited 70 packages in 8s

14 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
root@nasry-ahamadi:/home/nasry/api-mysql#
```

La commande `npm install express` est utilisée pour installer le framework Express.js dans un projet Node.js. Voici une explication détaillée de cette commande et de son fonctionnement :

### 11.5.2 Décomposition de la commande

- ✚ **npm** : C'est le gestionnaire de paquets officiel de Node.js. Il permet d'installer, de gérer et de partager des packages (bibliothèques ou outils) pour des projets Node.js.
- ✚ **install** : C'est une commande de npm qui permet d'installer un package. Elle peut être abrégée en `i` (par exemple, `npm i express`).
- ✚ **express** : C'est le nom du package à installer. Ici, il s'agit du framework Express.js.

### 11.5.3 Où est installé Express.js ?

- ✚ **Dossier `node_modules`** : Tous les packages installés via npm sont stockés dans ce dossier. Express.js et ses dépendances seront donc disponibles dans `node_modules/express`.
- ✚ **Fichier `package-lock.json`** : Ce fichier est automatiquement généré ou mis à jour pour enregistrer les versions exactes des packages installés, garantissant la reproductibilité du projet.

### 11.5.4 Installation de mysql2

Pour installer mysql2, on tape la commande suivante :

```
root@nasry-ahamadi:/home/nasry/api-mysql# npm install mysql2
```

```
root@nasry-ahamadi:/home/nasry/api-mysql# npm install mysql2
added 12 packages, and audited 82 packages in 14s

15 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
root@nasry-ahamadi:/home/nasry/api-mysql#
```

La commande `npm install mysql2` est utilisée pour installer le package `mysql2` dans un projet Node.js. Ce package est une bibliothèque qui permet à Node.js d'interagir avec une base de données MySQL. Voici une explication détaillée de cette commande et de son utilité :

### 11.5.5 Décomposition de la commande

- ✚ **npm** : Le gestionnaire de paquets officiel de Node.js, utilisé pour installer et gérer des packages.
- ✚ **install** : Une commande npm pour installer un package. Elle peut être abrégée en `i` (par exemple, `npm i mysql2`).
- ✚ **mysql2** : Le nom du package à installer. Il s'agit d'une bibliothèque pour interagir avec une base de données MySQL.

### 11.5.6 Ce que fait cette commande

- ✚ **Télécharge le package mysql2** : npm va chercher la dernière version stable de `mysql2` dans le registre npm.
- ✚ **Installe mysql2 dans le projet** : Le package est téléchargé et installé dans le dossier `node_modules` du projet.
- ✚ **Met à jour le fichier package.json** : Si le fichier `package.json` existe, npm ajoute `mysql2` comme dépendance dans la section "dependencies". Si le fichier n'existe pas, il peut être créé avec la commande `npm init` avant d'installer `mysql2`.

### 11.5.7 Où est installé mysql2 ?

- ✚ **Dossier node\_modules** : Tous les packages installés via npm sont stockés dans ce dossier. `mysql2` et ses dépendances seront donc disponibles dans `node_modules/mysql2`.
- ✚ **Fichier package-lock.json** : Ce fichier est automatiquement généré ou mis à jour pour enregistrer les versions exactes des packages installés, garantissant la reproductibilité du projet.

### 11.5.8 Installation de body-parser

Pour installer body-parser, on tape la commande suivante :

```
root@nasry-ahamadi:/home/nasry/api-mysql# npm install body-parser
```

```
root@nasry-ahamadi:/home/nasry/api-mysql# npm install body-parser
up to date, audited 82 packages in 13s
15 packages are looking for funding
  run `npm fund` for details
found 0 vulnerabilities
root@nasry-ahamadi:/home/nasry/api-mysql#
```

## 11.6 Création du fichier app.js dans le répertoire de votre projet.

### 11.6.1 Importation des modules nécessaires

Pour importer les modules, on fait comme suite :

```
const express = require('express');
const mysql = require('mysql2');
const bodyParser = require('body-parser');
```

#### Explication :

-  **express** : Framework Node.js utilisé pour créer une API ou une application web rapidement.
-  **mysql2** : Module pour interagir avec une base de données MySQL.
-  **body-parser** : Middleware permettant de parser les requêtes HTTP et d'extraire les données JSON du corps des requêtes.

### 11.6.2 Initialisation de l'application et configuration du port

```
const app = express();
const PORT = 3000;
```

#### Explication :

- **app** : Instance de l'application créée avec Express.
- **PORT** : Port sur lequel l'application sera accessible.

### 11.6.3 Middleware pour traiter les données JSON

```
app.use(bodyParser.json());
```

#### Explication :

Configure l'application pour qu'elle puisse lire et traiter les données envoyées au format JSON dans le corps des requêtes HTTP.

### 11.6.4 Configuration de la connexion MySQL

```
const db = mysql.createConnection({  
  
  host: 'localhost',  
  
  user: 'nasry',  
  
  password: 'passer',  
  
  database: 'banque'  
});
```

#### Explication :

-  Paramètres pour se connecter à une base de données MySQL.
-  Les valeurs comme host, user, password, et database doivent être ajustées en fonction de votre configuration locale.

### 11.6.5 Connexion à la base de données

```
db.connect((err) => {  
  
  if (err) {  
  
    console.error('Erreur de connexion à la base de données :', err);  
  
    process.exit(1);  
  
  }  
  
  console.log('Connecté à la base de données MySQL.');
```

#### Explication :

- ✚ Vérifie si la connexion à la base de données réussit ou échoue.
- ✚ En cas d'échec, affiche une erreur et termine le processus.

## 11.6.6 Définition des routes CRUD

### 11.6.6.1 Récupérer tous les clients

```
app.get('/clients', (req, res) => {  
  
  db.query('SELECT * FROM clients', (err, results) => {  
  
    if (err) {  
  
      return res.status(500).json({ message: 'Erreur lors de la récupération des clients.'  
});  
  
    }  
  
    res.json(results);  
  
  });  
});
```

#### Explication :

- ✚ Méthode HTTP : GET
- ✚ URL : /clients
- ✚ Fonction : Renvoie tous les clients depuis la table clients.

### 11.6.6.2 Récupérer un client par ID

```
app.get('/clients/:id', (req, res) => {  
  
  const { id } = req.params;  
  
  db.query('SELECT * FROM clients WHERE id = ?', [id], (err, results) => {  
  
    if (err || results.length === 0) {  
  
      return res.status(404).json({ message: 'Client non trouvé.' });  
  
    }  
  
    res.json(results[0]);  
  
  });  
});
```



```
});
```

**Explication :**

- ✚ Méthode HTTP : GET
- ✚ URL : /clients/:id
- ✚ Fonction : Renvoie un client spécifique basé sur l'ID fourni dans l'URL.

### 11.6.6.3 Ajouter un nouveau client

```
app.post('/clients', (req, res) => {  
  
  const { prenom, nom, numcompte, code, solde } = req.body;  
  
  if (!prenom || !nom || !numcompte || !code || solde === undefined) {  
  
    return res.status(400).json({ message: 'Tous les champs sont obligatoires.' });  
  
  }  
  
  
  const query = 'INSERT INTO clients (prenom, nom, numcompte, code, solde) VALUES  
(?, ?, ?, ?, ?)';  
  
  db.query(query, [prenom, nom, numcompte, code, solde], (err, result) => {  
  
    if (err) {  
  
      return res.status(500).json({ message: 'Erreur lors de l'ajout du client.' });  
  
    }  
  
    res.status(201).json({ id: result.insertId, prenom, nom, numcompte, code, solde });  
  
  });  
  
});
```

**Explication :**

- ✚ Méthode HTTP : POST
- ✚ URL : /clients
- ✚ Fonction : Ajoute un nouveau client à la base de données avec les données fournies dans le corps de la requête.

#### 11.6.6.4 Mise à jour un client

```
app.put('/clients/:id', (req, res) => {  
  
  const { id } = req.params;  
  
  const { prenom, nom, numcompte, code, solde } = req.body;  
  
  if (!prenom || !nom || !numcompte || !code || solde === undefined) {  
    return res.status(400).json({ message: 'Tous les champs sont obligatoires.' });  
  }  
  
  const query = 'UPDATE clients SET prenom = ?, nom = ?, numcompte = ?, code = ?,  
solde = ? WHERE id = ?';  
  
  db.query(query, [prenom, nom, numcompte, code, solde, id], (err) => {  
    if (err) {  
      return res.status(500).json({ message: 'Erreur lors de la mise à jour du client.' });  
    }  
  
    res.json({ id, prenom, nom, numcompte, code, solde });  
  });  
});
```

##### Explication :

- ✚ Méthode HTTP : PUT
- ✚ URL : /clients/:id
- ✚ Fonction : Met à jour les informations d'un client existant en fonction de son ID.

#### 11.6.6.5 Supprimer un client

```
app.delete('/clients/:id', (req, res) => {  
  
  const { id } = req.params;
```

```

db.query('DELETE FROM clients WHERE id = ?', [id], (err) => {

  if (err) {

    return res.status(500).json({ message: 'Erreur lors de la suppression du client.' });

  }

  res.status(204).send();

});
});

```

#### Explication :

- ✚ Méthode HTTP : DELETE
- ✚ URL : /clients/:id
- ✚ Fonction : Supprime un client de la base de données en fonction de son ID.

### 11.6.7 Démarrage du serveur

```

app.listen(PORT, () => {

  console.log(`API démarrée sur http://localhost:${PORT}`);

});

```

#### Explication :

Démarre le serveur Express et le rend accessible à l'adresse spécifiée (http://localhost:3000).

## 11.7 Test de l'api avec le client curl

### 11.7.1 Affichage des clients

```
root@nasry-ahamadi:/home/nasry# curl -X GET http://localhost:3000/clients
```

```

root@nasry-ahamadi:/home/nasry# curl -X GET http://localhost:3000/clients
[{"id":1,"prenom":"Macky","nom":"SALL","numcompte":"1001","code":"1111","solde":"3000000"}]root@nasry-ahamadi:/home/nasry#

```

### 11.7.2 Récupérer un client par ID :

```
root@nasry-ahamadi:/home/nasry# curl -X GET http://localhost:3000/clients/1
```

```
root@nasry-ahamadi:/home/nasry# curl -X GET http://localhost:3000/clients/1
{"id":1,"prenom":"Macky","nom":"SALL","numcompte":"1001","code":"1111","solde":"3000000"}root@nasry-ahamadi:/home/nasry#
```

### 11.7.3 Insertion d'un client

```
root@nasry-ahamadi:/home/nasry# curl -X POST http://localhost:3000/clients -H "Content-Type: application/json" -d '{"prenom": "Diomaye","nom":"FAYE","numcompte": "1002","code": "2222","solde": 450000}'
```

```
root@nasry-ahamadi:/home/nasry# curl -X POST http://localhost:3000/clients -H "Content-Type: application/json" -d '{"prenom": "Diomaye","nom":"FAYE","numcompte": "1002","code": "2222","solde": 450000}'
{"id":2,"prenom":"Diomaye","nom":"FAYE","numcompte":"1002","code":"2222","solde":450000}root@nasry-ahamadi:/home/nasry#
```

### 11.7.4 Modification d'un client

```
root@nasry-ahamadi:/home/nasry# curl -X PUT http://localhost:3000/clients/1 -H "Content-Type: application/json" -d '{"prenom": "Ousmane","nom":"SONKO","numcompte": "1002","code": "2222","solde": 650000}'
```

```
root@nasry-ahamadi:/home/nasry# curl -X PUT http://localhost:3000/clients/1 -H "Content-Type: application/json" -d '{"prenom": "Ousmane","nom":"SONKO","numcompte": "1002","code": "2222","solde": 650000}'
{"id":"1","prenom":"Ousmane","nom":"SONKO","numcompte":"1002","code":"2222","solde":650000}root@nasry-ahamadi:/home/nasry#
```

Vérification dans la base de données.

```
mysql> select * from clients;
```

```
mysql> select * from clients;
+----+-----+-----+-----+-----+-----+
| id | prenom | nom   | numcompte | code | solde |
+----+-----+-----+-----+-----+-----+
| 1  | Ousmane | SONKO | 1002      | 2222 | 650000 |
| 2  | Diomaye | FAYE  | 1002      | 2222 | 450000 |
+----+-----+-----+-----+-----+-----+
2 rows in set (0,00 sec)

mysql>
```

### 11.7.5 Suppression d'un client

```
root@nasry-ahamadi:/home/nasry# curl -X DELETE http://localhost:3000/clients/1
```

```
root@nasry-ahamadi:/home/nasry# curl -X DELETE http://localhost:3000/clients/1
root@nasry-ahamadi:/home/nasry#
```

```
mysql> select * from clients;
```

```
mysql> select * from clients;
+-----+-----+-----+-----+-----+-----+
| id | prenom | nom | numcompte | code | solde |
+-----+-----+-----+-----+-----+-----+
| 1 | Ousmane | SONKO | 1002 | 2222 | 650000 |
| 2 | Diomaye | FAYE | 1002 | 2222 | 450000 |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0,00 sec)

mysql>
```

```
root@nasry-ahamadi:/home/nasry# curl -X DELETE http://localhost:3000/clients/1
```

```
root@nasry-ahamadi:/home/nasry# curl -X DELETE http://localhost:3000/clients/1
root@nasry-ahamadi:/home/nasry#
```

### 11.7.6 Vérification cote base de données

✚ Avant suppression :

```
mysql> select * from clients;
+-----+-----+-----+-----+-----+-----+
| id | prenom | nom | numcompte | code | solde |
+-----+-----+-----+-----+-----+-----+
| 1 | Ousmane | SONKO | 1002 | 2222 | 650000 |
| 2 | Diomaye | FAYE | 1002 | 2222 | 450000 |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0,00 sec)

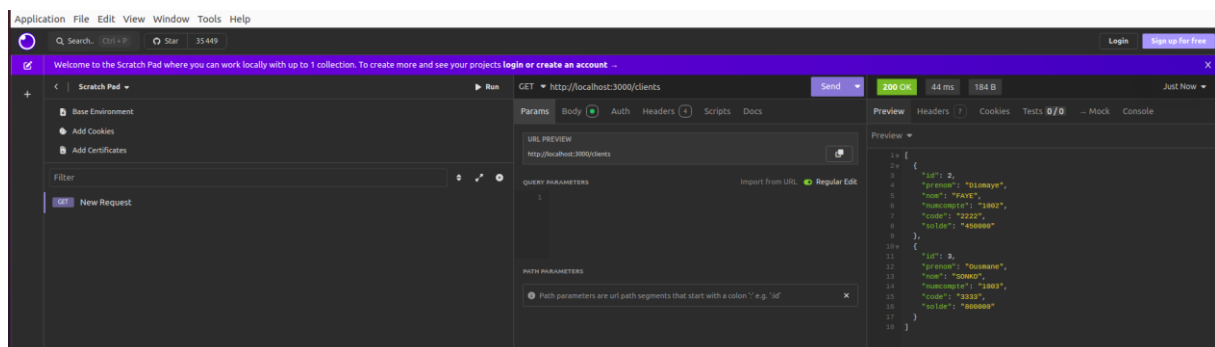
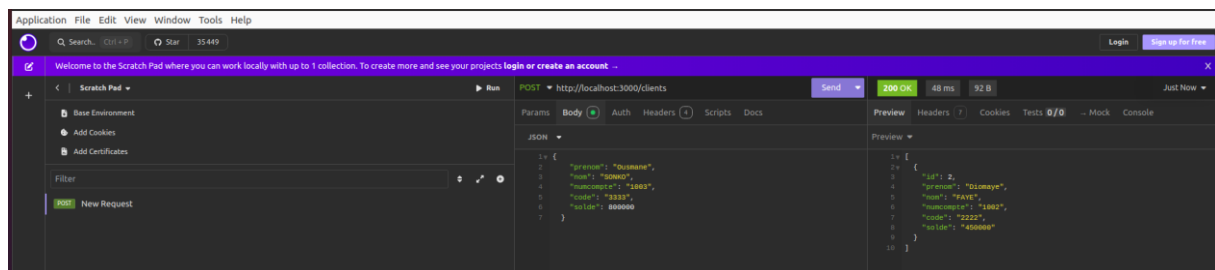
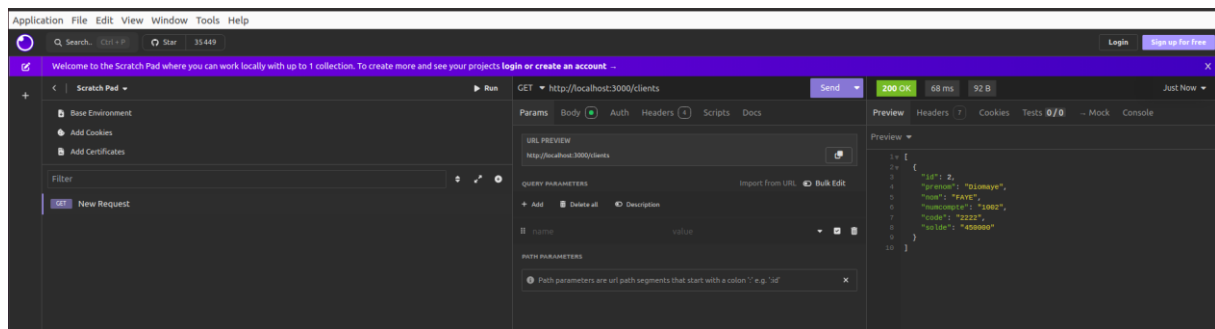
mysql>
```

✚ Après suppression :

```
mysql> select * from clients;
+-----+-----+-----+-----+-----+-----+
| id | prenom | nom | numcompte | code | solde |
+-----+-----+-----+-----+-----+-----+
| 2 | Diomaye | FAYE | 1002 | 2222 | 450000 |
+-----+-----+-----+-----+-----+-----+
1 row in set (0,00 sec)

mysql>
```

## 11.8 Utilisation du client isomia



# Chapitre 12 : Documentation d'API

## 12.1 Présentation

La documentation d'API est une partie essentielle du développement logiciel. Elle facilite la compréhension et l'utilisation des API par les développeurs, tout en améliorant la collaboration entre les équipes. Ce chapitre explore les concepts théoriques et propose une approche pratique pour générer une documentation d'API avec swagger-autogen et swagger-ui-express.

## 12.2 Qu'est-ce que la documentation d'API ?

La documentation d'API est un guide détaillé décrivant comment une API fonctionne et comment elle doit être utilisée. Elle comprend des informations sur :

- ✚ Les endpoints disponibles.
- ✚ Les méthodes HTTP utilisées (GET, POST, PUT, DELETE).
- ✚ Les paramètres requis et facultatifs.
- ✚ Les réponses attendues (codes HTTP, corps des réponses).
- ✚ Les erreurs possibles et leur gestion.
- ✚ Fournir des exemples d'utilisation.

## 12.3 Pourquoi documenter une API ?

### 12.3.1 Avantages pour les développeurs

- ✚ **Facilité d'intégration** : Une documentation claire réduit le temps nécessaire pour comprendre et utiliser l'API (utiliser facilement l'API sans explorer son code source).
- ✚ **Réduction des erreurs** : En connaissant les spécifications, les développeurs évitent les erreurs courantes.
- ✚ **Accélération du développement** : Fournit une référence centralisée pour toutes les fonctionnalités de l'API.

### 12.3.2 Avantages pour les équipes

- ✚ **Meilleure collaboration** : Partage des spécifications entre les équipes backend, frontend et QA (Quality Assurance).
- ✚ **Standardisation** : Assure une structure uniforme dans toutes les API d'un projet.
- ✚ **Scalabilité** : Une documentation bien structurée facilite l'ajout de nouvelles fonctionnalités.

## 12.4 Bonnes pratiques pour une documentation d'API

- ✚ **Utiliser un format standardisé** : Swagger/OpenAPI est largement utilisé.
- ✚ **Inclure des exemples** : Fournir des exemples pour chaque endpoint et méthode.
- ✚ **Documenter les erreurs** : Décrire les codes d'erreur et leur signification.
- ✚ **Maintenir à jour** : La documentation doit refléter fidèlement les fonctionnalités actuelles de l'API.

## 12.5 Outils populaires pour documenter les API

| Outil           | Description                                                                    |
|-----------------|--------------------------------------------------------------------------------|
| Swagger/OpenAPI | Norme standard pour documenter les API RESTful.                                |
| Postman         | Permet de créer et partager une documentation à partir de collections.         |
| Redoc           | Génère une documentation statique et interactive à partir de fichiers OpenAPI. |
| Swagger UI      | Fournit une interface graphique interactive pour explorer les API.             |

## 12.6 Génération d'une documentation d'API avec Swagger

### 12.6.1 Aperçu des outils Swagger

Swagger est un écosystème d'outils basé sur la spécification OpenAPI qui aide à concevoir, construire, documenter et consommer des API RESTful.

### 12.6.2 Les principaux outils Swagger :

- ✚ **swagger-autogen** : Génère automatiquement des fichiers de spécifications OpenAPI à partir du code source.
- ✚ **swagger-ui-express** : Fournit une interface utilisateur graphique et interactive pour explorer les API.
- ✚ **Swagger Editor** : Permet de créer et modifier manuellement les spécifications Swagger (<https://editor.swagger.io/>).

## 12.7 Mise en œuvre

### 12.7.1 Installer les modules nécessaires

Ajoutez les dépendances Swagger à votre projet Node.js :



```
root@nasry-ahamadi:/home/nasry/api-mysql# npm install swagger-autogen swagger-ui-express
```

```
root@nasry-ahamadi:/home/nasry/api-mysql# npm install swagger-autogen swagger-ui-express
up to date, audited 99 packages in 1s
16 packages are looking for funding
  run `npm fund` for details
found 0 vulnerabilities
root@nasry-ahamadi:/home/nasry/api-mysql#
```

## 12.7.2 Préparation des fichiers

Création des fichiers nécessaires :

- ✚ **app.js** : Le fichier principal de l'application qui est déjà créé en haut.
- ✚ **swagger.js** : Fichier pour générer automatiquement la documentation Swagger.

## 12.7.3 Génération de la spécification Swagger

### ✚ Création du fichier swagger.js

Ce fichier génère automatiquement le fichier Swagger à partir des routes existantes.

```
root@nasry-ahamadi:/home/nasry/api-mysql# nano swagger.js
```

```
root@nasry-ahamadi:/home/nasry/api-mysql# nano swagger.js
root@nasry-ahamadi:/home/nasry/api-mysql#
```

```
const swaggerAutogen = require('swagger-autogen')();

const doc = {
  info: {
    title: 'API Documentation',
    description: 'Documentation générée automatiquement avec Swagger.',
  },
  host: 'localhost:3000',
  schemes: ['http'],
};

const outputFile = './swagger-output.json'; // Fichier de sortie
```

```
const endpointsFiles = ['./app.js']; // Fichier contenant vos routes
```

```
swaggerAutogen(outputFile, endpointsFiles).then(() => {  
  console.log('Documentation Swagger générée avec succès.');
```

```
});
```

```
GNU nano 4.8                                swagger.js  
const swaggerAutogen = require('swagger-autogen')();  
  
const doc = {  
  info: {  
    title: 'API Documentation',  
    description: 'Documentation générée automatiquement avec Swagger.',  
  },  
  host: 'localhost:3000',  
  schemes: ['http'],  
};  
  
const outputFile = './swagger-output.json'; // Fichier de sortie  
const endpointsFiles = ['./app.js']; // Fichier contenant vos routes  
  
swaggerAutogen(outputFile, endpointsFiles).then(() => {  
  console.log('Documentation Swagger générée avec succès.');
```

### Explications :

- ✚ **doc** : Définit les métadonnées de l'API comme le titre, la description, l'hôte, et les schémas (HTTP/HTTPS).
- ✚ **outputFile** : Spécifie où sera enregistré le fichier JSON contenant la documentation Swagger. **endpointsFiles** : Liste les fichiers à analyser pour détecter les routes.

### Lancement de la génération

Exécutez la commande suivante pour générer le fichier swagger-output.json :

```
root@nasry-ahamadi:/home/nasry/api-mysql# node swagger.js
```

```
root@nasry-ahamadi:/home/nasry/api-mysql# node swagger.js  
Swagger-autogen: Success ✓  
Documentation Swagger générée avec succès.  
root@nasry-ahamadi:/home/nasry/api-mysql#
```

## 12.7.4 Intégration du Swagger UI dans l'application

- ✚ **Ajout de Swagger UI dans app.js**

Ajoutez Swagger UI pour rendre la documentation accessible via une interface graphique.

```
root@nasry-ahamadi:/home/nasry/api-mysql# nano app.js
```

```
root@nasry-ahamadi:/home/nasry/api-mysql# nano app.js
root@nasry-ahamadi:/home/nasry/api-mysql#
```

```
const swaggerUi = require('swagger-ui-express');

const swaggerDocument = require('./swagger-output.json');
```

```
const swaggerUi = require('swagger-ui-express');
const swaggerDocument = require('./swagger-output.json');
```

```
// Middleware pour Swagger
```

```
app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerDocument));
```

```
// Middleware pour Swagger
app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerDocument));
```

### Teste de la documentation

Démarrage de notre application(api) :

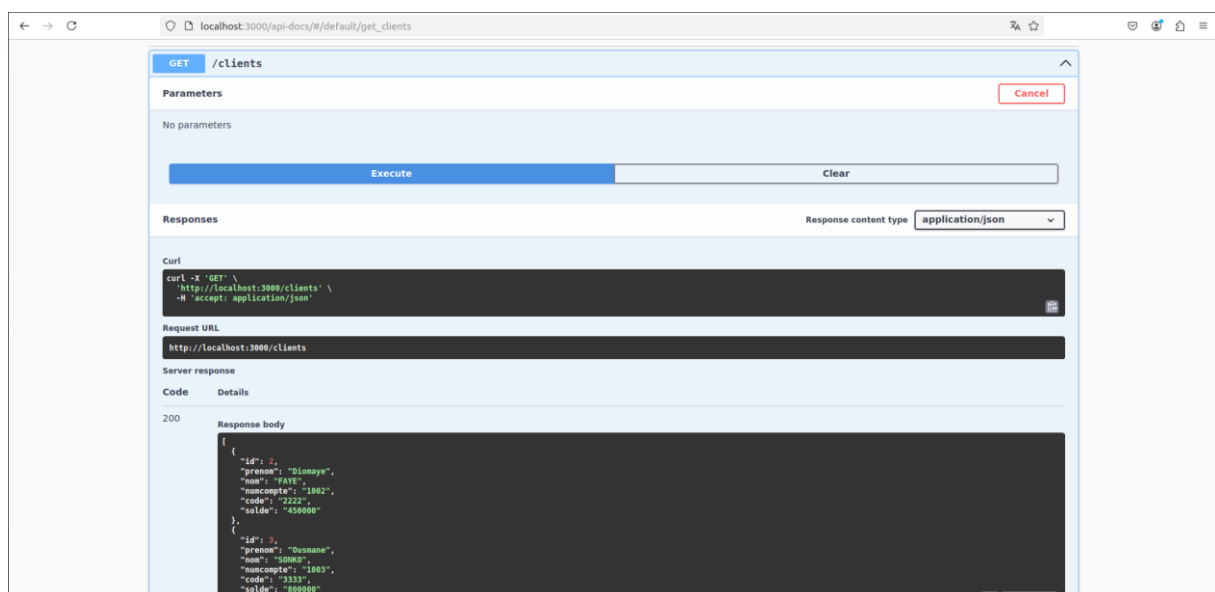
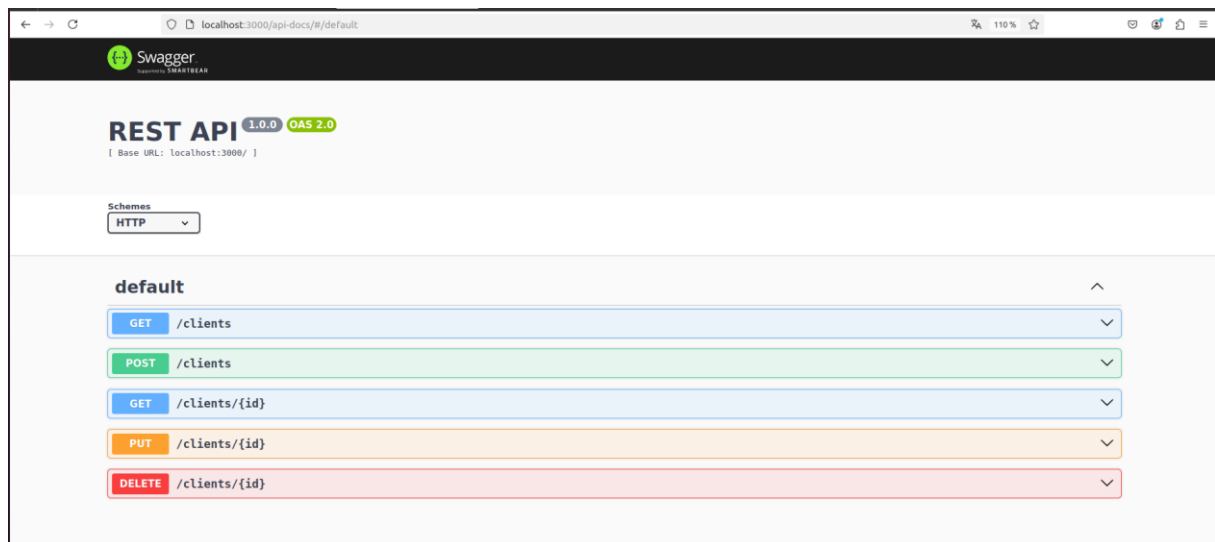
```
root@nasry-ahamadi:/home/nasry/api-mysql# node app.js
```

```
root@nasry-ahamadi:/home/nasry/api-mysql# node app.js
API démarrée sur http://localhost:3000
Connecté à la base de données MySQL.
```

On ouvre notre navigateur et on accède à l'url suivante :

**<http://localhost:3000/api-docs/>**

Nous verrons une interface Swagger interactive listant tous vos endpoints. Nous pouvons tester les endpoints directement depuis cette interface.



# Chapitre 13 : Sécurité des API

## 13.1 Présentation

La sécurité des API est essentielle pour protéger les données des utilisateurs, prévenir les cyberattaques et garantir la confidentialité des communications. Ce chapitre explore les principes fondamentaux de la sécurité, les menaces courantes et les solutions pratiques pour sécuriser une API développée avec Node.js.

## 13.2 Concepts fondamentaux de la sécurité des API

### 13.2.1 Définition et enjeux de la sécurité des API

Une API est une interface permettant à des applications d'échanger des données. Protéger une API signifie :

- ✚ Assurer la confidentialité des données transmises.
- ✚ Vérifier l'authenticité et l'autorisation des utilisateurs.
- ✚ Prévenir les attaques et exploitations malveillantes.

## 13.3 Principes fondamentaux de la sécurité des API

- ✚ Principe du moindre privilège : Un utilisateur doit avoir uniquement les permissions nécessaires.
- ✚ Zéro confiance (Zero Trust Security) : Toujours vérifier l'identité des utilisateurs et des services.
- ✚ Authentification et Autorisation : Vérifier qui accède et à quoi.
- ✚ Chiffrement et intégrité : Protéger les données avec HTTPS/TLS et des signatures numériques.

## 13.4 Pourquoi sécuriser une API ?

Une API exposée sur Internet peut être vulnérable aux attaques si elle n'est pas bien sécurisée. Il est essentiel de protéger :

- ✚ Les données sensibles des utilisateurs.
- ✚ L'accès aux ressources (authentification et autorisation).
- ✚ Les performances du serveur en évitant les abus.

## 13.5 Principaux risques de sécurité des API

- ✚ Injection SQL & NoSQL : Manipulation de la base de données via des entrées malveillantes.
- ✚ Attaques XSS (Cross-Site Scripting) : Exécution de scripts malveillants dans le navigateur.
- ✚ CSRF (Cross-Site Request Forgery) : Exploitation d'une session active pour exécuter des actions frauduleuses.
- ✚ Attaques DDoS (Déni de service) : Saturation du serveur avec un grand nombre de requêtes.
- ✚ Vol d'identité et accès non autorisé : Absence de protection d'authentification et d'autorisation.

## 13.6 Menaces courantes sur les API

### 13.6.1 Injection SQL et NoSQL

L'injection SQL et l'injection NoSQL sont deux types d'attaques qui visent les bases de données en exploitant des failles dans les requêtes mal sécurisées. Ces attaques permettent à un attaquant d'exécuter des requêtes malveillantes pour manipuler ou exfiltrer des données.

#### 13.6.2 Injection SQL (SQLi)

L'injection SQL (SQL Injection) est une attaque qui cible les bases de données relationnelles utilisant SQL (comme MySQL, PostgreSQL, SQL Server, etc.). Elle se produit lorsque des entrées utilisateur sont insérées directement dans une requête SQL sans être correctement sécurisées.

#### 13.6.3 Injection NoSQL

L'injection NoSQL est une attaque similaire, mais ciblant les bases de données NoSQL comme MongoDB, Firebase, CouchDB, etc. Ces bases ne reposent pas sur SQL mais utilisent des documents JSON, ce qui permet des attaques différentes.

- ✚ Problème : Un attaquant injecte des requêtes SQL ou NoSQL malveillantes.
- ✚ Solution : Utiliser des requêtes préparées et des ORM.

#### 13.6.4 Vol d'identité (Attaque JWT)

- ✚ Problème : Un attaquant réutilise un token JWT volé.
- ✚ Solution :
  - Configurer une expiration courte des tokens.

- Mettre en place un refresh token sécurisé.
- Signer les JWT avec une clé secrète robuste.

### 13.6.5 Cross-Site Scripting (XSS)

Le Cross-Site Scripting (XSS) est une vulnérabilité de sécurité qui permet à un attaquant d'injecter du code JavaScript malveillant dans une page web consultée par un utilisateur. Ce type d'attaque peut être utilisé pour voler des cookies, rediriger des utilisateurs vers des sites malveillants, ou exécuter des actions à leur insu.

- ✚ Problème : Un attaquant injecte du JavaScript malveillant via l'API.
- ✚ Solution : Filtrer et encoder les entrées utilisateur.

### 13.6.6 Attaques CSRF (Cross-Site Request Forgery)

Le Cross-Site Request Forgery (CSRF) est une attaque où un attaquant force un utilisateur authentifié à exécuter des actions non autorisées sur un site web où il est connecté. Contrairement au XSS, qui injecte du JavaScript malveillant, le CSRF exploite la confiance d'un site dans l'authentification de l'utilisateur.

### 13.6.7 Comment fonctionne une attaque CSRF ?

- ✚ L'utilisateur est connecté à un site web légitime (ex. une banque).
- ✚ L'attaquant envoie un lien malveillant (via email, forum, site piégé, etc.).
- ✚ L'utilisateur clique sur le lien (sans savoir qu'il exécute une action sur le site légitime).
- ✚ Le site légitime exécute la requête car l'utilisateur est authentifié (via cookies de session).

Problème : Un utilisateur authentifié exécute une requête non désirée.

Solution : Utiliser des tokens CSRF et valider l'origine des requêtes.

### 13.6.8 Comment se protéger contre CSRF ?

#### 13.6.8.1 Utiliser un Token CSRF (Anti-CSRF Token)

Générer un jeton unique pour chaque requête sensible.

#### 13.6.8.2 Vérifier l'Origine et le Referer

Refuser les requêtes si Origin ou Referer sont absents ou suspects :

### 13.6.8.3 Utiliser des Cookies SameSite=Strict

Empêcher les requêtes venant d'un autre site d'envoyer des cookies :

### 13.6.9 Attaque par Force Brute

L'attaque par force brute est une méthode où un attaquant tente systématiquement toutes les combinaisons possibles d'identifiants (mots de passe, codes PIN, clés de chiffrement, etc.) jusqu'à trouver la bonne.

Cette attaque est souvent utilisée pour :

- ✚ Craquer des mots de passe
- ✚ Accéder à des comptes utilisateurs

**Problème :** Tentatives répétées de connexion pour deviner un mot de passe.

**Solution :** Limiter le nombre de requêtes avec rate-limiting.

### 13.6.10 Types d'attaques par Force Brute

#### 13.6.10.1 Attaque par dictionnaire

L'attaquant utilise une liste de mots de passe courants au lieu de tester toutes les combinaisons possibles.

#### 13.6.10.2 Attaque exhaustive (Brute Force pure)

L'attaquant essaie toutes les combinaisons possibles de caractères.

#### 13.6.10.3 Attaque par force brute sur le hachage

L'attaquant essaie différentes entrées jusqu'à ce que le hachage corresponde à celui d'un mot de passe enregistré.

### 13.6.11 Comment se protéger contre les attaques par Force Brute ?

#### 13.6.11.1 Limiter le nombre de tentatives

Bloquer un compte après plusieurs tentatives incorrectes.

#### 13.6.11.2 Utiliser une authentification forte

Exiger des mots de passe robustes :

- ✚ Minimum 12 caractères
- ✚ Mélanger lettres, chiffres et symboles



- ✚ Ne pas utiliser des mots courants

### **13.6.11.3 Implémenter une authentification multi-facteurs (MFA)**

Même si l'attaquant trouve le mot de passe, il lui faut encore un code unique (OTP, Google Authenticator, etc.).

### **13.6.11.4 Hacher et saler les mots de passe**

Ne jamais stocker un mot de passe en clair !

### **13.6.11.5 Surveiller les tentatives de connexion**

Détecter et bloquer les adresses IP suspectes (via un pare-feu applicatif WAF).

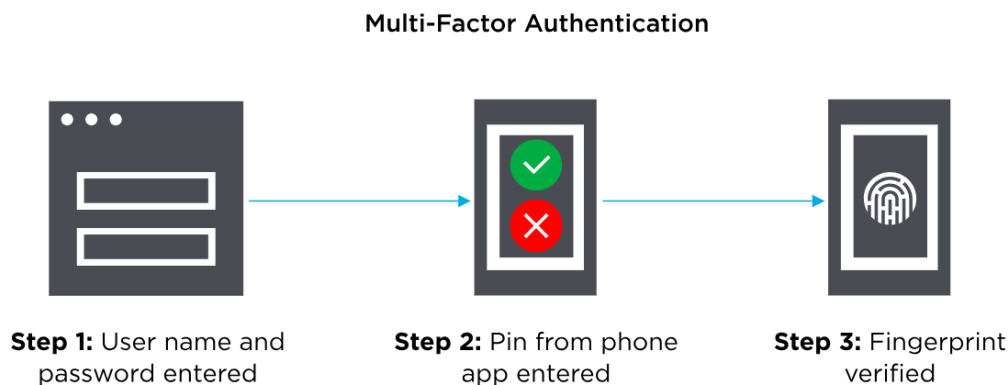
## **13.7 Authentification des utilisateurs dans une API**

### **13.7.1 Qu'est-ce que l'authentification et l'autorisation ?**

- ✚ Authentification : Vérifier qui est l'utilisateur (exemple : connexion avec un identifiant et mot de passe).
- ✚ Autorisation : Définir ce qu'un utilisateur a le droit de faire (exemple : accès administrateur).

### **13.7.2 Authentification multi-facteurs (MFA/2FA)**

L'authentification multifacteur (MFA) est une méthode d'authentification dans laquelle l'utilisateur doit fournir au minimum deux facteurs de vérification pour accéder à une ressource de type application, compte en ligne ou VPN. La MFA est une composante essentielle à la base de toute politique de gestion des accès et des identités (IAM, pour Identity and Access Management) se voulant solide. Au lieu de se contenter d'un nom d'utilisateur et d'un mot de passe, la MFA exige un ou plusieurs facteurs de vérification supplémentaires, ce qui réduit la probabilité qu'une cyberattaque puisse réussir.



### 13.7.3 Pourquoi la MFA est-elle importante ?

Le principal avantage de la MFA est d'améliorer la sécurité de votre entreprise en exigeant à vos utilisateurs de fournir plus qu'un nom d'utilisateur et un mot de passe pour s'identifier. Les noms d'utilisateur et les mots de passe sont importants, mais ils sont vulnérables aux attaques par force brute et peuvent être volés par des tiers. Le recours à un facteur MFA de type empreinte numérique ou clé matérielle renforce la garantie que votre entreprise est protégée des cybercriminels.

### 13.7.4 Comment fonctionne la MFA ?

La MFA exige des informations de vérification supplémentaires (facteurs). L'un des facteurs MFA les plus courants que rencontrent les utilisateurs sont les mots de passe à usage unique (OTP, pour One-Time Password). Les OTP sont ces codes comportant de 4 à 8 chiffres que vous recevez souvent par e-mail, SMS ou certaines applications mobiles. Avec les OTP, un nouveau code est généré de façon périodique ou à chaque fois qu'une demande d'authentification est envoyée. Le code est généré à partir d'une valeur initiale attribuée à l'utilisateur lorsqu'il s'inscrit pour la première fois et d'un autre facteur qui peut être un compteur incrémenté ou une valeur temporelle.

**Solution :** Ajouter une vérification avec OTP via email ou Google Authenticator.

### 13.7.5 Différents types d'authentification en API

| Méthode              | Description                                            | Cas d'utilisation        |
|----------------------|--------------------------------------------------------|--------------------------|
| JWT (JSON Web Token) | Un jeton signé qui vérifie l'identité de l'utilisateur | APIs REST, microservices |

|                         |                                                                     |                               |
|-------------------------|---------------------------------------------------------------------|-------------------------------|
| OAuth2 / OpenID Connect | Authentification via un tiers<br>(Google, Facebook, GitHub)         | Applications web et mobiles   |
| Session-Based Auth      | Authentification avec<br>stockage côté serveur                      | Applications monolithiques    |
| API Key                 | Une clé secrète envoyée<br>avec chaque requête                      | Services tiers, API publiques |
| Passport.js             | Middleware supportant<br>plusieurs stratégies<br>d'authentification | Applications full-stack       |